Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

---

Fase VI · Diseño y construcción de optimizadores con IA

# Módulo 27

## Optimización geométrica y software 3D

Representaciones diferenciables, topología, simuladores, renderizado y pipelines 3D

---

Apuntes del curso

Edición de octubre de 2026

# Presentación del módulo

Un optimizador geométrico busca una forma: la distribución de material de una pieza, el perfil de una aleta, la malla de un objeto reconstruido a partir de fotografías o el nivel de detalle con que se dibuja una escena. En todos los casos aparece la misma cadena: unos parámetros generan una geometría, un simulador o un renderizador la evalúan, una pérdida resume el resultado y un gradiente, calculado con el método adjunto o con diferenciación automática, indica cómo modificar los parámetros. Este módulo estudia los eslabones de esa cadena y los ensambla en casos completos. Empieza por las representaciones 3D y su diferenciabilidad; sigue con la optimización de topología por el método SIMP, que es el ejemplo clásico de diseño guiado por sensibilidades adjuntas; continúa con los simuladores diferenciables y los modelos sustitutos neuronales, y con el renderizado diferenciable, que convierte la reconstrucción 3D (incluidos NeRF y el *Gaussian splatting*) en un problema de descenso de gradiente. Los dos últimos temas pasan del laboratorio a la producción: la optimización de los pipelines de software 3D y videojuegos (simplificación de mallas, niveles de detalle, compresión de texturas, empaquetado UV, automatización con Blender y presupuestos de milisegundos), y la validación de la geometría producida por modelos generativos antes de fabricarla o cargarla en un motor. Sobre productos comerciales (Autodesk Fusion, nTop, Unreal Engine) solo se usa información pública, que se cita.

## Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. describir mallas, nubes de puntos, vóxeles, funciones de distancia con signo y campos neuronales, decidir cuál conviene a un problema y calcular derivadas de magnitudes geométricas respecto a sus parámetros;
2. formular el problema de flexibilidad mínima con la interpolación SIMP, deducir sus sensibilidades por el método adjunto, justificar los filtros y la actualización por criterios de optimalidad, e implementar el caso de la viga MBB;
3. derivar a través de un simulador mediante el teorema de la función implícita, construir y validar un modelo sustituto y acotar el error que comete al optimizar sobre él;
4. explicar por qué la rasterización clásica no es diferenciable, cómo la suavizan los renderizadores diferenciables y cómo NeRF y el *Gaussian splatting* se entrenan por descenso de gradiente;
5. plantear como problemas de optimización la simplificación de mallas por error cuádrico, la selección de niveles de detalle con un presupuesto de tiempo por fotograma, la compresión de texturas y el empaquetado UV;
6. validar automáticamente la geometría generada por IA (variedad, orientación, volumen, voladizos, espesor) y diseñar un bucle de generación y verificación robusto.

## Requisitos previos

De la Fase 0 se usan el producto vectorial y el producto mixto (0.7, §1.4), los multiplicadores de Lagrange (0.7, §5.4) y la independencia de sucesos (0.8, §4.4). Del módulo 3 se usan las matrices simétricas y el teorema espectral (tema 3) y el análisis de componentes principales (§5.4); del módulo 4, la regla de la cadena y el jacobiano (tema 2), la convexidad (tema 5) y la comprobación de gradientes por diferencias finitas (tema 6); del módulo 7, el descenso de gradiente (tema 2) y Adam (§5.3); del módulo 8, los mínimos cuadrados y las ecuaciones normales (§2.2) y la validación (§1.4 y §6.5); del módulo 9, el perceptrón multicapa (tema 1); del módulo 10, el modo inverso (tema 3), los productos vector-jacobiano (§4.3), el recálculo por puntos de control (§5.2) y los adjuntos como multiplicadores de Lagrange (§5.4), que son la base del método adjunto de este módulo; de los módulos 12 y 15, la tokenización y la generación autorregresiva, que reaparecen en los generadores de mallas; del módulo 17, el modelo *roofline* (§2.2), y del módulo 21, la sobreoptimización de una recompensa aprendida (§6.4), que tiene un análogo exacto en la optimización sobre un sustituto o contra un validador.

## Organización

| Tema | Contenido | Fuentes principales |
|---|---|---|
| 1 | Representaciones 3D y su diferenciabilidad | [4], [30], [27], [29] |
| 2 | Optimización de forma y de topología | [10], [13], [36], [1] |
| 3 | Simuladores diferenciables y modelos sustitutos | [12], [7], [16], [25] |
| 4 | Renderizado diferenciable y reconstrucción 3D | [26], [28], [20], [2] |
| 5 | Optimización de pipelines de software 3D y videojuegos | [17], [19], [44], [8] |
| 6 | IA generativa para 3D y validación de la geometría | [32], [35], [2], [22] |

Cada tema combina teoría formal con al menos un caso de estudio desarrollado con datos numéricos (sintéticos, pero con unidades y órdenes de magnitud realistas), una tabla de tecnologías y una observación sobre los errores más frecuentes en la práctica. Termina con una lista de lecturas y con ejercicios graduados en tres niveles: ⋆ comprensión, ⋆⋆ formulación y cálculo a mano, y ⋆⋆⋆ demostración o diseño razonado. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas están en el apéndice A. El cuaderno `M27-practica.ipynb` implementa los seis casos de estudio en numpy, scipy, scikit-learn y torch, y se ejecuta en una CPU en menos de un minuto.

## Sobre las fuentes

El texto es de redacción propia. Las representaciones del tema 1 siguen la clase «Classical 3D Geometry Representations» del curso CS348n de Stanford [4] y su tarea 2 [5], el artículo de DeepSDF [30] y el de *marching cubes* [27]. La optimización de topología del tema 2 sigue el capítulo 1 del libro de Bendsøe y Sigmund [10], el artículo fundacional de Bendsøe [14], los códigos de 99 y 88 líneas [36, 13], el estudio de inestabilidades de Sigmund y Petersson [37] y el análisis de filtros de Bourdin [15]; el método adjunto se presenta como en las notas de cálculo

matricial del MIT [1] y en las de S. G. Johnson para 18.335J [7]. El tema 3 sigue los capítulos de modelos sustitutos de Kochenderfer y Wheeler [12] y los artículos originales de DiffTaichi [16], PINN [33], FNO [25] y MeshGraphNets [31]; la física de la aleta procede del capítulo 3 de Incropera *et al.* [11]. El tema 4 sigue SoftRas [26], PyTorch3D [34], nvdiffrast [21], NeRF [28], el *Gaussian splatting* [20] y las tareas 2 y 3 del curso 16-825 de Carnegie Mellon [2]. El tema 5 sigue a Garland y Heckbert [17], las clases de simplificación de UCSD [8] y Princeton [3], la presentación de Nanite [19] y la documentación de Microsoft [44] y de Blender [40]. El tema 6 sigue DreamFusion [32], Shap-E [18], MeshGPT [35], la tarea 4 de 16-825 [2] y el filtro de fabricación aditiva de Langelaar [22]. Los casos industriales (soporte de asiento de General Motors, Autodesk Fusion, nTop) se describen solo con la información que sus autores han publicado [38, 39, 45].

## Notación

| Símbolo | Significado |
|---|---|
| $\theta$ | parámetros de diseño (forma, densidades, coeficientes) |
| $(V, F)$, $E$ | malla: vértices, caras (triángulos) y aristas; $\chi = \lvert V\rvert - \lvert E\rvert + \lvert F\rvert$ es la característica de Euler |
| $f : \mathbb{R}^3 \to \mathbb{R}$ | función implícita o de distancia con signo (SDF): $f < 0$ dentro, $f = 0$ en la superficie |
| $x_e \in [0,1]$, $\tilde{x}_e$ | densidad de diseño del elemento $e$ y densidad física (filtrada) |
| $p$, $E_0$, $E_{\min}$ | exponente de penalización SIMP, módulo del material sólido y módulo mínimo |
| $K$, $U$, $F$ | matriz de rigidez, desplazamientos y cargas, con $KU = F$ |
| $c = F^\top U$ | flexibilidad (*compliance*): trabajo de las cargas; cuanto menor, más rígida la pieza |
| $k_0$, $u_e$ | matriz de rigidez del elemento con $E = 1$ y desplazamientos de sus nodos |
| $H_{ei}$, $r_{\min}$ | pesos del filtro y radio del filtro |
| $R(u, \theta) = 0$ | ecuaciones discretas de un simulador; $\lambda$ es el vector adjunto |
| $\hat{s}$ | modelo sustituto de una respuesta simulada $s$ |
| $\sigma(\cdot)$, sig | densidad volumétrica (NeRF) y función sigmoide $\mathrm{sig}(z) = 1/(1 + e^{-z})$ |
| $\alpha_i$, $T_i$, $w_i$ | opacidad, transmitancia y peso de la muestra $i$ de un rayo |
| $Q$ | cuádrica de error de un vértice (matriz simétrica $4 \times 4$) |
| $\bar{v} = (v, 1)$ | coordenadas homogéneas de un punto $v \in \mathbb{R}^3$ |

# Índice

**Presentación del módulo** **ii**

**1 Representaciones 3D y su diferenciabilidad** **1**

1.1 El optimizador geométrico . . . 1
1.2 Representaciones explícitas: mallas, nubes de puntos y vóxeles . . . 2
1.3 Representaciones implícitas y funciones de distancia con signo . . . 3
1.4 Campos neuronales . . . 4
1.5 Diferenciabilidad y elección de la representación . . . 4
1.6 Caso de estudio: ajuste de una esfera a un escaneo parcial . . . 5
Ejercicios del tema 1 . . . 6

**2 Optimización de forma y de topología** **8**

2.1 El problema de flexibilidad mínima . . . 8
2.2 La interpolación SIMP . . . 8
2.3 Sensibilidades por el método adjunto . . . 9
2.4 Actualización por criterios de optimalidad . . . 10
2.5 Inestabilidades numéricas y filtros . . . 11
2.6 Caso de estudio: la viga MBB . . . 11
2.7 Fabricación aditiva y diseño generativo . . . 12
Ejercicios del tema 2 . . . 14

**3 Simuladores diferenciables y modelos sustitutos** **15**

3.1 Derivar a través de un simulador . . . 15
3.2 Diferenciación automática de simuladores . . . 16
3.3 Modelos sustitutos . . . 16
3.4 Caso de estudio: el perfil de una aleta disipadora . . . 17
Ejercicios del tema 3 . . . 19

**4 Renderizado diferenciable y reconstrucción 3D** **20**

4.1 Formación de la imagen y rasterización . . . 20
4.2 Rasterización suave . . . 21
4.3 Renderizado volumétrico y NeRF . . . 21
4.4 *Gaussian splatting* . . . 22
4.5 Caso de estudio: ajuste de una forma a siluetas . . . 22

Ejercicios del tema 4 . . . 24

**5 Optimización de pipelines de software 3D y videojuegos 26**
5.1 El presupuesto de rendimiento como restricción . . . 26
5.2 Simplificación de mallas por error cuádrico . . . 26
5.3 Niveles de detalle y error en pantalla . . . 27
5.4 Selección de niveles de detalle con presupuesto . . . 28
5.5 Compresión de texturas . . . 29
5.6 Empaquetado UV . . . 29
5.7 Contenido procedural y automatización con Blender . . . 29
Ejercicios del tema 5 . . . 31

**6 IA generativa para 3D y validación de la geometría 33**
6.1 Tres maneras de generar geometría . . . 33
6.2 Validación de la geometría . . . 34
6.3 Generar y verificar . . . 35
6.4 Caso de estudio: validación de mallas generadas . . . 36
Ejercicios del tema 6 . . . 37

**A Soluciones de los ejercicios 39**
A.1 Tema 1 . . . 39
A.2 Tema 2 . . . 42
A.3 Tema 3 . . . 46
A.4 Tema 4 . . . 50
A.5 Tema 5 . . . 53
A.6 Tema 6 . . . 57

**Bibliografía 61**

Tema 1

# Representaciones 3D y su diferenciabilidad

Antes de optimizar una forma hay que decidir cómo se describe con números. La elección no es neutral: determina qué cambios de forma son posibles (por ejemplo, si pueden aparecer agujeros), cuánta memoria hace falta y, sobre todo, si la magnitud que se quiere optimizar es una función derivable de los parámetros. Este tema presenta el esquema común de todos los optimizadores del módulo y repasa las representaciones clásicas y neuronales siguiendo la clase «Classical 3D Geometry Representations» del curso CS348n de Stanford [4], el artículo de DeepSDF [30], el de *marching cubes* [27] y el de las codificaciones con tablas *hash* de Müller *et al.* [29].

## 1.1 El optimizador geométrico

**Definición 1.1 (Optimizador geométrico).** Sea $\Theta \subseteq \mathbb{R}^n$ un conjunto de parámetros de diseño. Un *optimizador geométrico* resuelve

$$\min_{\theta \in \Theta} \; J(\theta) = \mathcal{L}\big(S(G(\theta))\big) \quad \text{sujeto a} \quad g_j(\theta) \le 0, \; j = 1, \dots, m,$$

donde $G$ construye una geometría a partir de $\theta$, $S$ la evalúa (un simulador físico, que devuelve campos como desplazamientos o temperaturas, o un renderizador, que devuelve imágenes), $\mathcal{L}$ resume la evaluación en un número (flexibilidad, calor disipado, error frente a fotografías) y las $g_j$ expresan restricciones de fabricación, de volumen o de presupuesto.

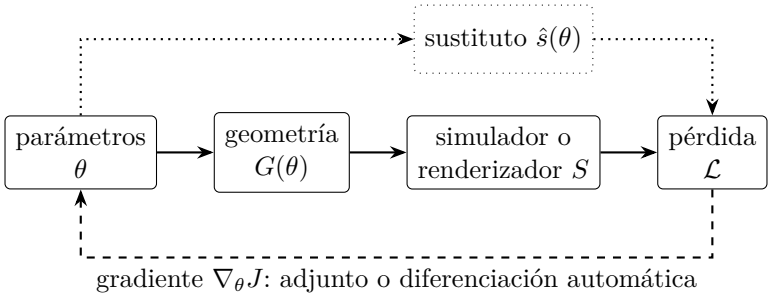



Figura 1.1: Esquema común de los optimizadores del módulo. La rama punteada sustituye el simulador por un modelo aprendido (tema 3).

El gradiente $\nabla_\theta J$ puede obtenerse de tres maneras. Las diferencias finitas (módulo 4, tema 6) necesitan $n + 1$ evaluaciones de $S$ y son inviables cuando $n$ es del orden de miles. El modo inverso de la diferenciación automática o, cuando $S$ resuelve un sistema de ecuaciones, el método adjunto (módulo 10, §5.4, y tema 2) calculan el gradiente completo con un coste

comparable a una o dos evaluaciones, independientemente de $n$. Y cuando $S$ no es derivable o es demasiado caro, se aprende un sustituto $\hat{s}$ con unas decenas de evaluaciones y se optimiza sobre él (tema 3). En los tres casos, la primera condición es que la geometría $G(\theta)$ dependa de $\theta$ de forma suave, y eso depende de la representación.

## 1.2 Representaciones explícitas: mallas, nubes de puntos y vóxeles

**Definición 1.2 (Malla triangular).** Una *malla triangular* es un par $(V, F)$ formado por una lista de vértices $v_1, \dots, v_{|V|} \in \mathbb{R}^3$ y una lista de caras, cada una un triplete ordenado $(a, b, c)$ de índices de vértices. El orden de los índices define la orientación de la cara y su normal $n_f = (v_b - v_a) \times (v_c - v_a)/\|(v_b - v_a) \times (v_c - v_a)\|$. El conjunto $E$ de aristas está formado por los pares no ordenados de vértices consecutivos de alguna cara. La malla es *cerrada* si cada arista pertenece exactamente a dos caras, y está *orientada de forma coherente* si cada arista compartida se recorre en sentidos opuestos en sus dos caras.

**Proposición 1.3.** *En una malla triangular cerrada,* $2|E| = 3|F|$. *En particular, el número de caras es par.*

*Demostración.* Se cuentan de dos maneras los pares (arista, cara que la contiene). Cada cara tiene tres aristas, luego hay $3|F|$ pares; cada arista está en exactamente dos caras, luego hay $2|E|$ pares. Como $3|F| = 2|E|$ es par, $|F|$ es par. ∎

La característica de Euler $\chi = |V| - |E| + |F|$ de una malla cerrada y conexa que no se corta a sí misma depende solo de su topología: $\chi = 2 - 2g$, donde $g$ es el género o número de «asas» (0 para una esfera, 1 para un toro). Este resultado de topología se admite sin demostración.

**Corolario 1.4.** *Si una malla triangular cerrada tiene* $\chi = 2$ *(topología de esfera), entonces* $|F| = 2|V| - 4$ *y* $|E| = 3|V| - 6$.

*Demostración.* De la proposición 1.3, $|E| = \frac{3}{2}|F|$. Sustituyendo en $|V| - |E| + |F| = 2$ se obtiene $|V| - \frac{1}{2}|F| = 2$, es decir, $|F| = 2|V| - 4$, y entonces $|E| = \frac{3}{2}(2|V| - 4) = 3|V| - 6$. ∎

El corolario da una regla de memoria útil: una malla cerrada tiene aproximadamente el doble de caras que de vértices. Con posiciones en coma flotante de 32 bits (12 bytes por vértice) e índices enteros de 32 bits (12 bytes por cara), cuesta unos 36 bytes por vértice.

**Definición 1.5 (Nube de puntos y distancia de Chamfer).** Una *nube de puntos* es un conjunto finito $A \subset \mathbb{R}^3$, opcionalmente con normales. La *distancia de Chamfer* entre dos nubes $A$ y $B$ es

$$\mathrm{CD}(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|a - b\|^2.$$

La distancia de Chamfer es derivable respecto a las coordenadas de los puntos salvo en los empates del mínimo: el gradiente respecto a $a$ empuja $a$ hacia su vecino más próximo en $B$. Es la pérdida habitual para ajustar nubes de puntos [2]. Una *rejilla de vóxeles* de resolución $n$ guarda un valor (ocupación, densidad o distancia) en cada una de las $n^3$ celdas de una caja; es la representación más sencilla de derivar, porque cada parámetro es un valor de celda, pero su memoria crece con el cubo de la resolución.

**Proposición 1.6 (Volumen de una malla cerrada y su gradiente).** *Sea $(V, F)$ una malla triangular cerrada, orientada de forma coherente con normales hacia fuera, que encierra la región $\Omega$. Entonces*

$$\mathrm{Vol}(\Omega) = \frac{1}{6}\sum_{(a,b,c)\in F} \det(v_a, v_b, v_c) = \frac{1}{6}\sum_{(a,b,c)\in F} v_a \cdot (v_b \times v_c),$$

*y la derivada respecto a la posición de un vértice $v_i$ es $\nabla_{v_i}\mathrm{Vol} = \frac{1}{6}\sum v_b \times v_c$, sumando sobre las caras que contienen a $i$, escritas en el orden cíclico $(i, b, c)$.*

*Demostración.* Por el teorema de la divergencia aplicado al campo $\phi(x) = x/3$, cuya divergencia vale 1, $\mathrm{Vol}(\Omega) = \frac{1}{3}\int_{\partial\Omega} x \cdot n\, dA$. Sobre una cara plana $x \cdot n$ es constante e igual a $v_a \cdot n_f$, y el área por la normal es $A_f n_f = \frac{1}{2}(v_b - v_a) \times (v_c - v_a)$. Por tanto, la cara aporta $\frac{1}{3}v_a \cdot \frac{1}{2}(v_b - v_a) \times (v_c - v_a)$. Desarrollando el producto vectorial, $(v_b - v_a) \times (v_c - v_a) = v_b \times v_c - v_b \times v_a - v_a \times v_c$, y los dos últimos términos son perpendiculares a $v_a$; queda $\frac{1}{6}v_a \cdot (v_b \times v_c)$, que es el producto mixto (0.7, §1.4). El determinante es lineal en cada columna e invariante por permutaciones cíclicas, así que cada cara que contiene a $v_i$ puede escribirse como $\det(v_i, v_b, v_c)$, cuyo gradiente respecto a $v_i$ es $v_b \times v_c$. ∎

**Ejemplo 1.7.** Sea el tetraedro de vértices $v_0 = 0$, $v_1 = (a, 0, 0)$, $v_2 = (0, a, 0)$, $v_3 = (0, 0, a)$ con $a = 30$ mm y caras orientadas hacia fuera $(0, 2, 1)$, $(0, 1, 3)$, $(0, 3, 2)$ y $(1, 2, 3)$. Las tres primeras contienen el origen y aportan determinante nulo; la cuarta aporta $\det(v_1, v_2, v_3) = a^3$. Luego $\mathrm{Vol} = a^3/6 = 4500$ mm³. Para $v_3$, las caras que lo contienen, en orden cíclico, son $(3, 0, 1)$, $(3, 2, 0)$ y $(3, 1, 2)$, y $\nabla_{v_3}\mathrm{Vol} = \frac{1}{6}(v_0 \times v_1 + v_2 \times v_0 + v_1 \times v_2) = \frac{1}{6}(0, 0, a^2) = (0, 0, 150)$ mm². Es lo esperado: $\mathrm{Vol} = \frac{1}{3} \cdot \text{base} \cdot \text{altura}$ con base $a^2/2 = 450$ mm², luego $\partial\,\mathrm{Vol}/\partial(\text{altura}) = 150$ mm².

## 1.3 Representaciones implícitas y funciones de distancia con signo

**Definición 1.8 (Superficie implícita y SDF).** Una función continua $f : \mathbb{R}^3 \to \mathbb{R}$ define el sólido $\Omega = \{f \le 0\}$ y la superficie implícita $\{f = 0\}$. Se dice que $f$ es una *función de distancia con signo* (SDF) de $\Omega$ si $f(x) = -\operatorname{dist}(x, \partial\Omega)$ para $x \in \Omega$ y $f(x) = \operatorname{dist}(x, \partial\Omega)$ fuera de $\Omega$.

Por ejemplo, $f(x) = \|x - c\| - r$ es la SDF de la bola de centro $c$ y radio $r$, y en todo punto $x \neq c$ su gradiente $\nabla f = (x - c)/\|x - c\|$ tiene norma 1.

**Proposición 1.9 (Una SDF es 1-lipschitziana).** *Si $f$ es la SDF de $\Omega$, entonces $|f(x) - f(y)| \le \|x - y\|$ para todo $x, y$. En consecuencia, en los puntos en que $f$ es derivable, $\|\nabla f\| \le 1$.*

*Demostración.* Sea $d(x) = \operatorname{dist}(x, \partial\Omega)$. Si $x$ e $y$ están del mismo lado, para todo $z \in \partial\Omega$ se tiene $d(x) \le \|x - z\| \le \|x - y\| + \|y - z\|$; tomando el ínfimo en $z$, $d(x) \le \|x - y\| + d(y)$, y por simetría $|d(x) - d(y)| \le \|x - y\|$. Si están en lados opuestos, el segmento que los une corta $\partial\Omega$ en algún punto $z$ (por continuidad de $f$ y el teorema de Bolzano), y $|f(x) - f(y)| = d(x) + d(y) \le \|x - z\| + \|z - y\| = \|x - y\|$. Para la consecuencia, $|\nabla f(x) \cdot u| = \lim_{t\to 0^+} |f(x + tu) - f(x)|/t \le 1$ para todo vector unitario $u$; con $u = \nabla f/\|\nabla f\|$ se obtiene $\|\nabla f\| \le 1$. ∎

De hecho, una SDF cumple casi en todas partes la *ecuación eikonal* $\|\nabla f\| = 1$, y su gradiente sobre la superficie es la normal exterior. Las operaciones booleanas son inmediatas:

si $f_1$ y $f_2$ describen $\Omega_1$ y $\Omega_2$, entonces $\min(f_1, f_2)$ describe la unión, $\max(f_1, f_2)$ la intersección y $\max(f_1, -f_2)$ la diferencia $\Omega_1 \setminus \Omega_2$. Los conjuntos son exactos, aunque los valores resultantes pueden ser solo cotas de la distancia (por ejemplo, dentro de una unión). Esta robustez de las operaciones es el argumento del modelado implícito que usan herramientas comerciales como nTop, que publica que sus operaciones booleanas, desfases y redondeos no fallan, y que modula celosías y espesores con campos escalares procedentes de simulaciones [45].

Para fabricar o renderizar con rasterización hay que convertir $f$ en una malla. El algoritmo de *marching cubes* [27] evalúa $f$ en los nodos de una rejilla y, en cada arista con cambio de signo entre $f_0 < 0$ y $f_1 > 0$, coloca un vértice por interpolación lineal en la fracción $t = f_0/(f_0 - f_1)$ de la arista; una tabla indexada por el patrón de signos de los ocho vértices del cubo da los triángulos. El coste es una evaluación de $f$ por nodo, $n^3$ en total, y el error de posición es del orden del paso de la rejilla.

## 1.4 Campos neuronales

Un *campo neuronal* representa $f$ con una red, $f_\varphi(x)$, típicamente un perceptrón multicapa (módulo 9). DeepSDF [30] ajusta una red de ocho capas de 512 unidades a valores de SDF muestreados con la pérdida $L^1$ recortada

$$\ell\big(f_\varphi(x), s\big) = \big|\mathrm{clip}(f_\varphi(x), -\delta, \delta) - \mathrm{clip}(s, -\delta, \delta)\big|,$$

con $\delta = 0{,}1$, que concentra la capacidad cerca de la superficie. Para representar una familia de formas, DeepSDF asigna a cada forma un código latente $z_i$ que se optimiza junto con los pesos, con un término $\|z_i\|^2/\sigma^2$ que procede de un prior gaussiano (un *autodecodificador*); a una forma nueva se le asigna su código optimizando solo $z$ con los pesos congelados. La analogía con un LLM es directa: los códigos $z_i$ desempeñan el papel de las filas de la matriz de embeddings (módulo 12, §4.1), que también son parámetros libres aprendidos por descenso de gradiente.

Cuando no se dispone de valores de SDF sino solo de puntos de la superficie, se impone $f_\varphi = 0$ en ellos y se añade la regularización eikonal $\mathbb{E}_x\big[(\|\nabla_x f_\varphi(x)\| - 1)^2\big]$, cuyo gradiente respecto a $x$ se obtiene por diferenciación automática [5]. Los perceptrones con entradas $(x, y, z)$ aprenden mal los detalles finos; dos remedios son la codificación posicional sinusoidal (tema 4), análoga a la del Transformer (módulo 14, §4.2), y las rejillas multirresolución de vectores entrenables indexadas con funciones *hash*, con las que Müller *et al.* entrenan campos neuronales de calidad en segundos [29].

## 1.5 Diferenciabilidad y elección de la representación

El cuadro 1.1 resume el compromiso. Una malla con conectividad fija es compacta y es lo que consumen los motores y las impresoras, pero su optimización no puede abrir agujeros; las representaciones volumétricas (densidades, SDF, campos) sí pueden, y por eso la optimización de topología (tema 2) trabaja con densidades por elemento. En todas ellas la derivabilidad se refiere a la magnitud que se evalúa: el volumen de una malla es un polinomio en las coordenadas (proposición 1.6), pero la imagen rasterizada de esa misma malla es constante a trozos (tema 4).

| Representación | Parámetros derivables | Cambios de topología | Memoria |
|---|---|---|---|
| Malla | posiciones de los vértices (conectividad fija) | no, salvo remallado | 4,1 MB |
| Nube de puntos | coordenadas de los puntos | no aplica (sin superficie) | — |
| Vóxeles (ocupación) | valor de cada celda | sí | 0,29–2,3 MB |
| SDF en rejilla | valor de cada nodo | sí | 9,2 MB |
| Campo neuronal | pesos de la red | sí | 7,4 MB |

Cuadro 1.1: Comparación para un soporte que cabe en una caja de $120 \times 60 \times 40$ mm con resolución de 0,5 mm. Vóxeles: $240 \cdot 120 \cdot 80 = 2\,304\,000$ celdas (1 bit o 1 byte por celda). SDF: 4 bytes por nodo. Malla: superficie de la caja 28 800 mm$^2$ en triángulos rectángulos de catetos 0,5 mm, 230 400 caras y 115 200 vértices. Red: 8 capas de 512 unidades, $1{,}84 \cdot 10^6$ parámetros de 4 bytes.

## 1.6 Caso de estudio: ajuste de una esfera a un escaneo parcial

**Ejemplo 1.10 (Ajuste algebraico frente a ajuste geométrico).** *Contexto.* En una inspección dimensional, un escáner 3D captura solo la parte superior de un alojamiento esférico: 300 puntos de un casquete de 40° de semiapertura, con ruido gaussiano isótropo de desviación 0,5 mm. Los valores nominales son centro $c^\star = (12;\, -5;\, 30)$ mm y radio $r^\star = 25$ mm (en el cuaderno, los datos se generan con semilla fija).

*Formulación A (algebraica).* Como $\|x\|^2 = 2c \cdot x + (r^2 - \|c\|^2)$, el problema

$$\min_{c,\,k} \sum_{i=1}^{300} \big(\|x_i\|^2 - 2c \cdot x_i - k\big)^2$$

es de mínimos cuadrados lineales en $(c, k)$ (módulo 8, §2.2), con $r = \sqrt{k + \|c\|^2}$.

*Formulación B (geométrica).* Se minimizan los residuos de la SDF, que son las distancias euclídeas de los puntos a la esfera:

$$\min_{c,\,r} \sum_{i=1}^{300} \rho_i^2, \qquad \rho_i = \|x_i - c\| - r, \qquad \frac{\partial \rho_i}{\partial (c, r)} = \Big(-\frac{(x_i - c)^\top}{\|x_i - c\|},\ -1\Big).$$

Es un problema no lineal; se resuelve por Gauss–Newton, $\Delta = \arg\min_\Delta \|J\Delta + \rho\|^2$, partiendo de la solución algebraica.

*Elección del método.* Con ruido gaussiano en las coordenadas, el residuo geométrico es (aproximadamente) el que corresponde a la máxima verosimilitud; el residuo algebraico pondera cada punto por un factor $\|x_i - c\| + r$ y está sesgado cuando los datos cubren solo una parte de la superficie. La formulación A, barata y sin iteraciones, sirve como inicialización.

*Resultado.* El ajuste algebraico da $r = 23{,}977$ mm, un error de $-1{,}02$ mm, y un error de centro de 1,24 mm, con residuo cuadrático medio de 0,491 mm. Gauss–Newton converge en 8 iteraciones a $r = 25{,}418$ mm (error $+0{,}42$ mm), error de centro 0,39 mm y residuo medio 0,477 mm. El residuo es del orden del ruido, como debe ser; el error que queda en $r$ refleja que, con un casquete de 40°, el radio y la coordenada $z$ del centro están muy correlacionados: alejar el centro y aumentar el radio casi no cambia la superficie observada. La conclusión práctica es medir más superficie o fijar el radio nominal y ajustar solo el centro.

| Herramienta | Para qué sirve | Licencia | Cuándo elegirla |
|---|---|---|---|
| trimesh | cargar, reparar y medir mallas (volumen, estanqueidad) | MIT | validación y utilidades en Python puro |
| Open3D | nubes de puntos, registro, reconstrucción | MIT | datos de escáner y visualización |
| PyMeshLab | filtros de MeshLab (remallado, simplificación) | GPL-3.0 | procesado por lotes con algoritmos maduros |
| libigl | geometría discreta (laplacianos, parametrización) | MPL-2.0 | investigación y prototipos de procesado de geometría |
| scikit-image | *marching cubes* sobre rejillas | BSD-3 | extraer mallas de SDF o densidades |
| PyTorch3D | mallas y nubes por lotes, pérdidas y renderizado derivables | BSD | ajuste de formas con gradientes en PyTorch |
| Kaolin (NVIDIA) | operadores 3D derivables y conversiones | Apache-2.0 | ecosistema NVIDIA con GPU |

Cuadro 1.2: Tecnologías del tema 1. Licencias según los metadatos publicados por cada proyecto.

*Observación* 1.11 (Errores frecuentes en la práctica). (i) *Unidades y escala*: mezclar milímetros y metros, o normalizar la forma a $[-1,1]^3$ sin guardar la escala, invalida tolerancias y pérdidas. (ii) *Normales invertidas o malla abierta*: el volumen de la proposición 1.6 cambia de signo o deja de tener sentido; hay que validar antes de medir (tema 6). (iii) *Una función implícita no es una SDF*: $\|x\|^2 - r^2$ tiene la misma superficie que $\|x\| - r$, pero sus valores no son distancias; sin regularización eikonal, una pérdida que use valores de $f$ como distancias (espesores, márgenes) se equivoca. (iv) *Resolución de extracción*: *marching cubes* con un paso mayor que el espesor de una pared la hace desaparecer. (v) *Fugas de datos al evaluar un campo neuronal*: medir el error en los mismos puntos usados para ajustarlo sobrestima la calidad; se evalúa en puntos nuevos o con la distancia de Chamfer frente a la malla de referencia. (vi) *Ajustes algebraicos en datos parciales*: como en el ejemplo 1.10, son sesgados; deben refinarse con el residuo geométrico.

### Lecturas recomendadas para este tema

- L. Guibas, CS348n [4]: clases «Classical 3D Geometry Representations» y «Generative Models I» (DeepSDF).
- J. J. Park *et al.* [30]: §3 (SDF con redes y pérdida recortada) y §4 (autodecodificador).
- W. E. Lorensen y H. E. Cline [27]: el algoritmo de *marching cubes*.
- T. Müller *et al.* [29]: codificación multirresolución con tablas *hash*.
- CMU 16-825 [2], tarea 2, §1 «Exploring loss functions»: ajuste de vóxeles, nubes y mallas.

## Ejercicios del tema 1

**Ejercicio 1.1** ⋆ Indique, justificándolo, qué representación conviene en cada caso: (a) una optimización que debe poder abrir agujeros en una pieza; (b) un recurso que se cargará en un

motor de videojuegos; (c) la salida bruta de un escáner LiDAR; (d) el ajuste de una forma a fotografías con PyTorch3D partiendo de una esfera.

**Ejercicio 1.2** ⋆ Construya las listas de caras de un tetraedro y de un cubo triangulado (cada cara cuadrada dividida en dos triángulos). Cuente vértices, aristas y caras y compruebe la característica de Euler y la proposición 1.3.

*Fuente:* adaptado de CMU 16-825, «Learning for 3D Vision» (S. Tulsiani), tarea 1, §2.1 «Constructing a Tetrahedron» y §2.2 «Constructing a Cube» [2].

**Ejercicio 1.3** ⋆⋆ Una icosfera obtenida subdividiendo tres veces un icosaedro tiene 1280 caras y es cerrada con topología de esfera. Calcule $|E|$ y $|V|$ y la memoria en bytes de la malla con posiciones `float32` e índices `int32`.

**Ejercicio 1.4** ⋆⋆ Sean la esfera $f_e(x) = \|x\| - 20$ y la caja centrada de semilados $(10, 10, 5)$, con SDF $f_b$ (todo en mm). Calcule $f_e$, $f_b$, la unión, la intersección y la diferencia caja menos esfera en $p_1 = (0, 0, 0)$, $p_2 = (15, 0, 0)$ y $p_3 = (0, 0, 12)$. La SDF de una caja de semilados $b$ es $f_b(x) = \|\max(q, 0)\| + \min(\max_i q_i, 0)$, con $q = |x| - b$ componente a componente.

**Ejercicio 1.5** ⋆⋆ (a) En una arista de *marching cubes* que va de $x = 2$ mm, con $f = -0{,}3$, a $x = 3$ mm, con $f = 0{,}5$, ¿dónde se coloca el vértice? (b) Calcule la distancia de Chamfer entre $A = \{(0, 0), (1, 0)\}$ y $B = \{(0, 1), (2, 0)\}$ en el plano.

**Ejercicio 1.6** ⋆⋆ Una red predice $f_\varphi = 0{,}05;\ 0{,}30;\ -0{,}08;\ -0{,}40$ en cuatro puntos cuyas SDF verdaderas son $0{,}02;\ 0{,}12;\ 0{,}15;\ -0{,}02$. Calcule la pérdida $L^1$ recortada media con $\delta = 0{,}1$ e interprete los términos nulos.

*Fuente:* adaptado de Stanford CS348n (L. Guibas, primavera de 2023), tarea 2, problema 1(a) «Supervised SDF fitting» [5].

**Ejercicio 1.7** ⋆⋆ Para el tetraedro del ejemplo 1.7 con $a = 30$ mm, calcule $\nabla_{v_1} \mathrm{Vol}$ con la proposición 1.6 y compruebe el resultado con la fórmula base por altura. ¿Cuánto cambia el volumen si $v_1$ se desplaza 0,5 mm en la dirección $x$?

**Ejercicio 1.8** ⋆⋆⋆ (a) Demuestre que $f(x) = \|x - c\| - r$ cumple $\|\nabla f\| = 1$ en todo $x \neq c$. (b) Sea $g = \varphi \circ f$ con $\varphi$ derivable, estrictamente creciente y $\varphi(0) = 0$. Demuestre que $g$ describe el mismo sólido y que $\|\nabla g\| = \varphi'(f)$. (c) Para $g(x) = \|x\|^2 - 4$ (es decir, $\varphi(t) = t^2 + 4t$ con $r = 2$), calcule el término eikonal $(\|\nabla g\| - 1)^2$ en el punto $(2, 0, 0)$ de la superficie y explique qué corrige la regularización.

*Fuente:* adaptado de Stanford CS348n (L. Guibas, primavera de 2023), tarea 2, problema 1(b) «Eikonal training» [5].

**Ejercicio 1.9** ⋆⋆⋆ Sea una malla triangular cerrada y orientada de forma coherente. (a) Demuestre que $\sum_f (v_b - v_a) \times (v_c - v_a) = 0$. (b) Deduzca que la fórmula de la proposición 1.6 no cambia al trasladar todos los vértices un mismo vector $t$, y que $\sum_i \nabla_{v_i} \mathrm{Vol} = 0$.

Tema 2

# Optimización de forma y de topología

La optimización de topología responde a una pregunta de diseño mecánico: dado un volumen de trabajo, unos apoyos, unas cargas y una cantidad de material, ¿dónde se pone el material para que la pieza sea lo más rígida posible? A diferencia de la optimización de forma, que mueve una frontera ya existente, aquí se permite que aparezcan y desaparezcan agujeros. Este tema sigue el capítulo 1 de Bendsøe y Sigmund [10], el artículo de Bendsøe de 1989 que formuló el problema como distribución de material [14], los códigos didácticos de 99 y 88 líneas [36, 13], el estudio de inestabilidades de Sigmund y Petersson [37] y el análisis de filtros de Bourdin [15]. El método adjunto se presenta como en las notas de cálculo matricial del MIT [1, §6.3].

## 2.1 El problema de flexibilidad mínima

Se discretiza el dominio de diseño en $N$ elementos finitos cuadrados. El elemento $e$ tiene una densidad $x_e \in [0,1]$ (0 vacío, 1 material) y una matriz de rigidez $E_e\,k_0$, donde $k_0 \in \mathbb{R}^{8\times 8}$ es la del elemento bilineal con módulo unidad. El ensamblado da la matriz global $K(x) = \sum_e E_e(x_e)\,P_e^{\top} k_0 P_e$, simétrica y definida positiva una vez impuestos los apoyos, donde $P_e$ extrae los ocho grados de libertad del elemento.

**Definición 2.1 (Flexibilidad mínima con restricción de volumen).** Dadas las cargas $F$ y una fracción de volumen $f_V \in (0,1)$, el problema es

$$\min_{x\in[0,1]^N} \; c(x) = F^{\top}U(x) \quad \text{sujeto a} \quad K(x)\,U(x) = F, \qquad \frac{1}{N}\sum_{e=1}^{N} x_e = f_V .$$

La *flexibilidad* $c = F^{\top}U = U^{\top}KU$ es el trabajo de las cargas exteriores, igual al doble de la energía elástica almacenada; minimizarla equivale a maximizar la rigidez global.

Si las densidades solo pudieran valer 0 o 1, el problema sería combinatorio, con $2^N$ diseños. La relajación a $[0,1]$ lo vuelve continuo y derivable, pero entonces hay que conseguir que el óptimo no esté lleno de material «gris» de densidad intermedia, que no se puede fabricar.

## 2.2 La interpolación SIMP

**Definición 2.2 (SIMP modificado).** El método SIMP (*solid isotropic material with penalization*) interpola el módulo de Young como

$$E_e(x_e) = E_{\min} + x_e^{\,p}\,(E_0 - E_{\min}), \qquad p > 1,$$

con $E_{\min} > 0$ pequeño (por ejemplo $10^{-9}E_0$) para que $K$ no sea singular cuando hay elementos vacíos [13, §2.1].

La penalización funciona porque el volumen es lineal en $x_e$ y la rigidez no. Con $p = 3$, un elemento a densidad 0,5 consume la mitad del material pero aporta solo $0{,}5^3 = 0{,}125$ de la rigidez: la relación rigidez/volumen es $x_e^{p-1} = 0{,}25$, cuatro veces peor que la del material sólido. El optimizador aprende así a evitar los grises. El precio es la pérdida de convexidad.

**Teorema 2.3 (Convexidad con $p = 1$).** *Si $E_e(x_e) = E_{\min} + x_e(E_0 - E_{\min})$, es decir, con $p = 1$, la flexibilidad $c(x) = F^\top K(x)^{-1} F$ es una función convexa de $x$ en $[0,1]^N$.*

*Demostración.* Para una matriz $K$ simétrica definida positiva se cumple

$$F^\top K^{-1} F = \max_u \big(2F^\top u - u^\top K u\big),$$

porque la función de $u$ entre paréntesis es cóncava, su gradiente $2F - 2Ku$ se anula en $u = K^{-1}F$ y su valor ahí es $2F^\top K^{-1}F - F^\top K^{-1}F = F^\top K^{-1}F$. Con $p = 1$, $K(x)$ es afín en $x$, así que para cada $u$ fijo la función $x \mapsto 2F^\top u - u^\top K(x) u$ es afín. El máximo puntual de una familia de funciones afines es convexo (módulo 4, tema 5), luego $c$ es convexa. ∎

Con $p = 1$ (la «chapa de espesor variable») el problema es convexo y su óptimo global es fácil de alcanzar, pero contiene grandes zonas grises. Con $p = 3$, el valor estándar, desaparece la convexidad y el resultado depende del punto de partida y de los parámetros; el filtrado y las estrategias de continuación en $p$ mitigan el problema [37].

## 2.3 Sensibilidades por el método adjunto

**Teorema 2.4 (Método adjunto para ecuaciones de equilibrio).** *Sea $J(x) = g\big(x, U(x)\big)$ con $K(x)U = F$, $K(x)$ invertible y derivable, y $F$ independiente de $x$. Sea $\lambda$ la solución del* problema adjunto $K(x)^\top \lambda = \big(\partial g/\partial U\big)^\top$. *Entonces*

$$\frac{dJ}{dx_e} = \frac{\partial g}{\partial x_e} - \lambda^\top \frac{\partial K}{\partial x_e} U.$$

*Demostración.* Derivando $KU = F$ respecto a $x_e$ se obtiene $\frac{\partial K}{\partial x_e}U + K\frac{dU}{dx_e} = 0$, luego $\frac{dU}{dx_e} = -K^{-1}\frac{\partial K}{\partial x_e}U$. Por la regla de la cadena,

$$\frac{dJ}{dx_e} = \frac{\partial g}{\partial x_e} + \frac{\partial g}{\partial U}\frac{dU}{dx_e} = \frac{\partial g}{\partial x_e} - \frac{\partial g}{\partial U}K^{-1}\frac{\partial K}{\partial x_e}U.$$

Agrupando de izquierda a derecha, $\frac{\partial g}{\partial U}K^{-1} = \lambda^\top$ con $K^\top\lambda = (\partial g/\partial U)^\top$, y queda la fórmula. ∎

La demostración es la asociatividad del producto de jacobianos que distingue el modo inverso del directo (módulo 10, §4.2): multiplicar primero $\frac{\partial g}{\partial U}K^{-1}$ cuesta una sola resolución, sea cual sea el número $N$ de variables, mientras que calcular cada $dU/dx_e$ costaría $N$ resoluciones. El vector $\lambda$ es el multiplicador de Lagrange de la restricción $KU = F$ (módulo 10, §5.4).

**Corolario 2.5 (La flexibilidad es autoadjunta).** *Para $c = F^\top U$ se tiene $\lambda = U$ y*

$$\frac{\partial c}{\partial x_e} = -U^\top \frac{\partial K}{\partial x_e} U = -p\, x_e^{p-1}(E_0 - E_{\min})\, u_e^\top k_0 u_e \;\le 0.$$

*Demostración.* $g = F^\top U$ no depende explícitamente de $x$ y $(\partial g/\partial U)^\top = F$, luego el problema adjunto es $K\lambda = F$ ($K$ es simétrica), cuya solución es $U$. Además $\partial K/\partial x_e = p\,x_e^{p-1}(E_0 - E_{\min})P_e^\top k_0 P_e$, y $U^\top P_e^\top k_0 P_e U = u_e^\top k_0 u_e \geq 0$ porque $k_0$ es semidefinida positiva. ∎

El signo tiene una lectura física: añadir material nunca hace la estructura más flexible. Calcular el gradiente completo no cuesta ninguna resolución adicional; basta con la energía $u_e^\top k_0 u_e$ de cada elemento.

**Ejemplo 2.6 (Dos barras en serie).** Dos barras en serie, con rigideces $k_e = x_e^3 k_0$ ($k_0 = 1000$ N/mm), densidades $x = (0{,}8;\ 0{,}5)$ y carga $F = 100$ N en el extremo libre. Entonces $k = (512;\ 125)$ N/mm y $c = F^2(1/k_1 + 1/k_2) = 10^4(0{,}001953 + 0{,}008) = 99{,}53$ N · mm. Los alargamientos son $u_e = F/k_e$, y por el corolario 2.5,

$$\frac{\partial c}{\partial x_e} = -3x_e^2 k_0\, u_e^2 = -\frac{3x_e^2 k_0 F^2}{k_e^2}: \qquad \frac{\partial c}{\partial x_1} = -73{,}24, \quad \frac{\partial c}{\partial x_2} = -480 \ \text{ N}\cdot\text{mm}.$$

Con el mismo material añadido, la barra débil reduce la flexibilidad 6,6 veces más: la sensibilidad indica dónde se necesita el material.

## 2.4 Actualización por criterios de optimalidad

Con $V(x) = \sum_e v_e x_e$ (en la malla regular, $v_e = 1$), la lagrangiana del problema es $c(x) + \Lambda\big(V(x) - N f_V\big)$ más los términos de las cotas. Para un elemento con $0 < x_e < 1$, la condición de estacionariedad (0.7, §5.4) es $-\partial c/\partial x_e = \Lambda\, v_e$, es decir,

$$B_e := \frac{-\partial c/\partial x_e}{\Lambda\, \partial V/\partial x_e} = 1.$$

La actualización por criterios de optimalidad (OC) convierte esta condición en una iteración de punto fijo: si $B_e > 1$, el elemento rinde más que el precio $\Lambda$ del material y conviene aumentar $x_e$; si $B_e < 1$, disminuirla.

**Definición 2.7 (Actualización OC).** Con amortiguamiento $\eta = 1/2$ y límite de movimiento $m$ (0,2 en [13]),

$$x_e^{\text{nuevo}} = \operatorname{clip}\Big(x_e B_e^{\eta},\ \max(0, x_e - m),\ \min(1, x_e + m)\Big),$$

donde $\Lambda$ se elige por bisección para que $x^{\text{nuevo}}$ cumpla la restricción de volumen.

**Proposición 2.8.** *Si $\partial c/\partial x_e < 0$ para todo $e$, el volumen $V\big(x^{nuevo}(\Lambda)\big)$ es una función continua y no creciente de $\Lambda > 0$; por tanto, la bisección encuentra un $\Lambda$ que cumple la restricción siempre que esta sea alcanzable dentro de los límites de movimiento.*

*Demostración.* $B_e^\eta = \big(-\partial_e c/(\Lambda v_e)\big)^\eta$ es continua y estrictamente decreciente en $\Lambda$, la función clip es continua y no decreciente en su primer argumento, y $V$ es una suma con coeficientes positivos. Una función continua y monótona que toma valores por encima y por debajo del objetivo en los extremos del intervalo de búsqueda alcanza el objetivo (teorema de Bolzano), y la bisección lo localiza. ∎

## 2.5 Inestabilidades numéricas y filtros

Sin regularización, SIMP produce dos patologías bien documentadas [37]: *tableros de ajedrez*, zonas de elementos sólidos y vacíos alternados que el elemento bilineal considera artificialmente rígidas, y *dependencia de la malla*: al refinar aparecen cada vez más barras finas, porque el problema continuo carece de solución con microestructura acotada. La solución estándar es imponer una escala de longitud mínima mediante un filtro de radio $r_{\min}$, con pesos

$$H_{ei} = \max\big(0,\ r_{\min} - \operatorname{dist}(e,i)\big),$$

donde la distancia se mide entre centros de elementos.

**Definición 2.9 (Filtros de sensibilidades y de densidades).** El *filtro de sensibilidades* [36, 13] sustituye las derivadas por un promedio ponderado,

$$\widehat{\frac{\partial c}{\partial x_e}} = \frac{1}{\max(\gamma, x_e)\sum_i H_{ei}} \sum_i H_{ei}\, x_i\, \frac{\partial c}{\partial x_i}, \qquad \gamma = 10^{-3}.$$

El *filtro de densidades* [15] define densidades físicas $\tilde{x}_e = \sum_i H_{ei} x_i / \sum_i H_{ei}$, con las que se calcula la rigidez; las sensibilidades respecto a las variables de diseño se obtienen por la regla de la cadena,

$$\frac{\partial c}{\partial x_j} = \sum_e \frac{\partial c}{\partial \tilde{x}_e}\frac{\partial \tilde{x}_e}{\partial x_j} = \sum_e \frac{H_{ej}}{\sum_i H_{ei}}\,\frac{\partial c}{\partial \tilde{x}_e}.$$

El filtro de densidades tiene una justificación matemática: Bourdin demuestra que el problema filtrado tiene solución y que las aproximaciones por elementos finitos convergen [15]. El filtro de sensibilidades es heurístico (el gradiente filtrado no es el de ninguna función), pero funciona muy bien en la práctica. En ambos casos, el diseño que se presenta es el de las densidades físicas $\tilde{x}$, no el de las variables $x$ [13, §2.3]. La proyección de Heaviside, $\bar{x} = 1 - e^{-\beta\tilde{x}} + \tilde{x}e^{-\beta}$ con $\beta$ creciente, elimina los grises que el filtro de densidades deja en los bordes.

## 2.6 Caso de estudio: la viga MBB

**Ejemplo 2.10 (Viga MBB con el código de 88 líneas en numpy).** *Problema.* La viga MBB es una viga biapoyada con una carga puntual en el centro del borde superior. Por simetría se modela la mitad: un dominio de $60 \times 20$ elementos cuadrados de 1 mm, con los desplazamientos horizontales impedidos en el borde izquierdo (eje de simetría), el vertical impedido en la esquina inferior derecha (apoyo) y una carga vertical unitaria en la esquina superior izquierda. Fracción de volumen $f_V = 0{,}5$, $p = 3$, $E_0 = 1$, $E_{\min} = 10^{-9}$, $\nu = 0{,}3$, filtro de sensibilidades con $r_{\min} = 2{,}4$ (0,04 veces la anchura, como en los ensayos de [13, §3.4]).

*Formulación.* La de la definición 2.1 con SIMP, $N = 1200$ variables y $2 \cdot 61 \cdot 21 = 2562$ grados de libertad. En cada iteración: (1) ensamblar $K(\tilde{x})$ en formato disperso; (2) resolver $KU = F$; (3) calcular $u_e^\top k_0 u_e$ y las sensibilidades del corolario 2.5; (4) filtrar; (5) actualizar con OC (definición 2.7). Se para cuando el cambio máximo de densidad es menor que 0,01.

*Elección del método.* Las sensibilidades adjuntas cuestan cero resoluciones adicionales; por diferencias finitas, cada iteración necesitaría 1201 resoluciones. OC es el método más simple para un problema con una única restricción y gradientes de signo conocido (proposición 2.8); con varias restricciones se usa el método de las asíntotas móviles (MMA).

*Resultado.* La traducción a numpy y scipy del código de Andreassen *et al.* converge en 106 iteraciones (unos 1,6 s en una CPU) a una flexibilidad $c = 216{,}81$, el valor publicado en [13, fig. 3]; la flexibilidad inicial con densidad uniforme era 1007,0, de modo que la optimización la reduce un 78 %. Con filtro de densidades se obtiene $c = 233{,}71$, también idéntico al publicado, en 144 iteraciones, y un diseño con bordes más difusos (figura 2.1): la medida de grises $4\,\overline{\tilde{x}(1-\tilde{x})}$ vale 0,265 con el filtro de sensibilidades y 0,330 con el de densidades. Con la llamada por defecto del artículo, $r_{\min} = 1{,}5$, la flexibilidad baja a 203,19 porque se permiten barras más finas.

*Unidades físicas.* Para elementos cuadrados en tensión plana, $K$ escala con $E_0 t$ ($t$ espesor) y no depende del tamaño del elemento, así que $c_{\text{fís}} = \bar{c}\,F^2/(E_0 t)$. Con aluminio ($E_0 = 70$ GPa), $t = 10$ mm y $F = 1$ kN en el semimodelo (2 kN sobre la viga completa de $120 \times 20$ mm): $c_{\text{fís}} = 216{,}81 \cdot 10^6/(7 \cdot 10^8) = 0{,}310$ J, flecha bajo la carga $c/F = 0{,}310$ mm y masa del semimodelo $0{,}5 \cdot 60 \cdot 20 \cdot 10$ mm$^3 \cdot 2{,}70$ g/cm$^3 = 16{,}2$ g.

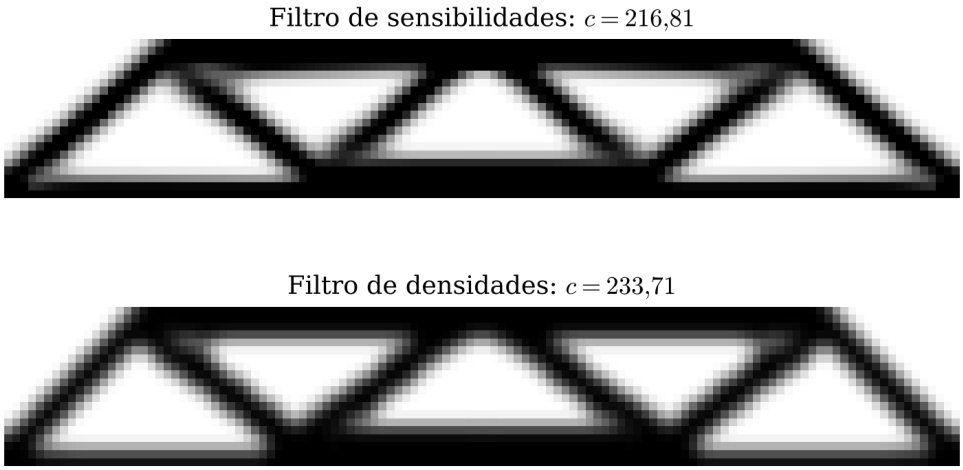



Figura 2.1: Diseños de la viga MBB completa (semimodelo de $60 \times 20$ elementos reflejado), $f_V = 0{,}5$, $p = 3$, $r_{\min} = 2{,}4$. Negro: material. Calculado con el cuaderno del módulo.

## 2.7 Fabricación aditiva y diseño generativo

Un diseño optimizado solo es útil si se puede fabricar. Las restricciones más comunes son el tamaño mínimo de las barras y de los huecos, que el radio del filtro controla; la dirección de fabricación, porque en fabricación aditiva las superficies que miran hacia abajo con poca inclinación necesitan soportes (tema 6); y, para mecanizado o fundición, la accesibilidad de la herramienta y el desmoldeo. Langelaar propuso un filtro que, capa a capa, solo permite material que esté sostenido por material de la capa inferior, de modo que el resultado es imprimible sin soportes [22]. Tras optimizar, el flujo habitual es umbralizar $\tilde{x}$, extraer la superficie (*marching cubes* o *squares*, tema 1), suavizarla, reconstruir una geometría CAD y *volver a verificar* con un análisis de elementos finitos más fino, incluyendo tensiones, que el problema de flexibilidad no controla.

Los programas comerciales de *diseño generativo* empaquetan este flujo. La documentación

pública de Autodesk Fusion describe métodos de fabricación seleccionables (sin restricciones, aditiva con dirección de impresión, fresado de 2,5, 3 y 5 ejes, corte en 2 ejes y fundición con espesor mínimo, ángulo de desmoldeo y dirección de expulsión), cada uno de los cuales genera su propio conjunto de resultados [39]. El caso publicado más citado es el soporte de cinturón de un asiento de General Motors: según Autodesk, el diseño generativo consolidó ocho piezas en una sola de acero inoxidable, un 40 % más ligera y un 20 % más resistente que la anterior [38]. nTop combina el modelado implícito con resultados de optimización de topología y con campos que modulan celosías y espesores [45]. No se han publicado los algoritmos internos de estos productos.

| Herramienta | Para qué sirve | Licencia o tipo | Cuándo elegirla |
|---|---|---|---|
| top88 y variantes (DTU) | códigos didácticos SIMP en MATLAB | descarga gratuita (DTU) | aprender y prototipar en 2D |
| numpy + scipy.sparse | montaje y resolución de elementos finitos | BSD | reproducir top88 en Python (cuaderno) |
| NLopt (MMA) | optimización con muchas restricciones | MIT | sustituir OC cuando hay varias restricciones |
| dolfin-adjoint | adjuntos automáticos sobre FEniCS | LGPL | EDP generales en 2D y 3D sin derivar a mano |
| Autodesk Fusion | diseño generativo con restricciones de fabricación | comercial | ingeniería de producto con CAD integrado |
| nTop | modelado implícito, celosías, campos | comercial | piezas para fabricación aditiva con celosías |

Cuadro 2.1: Tecnologías del tema 2.

*Observación* 2.11 (Errores frecuentes en la práctica). (i) *No filtrar*: aparecen tableros de ajedrez y el resultado depende de la malla. (ii) *Presentar $x$ en lugar de $\tilde{x}$* con el filtro de densidades: el diseño mostrado no es el analizado. (iii) $E_{\min} = 0$: $K$ se vuelve singular en cuanto un elemento se vacía. (iv) *Confundir rigidez con resistencia*: mínima flexibilidad no garantiza tensiones admisibles; hay que comprobarlas, sobre todo en esquinas entrantes. (v) *Mínimos locales*: con $p = 3$ el resultado depende de los parámetros; conviene la continuación en $p$ y comparar varias ejecuciones. (vi) *Olvidar la fabricación*: voladizos, cavidades cerradas que atrapan polvo o resina y espesores inferiores a la resolución de la máquina. (vii) *Coste*: en 3D, la resolución de $KU = F$ domina; se necesitan resolvedores iterativos con precondicionadores multimalla para pasar de unos miles a millones de elementos.

### Lecturas recomendadas para este tema

- M. P. Bendsøe y O. Sigmund [10], cap. 1 «Topology optimization by distribution of isotropic material».
- E. Andreassen *et al.* [13]: §2 (formulación, SIMP modificado, OC y filtros) y §3 (implementación).
- O. Sigmund y J. Petersson [37]: tableros de ajedrez, dependencia de la malla y mínimos locales.
- P. Bright, A. Edelman y S. G. Johnson [1], §6.3 «Reverse-mode "adjoint" differentiation».
- M. Langelaar [22]: restricciones de voladizo para fabricación aditiva.

## Ejercicios del tema 2

**Ejercicio 2.1** ⋆ Justifique si son verdaderas o falsas: (a) con $p = 1$ el diseño óptimo suele ser blanco y negro; (b) $E_{\min} > 0$ evita que $K$ sea singular; (c) minimizar la flexibilidad equivale a minimizar la flecha en todos los puntos; (d) el filtro de sensibilidades calcula el gradiente exacto de una función filtrada.

**Ejercicio 2.2** ⋆ Explique por qué el resultado de un programa de diseño generativo debe volver a analizarse por elementos finitos antes de fabricarlo, y cite dos comprobaciones que el problema de flexibilidad mínima no incluye.

**Ejercicio 2.3** ⋆⋆ Para $x_e = 0{,}5$, calcule la relación rigidez/volumen $E_e/(E_0 x_e)$ con $p = 1, 2, 3$ ($E_{\min} \approx 0$). ¿Qué densidad necesita un elemento con $p = 3$ para aportar la mitad de la rigidez del sólido?

**Ejercicio 2.4** ⋆⋆ Tres elementos con $x = (0{,}5;\ 0{,}5;\ 0{,}5)$, sensibilidades $\partial c/\partial x = (-4;\ -1;\ -0{,}25)$, $\partial V/\partial x_e = 1$, $\eta = 1/2$, $m = 0{,}2$ y volumen objetivo 1,5. Aplique la definición 2.7 y encuentre el valor de $\Lambda$ que cumple el volumen.

**Ejercicio 2.5** ⋆⋆ En una malla regular con $r_{\min} = 1{,}5$ y densidades uniformes $x_i = 0{,}5$, el elemento central de un bloque $3 \times 3$ tiene sensibilidad $-4$; sus cuatro vecinos laterales, $-2$, y sus cuatro vecinos diagonales, $-1$. Calcule la sensibilidad filtrada del elemento central.

**Ejercicio 2.6** ⋆⋆ Una fila de tres elementos con $r_{\min} = 1{,}5$ (peso propio 1,5 y peso 0,5 para los vecinos inmediatos) y $x = (1;\ 0{,}5;\ 0)$. Calcule las densidades físicas $\tilde{x}$ y, si $\partial c/\partial \tilde{x} = (-3;\ -2;\ -1)$, las sensibilidades respecto a $x$. Compruebe que la suma de sensibilidades se conserva.

*Fuente:* adaptado de IUPUI ME 60602 «Topology Optimization» (A. Tovar), clase 12 «Well-posedness, numerical problems, filters» [9].

**Ejercicio 2.7** ⋆⋆ El resultado adimensional $\bar{c} = 216{,}81$ de la viga MBB se fabrica en acero ($E_0 = 200$ GPa, densidad 7,85 g/cm³) con espesor $t = 5$ mm, elementos de 1 mm y una carga de 2 kN en el semimodelo. Calcule la flexibilidad física, la flecha bajo la carga y la masa del semimodelo.

**Ejercicio 2.8** ⋆⋆⋆ En un mecanismo flexible se quiere maximizar el desplazamiento de un nodo de salida, $g = L^{\top} U$, con $L$ el vector canónico de ese grado de libertad. (a) Escriba el problema adjunto y la sensibilidad $dg/dx_e$ con el teorema 2.4. ¿Cuántas resoluciones cuesta el gradiente? (b) Compruébelo en el sistema de dos muelles en serie del ejemplo 2.6 tomando como salida el nodo intermedio, $g = u_1$, cuyo valor exacto es $F/k_1$.

*Fuente:* adaptado de MIT 18.S096 «Matrix Calculus for Machine Learning and Beyond» (A. Edelman y S. G. Johnson, IAP 2023), §6.3 «Reverse-mode "adjoint" differentiation» [1].

**Ejercicio 2.9** ⋆⋆⋆ (a) Demuestre que en el problema de la definición 2.1 con SIMP se cumple $\partial c/\partial x_e \le 0$ y deduzca que en un óptimo con $0 < x_e < 1$ para algún $e$ la restricción de volumen puede tomarse como desigualdad activa $\sum_e x_e \le N f_V$. (b) Escriba las condiciones de Karush–Kuhn–Tucker con las cotas $0 \le x_e \le 1$ y deduzca que, para los elementos interiores, $B_e = 1$, y que en los elementos con $x_e = 1$ se cumple $B_e \ge 1$.

*Fuente:* adaptado de M. P. Bendsøe y O. Sigmund, *Topology Optimization*, cap. 1 [10].

Tema 3

# Simuladores diferenciables y modelos sustitutos

El tema 2 obtuvo el gradiente de un simulador de elementos finitos derivando a mano sus ecuaciones. Este tema generaliza la idea a cualquier simulador, estacionario o temporal, y presenta la alternativa cuando derivar no es posible o el simulador es demasiado caro: aprender un modelo sustituto con pocas simulaciones y optimizar sobre él. Se siguen las notas de S. G. Johnson sobre adjuntos para recurrencias [7], las de cálculo matricial del MIT [1, §6.3], los capítulos de modelos sustitutos de Kochenderfer y Wheeler [12], y los artículos de DiffTaichi [16], las redes informadas por la física [33], el operador neuronal de Fourier [25] y MeshGraphNets [31]. La física de la aleta procede del capítulo 3 de Incropera *et al.* [11].

## 3.1 Derivar a través de un simulador

Casi todo simulador discreto resuelve un sistema de ecuaciones $R(u,\theta)=0$, donde $u\in\mathbb{R}^m$ es el estado (temperaturas, desplazamientos, velocidades) y $\theta\in\mathbb{R}^n$ son los parámetros de diseño.

**Teorema 3.1 (Sensibilidad por la función implícita y el adjunto).** *Sean $R:\mathbb{R}^m\times\mathbb{R}^n\to\mathbb{R}^m$ y $J(\theta)=g\big(u(\theta),\theta\big)$ funciones continuamente derivables, con $R(u(\theta),\theta)=0$ y $\partial R/\partial u$ invertible en la solución. Entonces*

$$\frac{du}{d\theta}=-\Big(\frac{\partial R}{\partial u}\Big)^{-1}\frac{\partial R}{\partial \theta},\qquad \frac{dJ}{d\theta}=\frac{\partial g}{\partial\theta}-\lambda^{\top}\frac{\partial R}{\partial\theta},\quad \text{con}\quad \Big(\frac{\partial R}{\partial u}\Big)^{\top}\lambda=\Big(\frac{\partial g}{\partial u}\Big)^{\top}.$$

*Demostración.* El teorema de la función implícita garantiza que $u(\theta)$ es derivable cerca de la solución. Derivando la identidad $R(u(\theta),\theta)=0$ se obtiene $\frac{\partial R}{\partial u}\frac{du}{d\theta}+\frac{\partial R}{\partial\theta}=0$, de donde sale la primera fórmula. Sustituyéndola en $\frac{dJ}{d\theta}=\frac{\partial g}{\partial\theta}+\frac{\partial g}{\partial u}\frac{du}{d\theta}$ y agrupando $\frac{\partial g}{\partial u}\big(\frac{\partial R}{\partial u}\big)^{-1}=\lambda^{\top}$ se obtiene la segunda. ∎

El teorema 2.4 es el caso $R=K(x)U-F$. El coste del adjunto es una resolución con la matriz transpuesta, que en problemas lineales puede reutilizar la factorización ya calculada; el de la sensibilidad directa, $n$ resoluciones. Para simuladores temporales, $u_{k+1}=\Phi(u_k,\theta)$, el mismo razonamiento da una recurrencia adjunta que se recorre hacia atrás en el tiempo [7]:

$$\lambda_N=\Big(\frac{\partial g}{\partial u_N}\Big)^{\top},\qquad \lambda_{k-1}=\Big(\frac{\partial \Phi}{\partial u}\Big)_k^{\top}\lambda_k,\qquad \frac{dg}{d\theta}=\frac{\partial g}{\partial\theta}+\sum_{k=1}^{N}\lambda_k^{\top}\frac{\partial\Phi}{\partial\theta}\Big|_{u_{k-1}}.$$

Es la retropropagación a través del tiempo de una red recurrente (módulo 12, §5.3), con los mismos problemas: hay que guardar todos los estados, lo que se alivia con el recálculo por puntos de control (módulo 10, §5.2), y en dinámicas caóticas o con muchos pasos el gradiente puede explotar.

## 3.2 Diferenciación automática de simuladores

En lugar de derivar a mano, el simulador se puede escribir en un lenguaje que propague derivadas. JAX transforma programas de Python y NumPy (derivación, vectorización y compilación para GPU y TPU) [41]; NVIDIA Warp compila núcleos de Python para GPU que son derivables [47]; Taichi, con su extensión DiffTaichi, permite escribir simuladores físicos derivables, y sus autores informan de que su simulador elástico necesita unas 4,2 veces menos líneas que una versión CUDA escrita a mano, con velocidad similar, y es 188 veces más rápido que la implementación en TensorFlow [16]. PyTorch también sirve cuando el simulador se expresa con operaciones de tensores, como en el caso de estudio de este tema.

Tres advertencias limitan el enfoque. Los eventos discretos (contactos, cambios de malla, condicionales sobre el estado) producen funciones discontinuas cuya derivada automática es correcta pero inútil, porque vale cero o no ve el salto. Las derivadas de un resolvedor iterativo no deben calcularse a través de sus iteraciones, sino con el teorema 3.1 en la solución convergida. Y el coste de memoria del modo inverso crece con el número de pasos de tiempo.

## 3.3 Modelos sustitutos

**Definición 3.2 (Modelo sustituto).** Sea $s : \Theta \to \mathbb{R}$ una respuesta cara de evaluar (el resultado de un simulador). Un *modelo sustituto* es una función $\hat{s}_\varphi$ barata, ajustada con un conjunto de diseño $\{(\theta_i, s(\theta_i))\}_{i=1}^{M}$, con la que se sustituye a $s$ dentro de la optimización.

Los sustitutos clásicos son las superficies de respuesta polinómicas ajustadas por mínimos cuadrados y los procesos gaussianos, que además estiman su incertidumbre [12]. Los neuronales cubren desde un perceptrón que aproxima una respuesta escalar hasta modelos que aproximan campos completos: las redes informadas por la física (PINN) minimizan el residuo de la ecuación diferencial en puntos de colocación, calculando las derivadas de la red por diferenciación automática [33]; los operadores neuronales de Fourier aprenden la aplicación entre funciones de entrada y de salida, y sus autores informan de aceleraciones de hasta tres órdenes de magnitud frente a resolvedores clásicos [25]; MeshGraphNets aprende la dinámica sobre la malla de simulación con una red de grafos [31]. Bibliotecas como NVIDIA PhysicsNeMo (antes Modulus) agrupan estas arquitecturas para construir sustitutos de CFD, mecánica estructural o análisis térmico [46].

El diseño de experimentos importa tanto como el modelo. El muestreo por hipercubo latino divide el rango de cada variable en $M$ intervalos y toma exactamente una muestra por intervalo, de modo que sus proyecciones sobre cada eje quedan uniformemente repartidas [12].

**Proposición 3.3 (Error del óptimo de un sustituto).** *Sea $\Theta$ compacto, $s$ y $\hat{s}$ continuas y $\sup_{\theta\in\Theta}|\hat{s}(\theta) - s(\theta)| \le \varepsilon$. Si $\theta^\star$ minimiza $s$ y $\hat{\theta}$ minimiza $\hat{s}$ en $\Theta$, entonces*

$$s(\theta^\star) \le s(\hat{\theta}) \le s(\theta^\star) + 2\varepsilon.$$

*Demostración.* La primera desigualdad es la optimalidad de $\theta^\star$. Para la segunda, $s(\hat{\theta}) \le \hat{s}(\hat{\theta}) + \varepsilon \le \hat{s}(\theta^\star) + \varepsilon \le s(\theta^\star) + 2\varepsilon$, usando sucesivamente la cota de error, la optimalidad de $\hat{\theta}$ para $\hat{s}$ y otra vez la cota. ∎

La proposición es tranquilizadora y peligrosa a la vez: la garantía exige un error *uniforme* en todo $\Theta$, mientras que la validación solo mide el error en unos puntos. El optimizador busca

precisamente las zonas en que $\hat{s}$ es optimista, que suelen ser aquellas en que hay pocos datos. Es el mismo fenómeno que la sobreoptimización de un modelo de recompensa (módulo 21, §6.4). Por eso todo diseño propuesto por un sustituto se verifica con el simulador y, si hay presupuesto, se añade a los datos y se reajusta el modelo.

## 3.4 Caso de estudio: el perfil de una aleta disipadora

**Ejemplo 3.4 (Aleta de aluminio con sustituto neuronal y con simulador diferenciable).** *Contexto.* Un disipador de aluminio ($k = 200$ W/(m · K)) refrigerado por aire forzado ($h = 60$ W/(m² · K)) debe evacuar calor desde una base a $\theta_b = T_b - T_\infty = 50$ K. La masa de cada aleta está fijada: por cada metro de profundidad, el área del perfil es $A = 60$ mm². El perfil es trapecial, con espesor $t(x) = t_b(1 - s\,x/L)$ para $0 \le x \le L$, donde $L$ es la longitud y $s \in [0, 1)$ la conicidad; la condición de área fija da $t_b = A/\big(L(1 - s/2)\big)$. Por extrusión, la punta debe medir al menos $t_{\min} = 0{,}3$ mm.

*Modelo físico.* Con aleta delgada (número de Biot $ht/(2k) \ll 1$), conducción longitudinal y convección por las dos caras, la temperatura $\theta(x)$ por metro de profundidad satisface [11]

$$\frac{d}{dx}\Big(k\,t(x)\,\frac{d\theta}{dx}\Big) - 2h\,\theta = 0, \qquad \theta(0) = \theta_b, \qquad \theta'(L) = 0,$$

y el calor evacuado es $q = 2h\int_0^L \theta\,dx$. Para $s = 0$ hay solución cerrada, $q = \sqrt{2hkt}\,\theta_b \tanh(mL)$ con $m = \sqrt{2h/(kt)}$.

*Formulación.*

$$\max_{L,\,s}\; q(L, s) \quad \text{sujeto a} \quad t_b(1 - s) \ge t_{\min}, \quad 20 \le L \le 100 \text{ mm}, \quad 0 \le s \le 0{,}9.$$

El simulador discretiza la ecuación por volúmenes finitos con 400 celdas y resuelve un sistema tridiagonal; reproduce la solución cerrada de la aleta rectangular con un error relativo de $1{,}4 \cdot 10^{-6}$.

*Línea base.* Para $s = 0$, el óptimo con área fija cumple $\sinh(2\beta) = 6\beta$ con $\beta = mL$ (ejercicio 3.9), es decir, $\beta = 1{,}4192$. Con los datos, $L = 58{,}6$ mm, $t = 1{,}02$ mm y $q = 220{,}4$ W/m: unos 11 W para una aleta de 50 mm de profundidad.

*Método 1: sustituto neuronal.* Se supone que cada simulación es cara (en un diseño real sería un cálculo CFD conjugado de horas) y se dispone de 40. Se toman 60 puntos por hipercubo latino en $(L, s)$, se descartan los no fabricables y se simulan los 40 primeros factibles. Un perceptrón 2–32–32–1 con activación tanh, entrenado con 32 puntos, tiene un error cuadrático medio de 10,1 W/m (4,9 % de la media) en los 8 reservados. Reentrenado con los 40, su máximo en una rejilla factible de $201 \times 201$ está en $L = 62{,}4$ mm y $s = 0{,}815$, con predicción de 242,2 W/m; la simulación de verificación (la número 41) da 242,0 W/m.

*Método 2: simulador diferenciable.* El mismo simulador escrito con tensores de PyTorch (200 celdas, sistema denso) da $\partial q/\partial L$ y $\partial q/\partial s$ por diferenciación automática, que coinciden con diferencias finitas centradas en seis cifras. Ascenso de gradiente con Adam (módulo 7, §5.3) y *proyección* sobre la restricción de la punta, $s \le s_{\max}(L) = (1 - r)/(1 - r/2)$ con $r = t_{\min}L/A$, converge en 150 iteraciones a $L = 66{,}3$ mm y $s = 0{,}801$, con $q = 242{,}2$ W/m y la restricción activa (punta de 0,300 mm).

*Comprobación y conclusión.* Una búsqueda exhaustiva en $81 \times 96$ puntos (7776 simulaciones) da $q = 242{,}1$ W/m en $L = 66$ mm y $s = 0{,}80$. Los dos métodos llegan a menos del 0,1 % de ese

valor y mejoran la aleta rectangular óptima en un 9,8 % con la misma masa. El óptimo está sobre la restricción de fabricación: si se pudiera hacer la punta más fina, la aleta mejoraría hacia el perfil triangular. El sustituto encontró una solución algo distinta ($L = 62$ mm) con casi el mismo calor porque la función es muy plana cerca del óptimo; el método del gradiente necesita el simulador derivable, pero no un presupuesto de muestras.

| Herramienta | Para qué sirve | Licencia | Cuándo elegirla |
|---|---|---|---|
| JAX | derivación, vectorización y compilación de código NumPy | Apache-2.0 | simuladores nuevos escritos en estilo funcional |
| NVIDIA Warp | núcleos de simulación derivables en GPU | Apache-2.0 | física de partículas, telas o sólidos en GPU |
| Taichi (DiffTaichi) | lenguaje para simulación física derivable | Apache-2.0 | simuladores de partículas y rejillas de alto rendimiento |
| PyTorch | tensores y diferenciación automática | BSD-3 (con componentes de otras licencias) | simuladores pequeños unidos a redes neuronales |
| NVIDIA PhysicsNeMo | PINN, operadores neuronales y GNN para física | Apache-2.0 | sustitutos de CFD o de estructuras a escala industrial |
| scikit-learn | MLP, procesos gaussianos, validación | BSD-3 | sustitutos de pocas variables con pocos datos |
| dolfin-adjoint | adjuntos automáticos de modelos FEniCS | LGPL | EDP generales con elementos finitos |

Cuadro 3.1: Tecnologías del tema 3. Licencias según los metadatos publicados por cada proyecto.

*Observación* 3.5 (Errores frecuentes en la práctica). (i) *Sobreajuste del sustituto*: con 40 puntos y miles de pesos, la red interpola el ruido; hay que reservar datos de validación y regularizar. (ii) *Extrapolación*: el optimizador lleva el diseño a los bordes del dominio, donde hay menos datos; se verifica siempre con el simulador (proposición 3.3). (iii) *Fugas de datos*: si los puntos de validación son casi idénticos a los de entrenamiento (simulaciones repetidas con otra malla, por ejemplo), el error de validación es optimista. (iv) *La verdad tampoco es exacta*: el simulador tiene error de discretización; hay que comprobar la convergencia de malla antes de entrenar con él. (v) *Validez del modelo físico*: el caso supone aleta delgada; con $ht/k$ grande o radiación apreciable, el modelo unidimensional no vale. (vi) *Restricciones de seguridad*: un sustituto puede predecir un diseño factible que no lo es; las restricciones críticas se comprueban con el simulador o con un margen. (vii) *Coste mal contado*: el entrenamiento del sustituto y las simulaciones de verificación forman parte del presupuesto.

### Lecturas recomendadas para este tema

- M. J. Kochenderfer y T. A. Wheeler [12]: caps. «Sampling Plans», «Surrogate Models», «Probabilistic Surrogate Models» y «Surrogate Optimization».
- S. G. Johnson [7]: adjuntos para relaciones de recurrencia (MIT 18.335J).
- Y. Hu *et al.* [16]: programación diferenciable para simulación física.
- M. Raissi, P. Perdikaris y G. E. Karniadakis [33]; Z. Li *et al.* [25]; T. Pfaff *et al.* [31].

– F. P. Incropera *et al.* [11], cap. 3: conducción unidimensional y superficies extendidas.

## Ejercicios del tema 3

**Ejercicio 3.1** ⋆ Para cada caso, elija entre sustituto, simulador diferenciable o diferencias finitas y justifíquelo: (a) 3 parámetros y un CFD comercial de cajas negras que tarda 6 horas; (b) 5000 densidades y un código de elementos finitos propio; (c) un simulador de choques con contactos y 20 parámetros.

**Ejercicio 3.2** ⋆ Escriba la pérdida de una red informada por la física $\theta_\varphi(x)$ para la aleta rectangular del ejemplo 3.4: identifique el término de residuo y los de condiciones de contorno, y explique por qué no hace falta malla.

**Ejercicio 3.3** ⋆⋆ Para una aleta rectangular de $L = 40$ mm y $t = 1{,}5$ mm ($A = 60$ mm$^2$), con los datos del ejemplo 3.4, calcule $m$, $mL$, el calor $q$, la eficiencia $\eta = \tanh(mL)/(mL)$ y el número de Biot $ht/(2k)$.

*Fuente:* adaptado de F. P. Incropera *et al.*, *Fundamentals of Heat and Mass Transfer*, cap. 3 (aletas de sección uniforme) [11].

**Ejercicio 3.4** ⋆⋆ La viga MBB tiene 1200 variables y cada resolución de $KU = F$ tarda 10 ms. Compare el tiempo de calcular el gradiente por diferencias finitas hacia delante y por el método adjunto, y el de 100 iteraciones de optimización con cada uno.

**Ejercicio 3.5** ⋆⋆ Sea $R(u, \theta) = A(\theta)u - b$ con $A(\theta) = \left(\begin{smallmatrix} 2+\theta & -1 \\ -1 & 2 \end{smallmatrix}\right)$, $b = (1, 0)^\top$ y $J = u_1 + u_2$. En $\theta = 0$, calcule $u$, el vector adjunto $\lambda$ y $dJ/d\theta$, y compruébelo con una diferencia finita de paso 0,01.

**Ejercicio 3.6** ⋆⋆ Se simula la aleta rectangular del ejemplo 3.4 en $L = 30$, 60 y 90 mm, con resultados $q = 165{,}38$; 220,34 y 198,20 W/m. Ajuste un sustituto cuadrático $\hat{q}(L) = \beta_0 + \beta_1 L + \beta_2 L^2$ ($L$ en mm), calcule su máximo y compárelo con el óptimo verdadero ($L = 58{,}6$ mm, $q = 220{,}42$ W/m), sabiendo que el simulador da 218,13 W/m en el máximo del sustituto.

*Fuente:* adaptado de M. J. Kochenderfer y T. A. Wheeler, *Algorithms for Optimization*, cap. «Surrogate Models» [12].

**Ejercicio 3.7** ⋆⋆ Un componente se enfría según $\theta_{k+1} = (1 - a\Delta t)\,\theta_k$ con $\theta_0 = 50$ K, $\Delta t = 1$ s y $a = 0{,}1$ s$^{-1}$. Calcule $\theta_3$ y $d\theta_3/da$ derivando la expresión cerrada y con la recurrencia adjunta hacia atrás.

*Fuente:* adaptado de S. G. Johnson, «Adjoint methods and sensitivity analysis for recurrence relations», notas de MIT 18.335J (otoño de 2010) [7].

**Ejercicio 3.8** ⋆⋆⋆ (a) Demuestre la proposición 3.3. (b) Construya un ejemplo con $\Theta = [0, 1]$ en el que el sustituto tenga error nulo en tres puntos de validación y, sin embargo, su minimizador sea pésimo para $s$. ¿Qué enseña sobre la validación de sustitutos?

**Ejercicio 3.9** ⋆⋆⋆ (a) Demuestre que, para la aleta rectangular de área de perfil fija $A$, el calor $q(L) = \sqrt{2hkA/L}\,\theta_b \tanh\!\big(c\,L^{3/2}\big)$, con $c = \sqrt{2h/(kA)}$, es máximo cuando $\beta = cL^{3/2}$ cumple $\sinh(2\beta) = 6\beta$. (b) Con $h = 40$ W/(m$^2\cdot$K), $k = 170$ W/(m$\cdot$K), $A = 50$ mm$^2$ y $\theta_b = 50$ K, calcule $L$, $t$ y $q$ óptimos, usando $\beta = 1{,}4192$.

Tema 4

# Renderizado diferenciable y reconstrucción 3D

Reconstruir un objeto a partir de fotografías es un problema inverso: se busca la geometría (y la apariencia) cuyas imágenes renderizadas coinciden con las observadas. Si el renderizador es derivable, el problema se resuelve por descenso de gradiente, igual que el entrenamiento de una red. Este tema explica por qué la rasterización clásica no es derivable, cómo la suavizan SoftRas [26] y las bibliotecas actuales [34, 21, 42], y cómo NeRF [28] y el *Gaussian splatting* [20] convierten la reconstrucción de escenas en una optimización. Se apoya en las clases «Graphics Pipeline and Rasterization» de MIT 6.837 [6], en la clase de renderizado neuronal de CS348n [4] y en las tareas 2 y 3 del curso 16-825 de Carnegie Mellon [2].

## 4.1 Formación de la imagen y rasterización

Una cámara estenopeica con matriz de rotación $W$, centro $o$ y distancia focal $\phi$ (en píxeles) proyecta el punto $X$ en el píxel $\left(\phi\, X_c^{(1)}/X_c^{(3)},\ \phi\, X_c^{(2)}/X_c^{(3)}\right)$, con $X_c = W(X - o)$. La rasterización recorre los triángulos proyectados y asigna a cada píxel el triángulo más cercano que lo cubre (*z-buffer*) [6]. La silueta de una malla es la función indicadora de los píxeles cubiertos.

**Proposición 4.1 (La rasterización tiene gradiente nulo casi en todas partes).** *Sea $I_p(\theta) \in \{0, 1\}$ el valor de la silueta en el píxel $p$ (muestreado en su centro) como función de las coordenadas $\theta$ de los vértices. Entonces $I_p$ es constante a trozos: en todo $\theta$ en que ningún borde de triángulo proyectado pasa exactamente por el centro de $p$, $\nabla_\theta I_p = 0$.*

*Demostración.* La pertenencia del centro del píxel a un triángulo proyectado se decide con los signos de tres funciones de arista, continuas en $\theta$. Si ninguna se anula, sus signos se conservan en un entorno de $\theta$ y la cobertura no cambia, así que $I_p$ es constante en ese entorno. Los puntos en que alguna se anula forman un conjunto de medida nula. ∎

Una pérdida que compare siluetas rasterizadas tiene, por tanto, gradiente nulo salvo en un conjunto de medida nula, donde no existe: el descenso de gradiente no recibe información. Hay dos soluciones: suavizar el renderizador, o calcular explícitamente la contribución de los bordes (como hacen los métodos de antialias derivable de nvdiffrast [21] y los renderizadores físicos como Mitsuba 3 [42]).

## 4.2 Rasterización suave

**Definición 4.2 (Mapa de probabilidad de SoftRas).** Para el píxel $p_i$ y el triángulo proyectado $f_j$, SoftRas [26, §3.2] define

$$D_j^i = \operatorname{sig}\Big(\delta_j^i \, \frac{d^2(i,j)}{\sigma}\Big),$$

donde $d(i,j)$ es la distancia del píxel al borde del triángulo, $\delta_j^i = +1$ si el píxel está dentro y $-1$ si está fuera, y $\sigma > 0$ controla la nitidez. La silueta suave se obtiene agregando los triángulos como sucesos independientes [26, §3.3],

$$I_s^i = 1 - \prod_j \big(1 - D_j^i\big).$$

Cuando $\sigma \to 0$, $D_j^i$ tiende a la indicadora del triángulo y se recupera la rasterización clásica; con $\sigma > 0$, todos los triángulos cercanos reciben gradiente, incluso los ocultos y los vértices alejados del borde. El valor por defecto en el artículo es $\sigma = 10^{-4}$ en coordenadas de imagen normalizadas. La pérdida de siluetas más usada es $1 - \mathrm{IoU}$ suave, con $\mathrm{IoU}(S,T) = \sum_i S_i T_i / \sum_i (S_i + T_i - S_i T_i)$ [26, §4.1]. El compromiso es claro: un $\sigma$ grande da gradientes en una región amplia, útiles al principio, pero sesga la silueta (la «hincha»); un $\sigma$ pequeño es fiel pero casi no da gradiente. De ahí la práctica de *recocer* $\sigma$ de grande a pequeño, que se ve en el caso de estudio.

Las bibliotecas actuales implementan estas ideas con eficiencia. PyTorch3D proporciona estructuras de mallas y nubes por lotes, pérdidas 3D y un renderizador derivable modular [34]. nvdiffrast ofrece primitivas de alto rendimiento (rasterización, interpolación de atributos, filtrado de texturas y antialias) que aprovechan el hardware gráfico y se integran en PyTorch o TensorFlow [21]. Mitsuba 3, construido sobre el compilador Dr.Jit, calcula derivadas de simulaciones completas de transporte de luz respecto a la cámara, la geometría, los materiales o las texturas [42].

## 4.3 Renderizado volumétrico y NeRF

Un campo de radiancia neuronal (NeRF) representa una escena con una red que, para cada punto $X$ y dirección de vista $d$, devuelve una densidad $\sigma(X) \geq 0$ y un color $c(X,d)$ [28]. El color de un rayo $r(t) = o + td$ es

$$C(r) = \int_{t_n}^{t_f} T(t)\, \sigma(r(t))\, c(r(t), d)\, dt, \qquad T(t) = \exp\Big(-\int_{t_n}^{t} \sigma(r(u))\, du\Big),$$

donde $T(t)$ es la transmitancia, la probabilidad de que la luz recorra el rayo hasta $t$ sin ser absorbida. Con muestras $t_1 < \cdots < t_N$ y $\delta_i = t_{i+1} - t_i$, la cuadratura de NeRF [28, §4] es

$$\hat{C}(r) = \sum_{i=1}^{N} w_i\, c_i, \qquad w_i = T_i\, \alpha_i, \qquad \alpha_i = 1 - e^{-\sigma_i \delta_i}, \qquad T_i = \exp\Big(-\sum_{j<i} \sigma_j \delta_j\Big) = \prod_{j<i} (1 - \alpha_j).$$

**Proposición 4.3 (Los pesos del rayo).** *Con $\sigma_i \geq 0$, los pesos cumplen $w_i \geq 0$ y $\sum_{i=1}^{N} w_i = 1 - T_{N+1} \leq 1$, donde $T_{N+1} = \prod_{j \leq N}(1 - \alpha_j)$ es la transmitancia residual.*

*Demostración.* Como $\sigma_i\delta_i \geq 0$, se tiene $\alpha_i \in [0,1)$ y $T_i \in (0,1]$, luego $w_i \geq 0$. Además $w_i = T_i\alpha_i = T_i - T_i(1-\alpha_i) = T_i - T_{i+1}$, y la suma es telescópica: $\sum_{i=1}^{N}(T_i - T_{i+1}) = T_1 - T_{N+1} = 1 - T_{N+1}$. ∎

Los pesos forman, pues, una distribución de probabilidad (completada con el fondo, de peso $T_{N+1}$) sobre el punto del rayo en que la luz se emite; la profundidad esperada es $\sum_i w_i t_i$. El renderizado es derivable respecto a $\sigma_i$ y $c_i$, y por tanto respecto a los pesos de la red. NeRF aplica a las coordenadas una codificación posicional $\gamma(p) = \big(\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)\big)$ con $L = 10$ para la posición y $L = 4$ para la dirección, una red de 8 capas de 256 unidades, y minimiza el error cuadrático entre colores renderizados y observados con lotes de 4096 rayos; cada escena necesita entre $10^5$ y $3 \cdot 10^5$ iteraciones, uno o dos días en una GPU V100 [28, §5].

## 4.4 *Gaussian splatting*

El *Gaussian splatting* 3D [20] sustituye la red por un conjunto explícito de gaussianas anisótropas, cada una con media $\mu$, covarianza $\Sigma$, opacidad $o$ y color dependiente de la vista (coeficientes de armónicos esféricos). Para que la covarianza siga siendo válida durante la optimización se parametriza como

$$\Sigma = R\,S\,S^{\top}R^{\top},$$

con $R$ la rotación de un cuaternión normalizado y $S$ diagonal de escalas [20, §4]. Cada gaussiana se proyecta a la imagen con la covarianza $\Sigma' = JW\Sigma W^{\top}J^{\top}$, donde $J$ es el jacobiano de la aproximación afín de la proyección, y las gaussianas que cubren un píxel, ordenadas por profundidad, se componen con la misma fórmula que NeRF:

$$C = \sum_i c_i\,\alpha_i \prod_{j<i}(1-\alpha_j), \qquad \alpha_i = o_i \exp\Big(-\tfrac{1}{2}(p-\mu_i')^{\top}\Sigma_i'^{-1}(p-\mu_i')\Big).$$

La pérdida combina $L^1$ y D-SSIM con peso $\lambda = 0{,}2$; un control adaptativo clona o divide gaussianas donde el gradiente de posición es grande y elimina las casi transparentes, y un rasterizador por teselas de $16\times16$ píxeles con ordenación por radix en GPU permite renderizar en tiempo real (al menos 30 fotogramas por segundo a 1080p) escenas de uno a cinco millones de gaussianas [20]. Frente a NeRF, el *splatting* cambia una representación implícita y compacta, pero lenta de muestrear, por una explícita, rápida y más voluminosa en memoria.

## 4.5 Caso de estudio: ajuste de una forma a siluetas

**Ejemplo 4.4 (Reconstrucción de una pieza a partir de seis siluetas).** *Contexto.* Se quiere reconstruir una pieza a partir de fotografías segmentadas (máscaras binarias), por ejemplo para verificar una pieza de fundición o crear un recurso digital. La pieza cabe en un cubo de 40 mm de lado y es estrellada respecto a su centro: su superficie es $\{\rho\,\hat{x} : \rho = r(\hat{x})\}$ con

$$r(\hat{x}) = \exp\Big(\sum_{\ell=0}^{3}\sum_{m=-\ell}^{\ell} c_{\ell m}\,Y_{\ell m}(\hat{x})\Big),$$

donde $Y_{\ell m}$ son los armónicos esféricos reales y $\theta = (c_{\ell m}) \in \mathbb{R}^{16}$. La exponencial garantiza $r > 0$. Hay 6 vistas ortográficas de orientación conocida y aleatoria, de $40 \times 40$ píxeles, con 40 muestras por rayo (unidad de longitud: 20 mm).

*Renderizador suave.* La ocupación de un punto $X$ es $o(X) = \mathrm{sig}\big((r(\hat{x}) - \|X\|)/\tau\big)$, una versión suave de la indicadora de la pieza, y la silueta de un rayo con muestras $X_1, \ldots, X_{40}$ es $S = 1 - \prod_k \big(1 - o(X_k)\big)$, la agregación de SoftRas (definición 4.2). La base de armónicos de cada muestra se precalcula, de modo que una pasada cuesta un producto de una matriz de $6 \cdot 64\,000$ filas por $\theta$.

*Formulación.* $\min_\theta \; 1 - \mathrm{IoU}\big(S(\theta;\tau), T\big)$, con $T$ las seis máscaras binarias; se optimiza con Adam (tasa 0,03, 300 iteraciones) desde una esfera de radio 8 mm.

*Elección del método.* El gradiente de la pérdida respecto a los 16 coeficientes atraviesa la sigmoide de cada muestra; con $\tau$ fijo y pequeño, solo las muestras muy cercanas al contorno aportan, y con $\tau$ grande la silueta suave queda sesgada. Se compara $\tau = 0{,}03$ fijo con un recocido geométrico de $\tau$ desde 0,05 hasta 0,005 (de 1 mm a 0,1 mm).

*Resultado.* Con $\tau$ fijo, la IoU de las siluetas binarias es 0,827 y la IoU volumétrica (medida en una rejilla de $64^3$) es 0,744: la silueta suave de cada rayo acumula muchas muestras con ocupación intermedia y se hincha, y el optimizador compensa encogiendo la pieza. Con recocido, la IoU de siluetas es 0,995 y la volumétrica 0,991, y los coeficientes se recuperan con un error relativo inferior al 1 %. Cada ajuste tarda unos 4 s en una CPU.

*Límite fundamental.* Las siluetas solo determinan el *casco visual* del objeto, la intersección de los conos de visión de sus siluetas [23]: una concavidad que no se ve de perfil desde ninguna cámara es invisible. Aquí la recuperación es casi perfecta porque la familia de formas (16 coeficientes, armónicos de grado bajo) es pequeña; con formas generales hacen falta términos fotométricos o de profundidad.

| Herramienta | Para qué sirve | Licencia | Cuándo elegirla |
|---|---|---|---|
| PyTorch3D | rasterización suave de mallas y nubes, pérdidas 3D | BSD | ajuste de mallas e investigación en PyTorch |
| nvdiffrast | primitivas de rasterización derivable de alto rendimiento | licencia de código de NVIDIA | mallas grandes, texturas y antialias en GPU NVIDIA |
| Mitsuba 3 | renderizado físico (transporte de luz) derivable | BSD-3 (con cláusula adicional) | materiales, iluminación indirecta, medios participantes |
| Kaolin | conversiones y renderizado derivable | Apache-2.0 | ecosistema NVIDIA |
| nerfstudio | entrenamiento modular de NeRF | Apache-2.0 | reconstrucción de escenas con varias variantes de NeRF |
| gsplat | rasterizador de gaussianas | Apache-2.0 | *Gaussian splatting* con licencia permisiva |
| 3DGS original (Inria) | implementación de referencia de [20] | solo uso no comercial | investigación y comparación |

Cuadro 4.1: Tecnologías del tema 4, con las licencias publicadas en sus repositorios.

*Observación* 4.5 (Errores frecuentes en la práctica). (i) *Poses de cámara erróneas*: un error de calibración de un grado se paga con geometría deformada; las poses deben estimarse bien (o refinarse en la optimización). (ii) *Ambigüedad*: siluetas sin información fotométrica no ven las concavidades (casco visual). (iii) *Mínimos locales y sesgo del suavizado*: sin recocer $\sigma$ o $\tau$, el ajuste se atasca o converge sesgado, como en el ejemplo 4.4. (iv) *Fuga de vistas*: evaluar

NeRF o *splatting* en las mismas imágenes con que se entrenó mide memorización; se reservan vistas de prueba. (v) *Máscaras de mala calidad*: los errores de segmentación se convierten en geometría. (vi) *Licencias*: algunas implementaciones de referencia (3DGS de Inria, Instant-NGP, nvdiffrast) imponen restricciones de uso que impiden usarlas en un producto comercial sin permiso.

**Lecturas recomendadas para este tema**

- MIT 6.837 [6]: clases «Graphics Pipeline and Rasterization» I y II.
- S. Liu *et al.* [26]: §3.2 (mapas de probabilidad), §3.3 (agregación) y §4.1 (pérdida de siluetas).
- B. Mildenhall *et al.* [28]: §4 (renderizado volumétrico) y §5 (codificación posicional y muestreo jerárquico).
- B. Kerbl *et al.* [20]: §4–6.
- CMU 16-825 [2]: tarea 3, «Volume Rendering, NeRF, and 3D Gaussian Splatting».

## Ejercicios del tema 4

**Ejercicio 4.1** ⋆ Explique con la proposición 4.1 por qué no se puede ajustar una malla a siluetas con un rasterizador clásico y una pérdida de píxeles, y qué compromiso introduce el parámetro $\sigma$ de SoftRas.

**Ejercicio 4.2** ⋆ Un objeto es una taza con el asa hacia la derecha. Se dispone de 50 siluetas tomadas alrededor de ella. ¿Puede reconstruirse el interior de la taza? Razone con el concepto de casco visual y proponga qué información adicional lo permitiría.

**Ejercicio 4.3** ⋆⋆ Con $\sigma = 10^{-4}$ (coordenadas normalizadas), calcule $D$ para un píxel dentro de un triángulo a distancia 0,01 del borde, para uno fuera a distancia 0,01 y para uno fuera a distancia 0,02. Calcule la silueta agregada de un píxel cubierto por dos triángulos con $D = 0{,}7311$ y $D = 0{,}2689$.

**Ejercicio 4.4** ⋆⋆ Un rayo tiene tres muestras con $\delta_i = 0{,}5$, densidades $\sigma = (0{,}2;\ 1{,}0;\ 3{,}0)$, colores (en gris) $c = (0{,}1;\ 0{,}8;\ 0{,}4)$ y profundidades $t = (1{,}0;\ 1{,}5;\ 2{,}0)$. Calcule $\alpha_i$, $T_i$, los pesos, el color y la profundidad renderizados, y compruebe la proposición 4.3.

*Fuente:* adaptado de CMU 16-825, «Learning for 3D Vision» (S. Tulsiani), tarea 3, §1.5 «Volume rendering» [2].

**Ejercicio 4.5** ⋆⋆ Tres gaussianas proyectadas, ya ordenadas por profundidad, cubren un píxel con opacidades $o = (0{,}8;\ 0{,}6;\ 0{,}9)$ y exponentes $P = -\frac{1}{2}(p-\mu')^\top \Sigma'^{-1}(p-\mu') = (-0{,}5;\ -0{,}1;\ -2{,}0)$. Sus colores son rojo, verde y azul puros. Calcule $\alpha_i = o_i e^{P_i}$, las transmitancias y el color del píxel.

*Fuente:* adaptado de CMU 16-825, «Learning for 3D Vision» (S. Tulsiani), tarea 3, §5.4 «Compute Alphas and Transmittance» [2].

**Ejercicio 4.6** ⋆⋆ Calcule $\gamma(p)$ para $p = 0{,}25$ con $L = 2$. ¿Cuántas componentes tiene la codificación de una posición 3D con $L = 10$ y la de una dirección con $L = 4$?

**Ejercicio 4.7** ⋆⋆ Una silueta suave de cuatro píxeles vale $S = (0{,}9;\ 0{,}6;\ 0{,}2;\ 0)$ y la máscara objetivo es $T = (1;\ 1;\ 0;\ 0)$. Calcule la IoU suave, la pérdida $1 - \text{IoU}$ y la derivada de la IoU respecto a $S_3$. Interprete el signo.

**Ejercicio 4.8** ⋆⋆⋆ (a) Demuestre la proposición 4.3. (b) Demuestre que si se duplica el número de muestras partiendo cada intervalo en dos mitades con la misma densidad, los pesos de las dos mitades suman el peso original, de modo que la cuadratura es coherente al refinar. (c) Deduzca que la composición de gaussianas del *splatting* es la misma fórmula con $\alpha_i$ dado.

**Ejercicio 4.9** ⋆⋆⋆ (a) Demuestre que $\Sigma = RSS^{\top}R^{\top}$, con $R$ ortogonal y $S = \operatorname{diag}(s_1, s_2, s_3)$, es simétrica y semidefinida positiva, y que es definida positiva si y solo si todas las $s_i$ son no nulas. (b) Demuestre que $\Sigma' = JW\Sigma W^{\top}J^{\top}$ es semidefinida positiva para cualesquiera $J$ y $W$. (c) Explique por qué optimizar directamente las seis entradas de una matriz simétrica $\Sigma$ con descenso de gradiente puede producir matrices no válidas.

Tema 5

# Optimización de pipelines de software 3D y videojuegos

En producción, la geometría no se optimiza solo para que una pieza sea rígida o una reconstrucción fiel, sino para que una escena quepa en un presupuesto: milisegundos por fotograma, megabytes de memoria de vídeo, tamaño de descarga. Este tema formula como problemas de optimización las decisiones habituales de un pipeline 3D: cuántos triángulos conservar al simplificar una malla, qué nivel de detalle dibujar, cómo comprimir texturas y cómo empaquetar sus mapas UV. Sigue el artículo de Garland y Heckbert sobre el error cuádrico [17], las clases de simplificación de UCSD [8] y Princeton [3], la presentación de Nanite de Epic Games [19], la documentación de compresión por bloques de Microsoft [44], el artículo de mapas conformes por mínimos cuadrados [24] y la API de Python de Blender [40].

## 5.1 El presupuesto de rendimiento como restricción

**Definición 5.1 (Presupuesto por fotograma).** A una frecuencia objetivo de $\nu$ fotogramas por segundo corresponde un presupuesto de $1000/\nu$ ms por fotograma: 33,3 ms a 30 Hz, 16,7 ms a 60 Hz, 11,1 ms a 90 Hz (habitual en realidad virtual). Si la CPU prepara el fotograma $k{+}1$ mientras la GPU dibuja el $k$, el tiempo por fotograma es aproximadamente $\max(t_{\text{CPU}}, t_{\text{GPU}})$, y cada uno debe caber en el presupuesto.

El presupuesto se reparte entre sistemas (geometría, iluminación, posprocesado, física, lógica) y cada partida actúa como una restricción $\sum_j t_j(\theta) \le B$ de un problema de optimización cuya función objetivo es la calidad visual. Dos detalles importan en la práctica. Primero, la restricción debe imponerse sobre un percentil alto (por ejemplo, el 99) del tiempo por fotograma y no sobre la media, porque los tirones que percibe el jugador son los fotogramas lentos. Segundo, los costes deben medirse en el hardware de destino: el modelo *roofline* (módulo 17, §2.2) explica por qué una misma reducción de triángulos ahorra mucho cuando la GPU está limitada por el procesamiento de vértices y casi nada cuando lo está por el sombreado de píxeles.

## 5.2 Simplificación de mallas por error cuádrico

La simplificación reduce el número de triángulos colapsando aristas: los dos extremos se sustituyen por un único vértice y desaparecen las caras degeneradas. La cuestión es qué arista colapsar y dónde colocar el nuevo vértice. Garland y Heckbert [17] lo resuelven con una medida de error que se acumula sumando matrices.

**Definición 5.2 (Cuádrica de error).** Sea un plano $n^\top x + d = 0$ con $\|n\| = 1$ y sea $p = (n, d) \in \mathbb{R}^4$. Su *cuádrica fundamental* es $K_p = pp^\top$. La cuádrica de un vértice es $Q_v = \sum_p K_p$,

sumando sobre los planos de las caras que lo contienen, y el error de colocar el vértice en $v$ es $\Delta(v) = \bar{v}^\top Q \bar{v}$, con $\bar{v} = (v, 1)$. Al colapsar $(v_1, v_2)$ se toma $Q = Q_1 + Q_2$.

**Proposición 5.3 (Propiedades del error cuádrico).** *(a) $\Delta(v) = \sum_p (n_p^\top v + d_p)^2$ es la suma de los cuadrados de las distancias de $v$ a los planos. (b) $Q$ es simétrica y semidefinida positiva, y $\Delta$ es una función convexa de $v$. (c) Si $Q = \left(\begin{smallmatrix} A & b \\ b^\top & c \end{smallmatrix}\right)$ con $A \in \mathbb{R}^{3\times 3}$ invertible, el minimizador es $v^\star = -A^{-1}b$, solución del sistema lineal que se obtiene al sustituir la última fila de $Q$ por $(0, 0, 0, 1)$; $A$ es invertible si y solo si las normales de los planos generan $\mathbb{R}^3$.*

*Demostración.* (a) $\bar{v}^\top p p^\top \bar{v} = (p^\top \bar{v})^2 = (n^\top v + d)^2$, que es la distancia al cuadrado al plano porque $\|n\| = 1$; sumando sobre los planos se obtiene la fórmula. (b) Cada $pp^\top$ es simétrica y $y^\top p p^\top y = (p^\top y)^2 \geq 0$; la suma conserva ambas propiedades. $\Delta(v) = v^\top A v + 2b^\top v + c$ con $A = \sum_p n_p n_p^\top$ semidefinida positiva, luego es convexa (módulo 4, tema 5). (c) El gradiente $2Av + 2b$ se anula en $v^\star = -A^{-1}b$, que es mínimo global por convexidad. Las tres primeras filas del sistema modificado son $Av + b = 0$ y la cuarta fija la coordenada homogénea a 1. Por último, $y^\top A y = \sum_p (n_p^\top y)^2 = 0$ si y solo si $y$ es ortogonal a todas las normales, así que $A$ es definida positiva (e invertible) si y solo si las normales generan $\mathbb{R}^3$. ∎

**Ejemplo 5.4 (Una esquina).** Un vértice en el origen con tres caras en los planos $x = 0$, $y = 0$ y $z = 0$ tiene $A = I$ y $b = 0$: su posición óptima es el origen, con error nulo, y moverlo a (0,1; 0,2; 0,3) cuesta $0{,}01 + 0{,}04 + 0{,}09 = 0{,}14$. Si solo hubiera dos planos, $x = 0$ y $z = 0$, $A$ sería singular: todo el eje $y$ tendría error nulo, y el algoritmo recurre a evaluar los extremos y el punto medio de la arista [17, §4].

El algoritmo calcula las cuádricas iniciales, el coste $\Delta(v^\star)$ de cada arista candidata y las guarda en un montículo; en cada paso colapsa la de menor coste y actualiza las vecinas [17, §4.1]. Cada vértice guarda solo 10 números (la matriz simétrica), y el artículo informa de que una malla de 69 451 caras se simplifica en unos 15 s en el hardware de 1997. La simplificación por colapso de aristas es la que ofrecen el modificador *Decimate* de Blender en modo `COLLAPSE` [40] y la biblioteca meshoptimizer [43], y Nanite usa expresamente el error cuádrico [19]. En el cuaderno, una esfera de 20 mm de radio con 528 caras se reduce a 200 caras con una desviación máxima de los vértices de 0,30 mm, conservando $\chi = 2$.

## 5.3 Niveles de detalle y error en pantalla

Un objeto lejano ocupa pocos píxeles, y dibujarlo con todos sus triángulos desperdicia tiempo. Los *niveles de detalle* (LOD) son versiones simplificadas que se eligen según la distancia.

**Proposición 5.5 (Error proyectado).** *Con una cámara perspectiva de campo de visión vertical $\phi_v$ y una imagen de $H$ píxeles de alto, un error geométrico $\varepsilon$ a distancia $d$ ocupa aproximadamente*

$$\varepsilon_{px} = \frac{\varepsilon\, H}{2\, d\, \tan(\phi_v/2)}$$

*píxeles. En consecuencia, el nivel con error $\varepsilon$ puede usarse sin superar un umbral de $\tau$ píxeles a partir de la distancia $d^\star = \varepsilon H / \big(2\tau \tan(\phi_v/2)\big)$.*

*Demostración.* A distancia $d$, la altura visible es $2d\tan(\phi_v/2)$ y se reparte entre $H$ píxeles, de modo que un píxel mide $2d\tan(\phi_v/2)/H$. Dividiendo $\varepsilon$ por ese tamaño se obtiene $\varepsilon_{\text{px}}$, que es decreciente en $d$; despejando $\varepsilon_{\text{px}} = \tau$ se obtiene $d^\star$. ∎

Nanite, el sistema de geometría virtualizada de Unreal Engine 5, lleva la idea al extremo [19]: divide cada malla en grupos de 128 triángulos, los simplifica jerárquicamente con el error cuádrico en un grafo acíclico dirigido de niveles, proyecta el error de simplificación a píxeles y, en cada fotograma, elige un corte del grafo en que el error queda por debajo de aproximadamente un píxel; los triángulos diminutos se rasterizan por software. Los autores informan de unos 25 millones de triángulos rasterizados por fotograma en su demostración.

## 5.4 Selección de niveles de detalle con presupuesto

**Definición 5.6 (Mochila de elección múltiple).** Hay $G$ grupos de objetos; el grupo $g$ admite los niveles $\ell = 0, \dots, L_g$, con coste $t_{g\ell}$ (ms) y pérdida de calidad $q_{g\ell}$. Se elige un nivel por grupo, $e = (\ell_1, \dots, \ell_G)$, resolviendo

$$\min_{e} \; Q(e) = \sum_{g} q_{g\ell_g} \quad \text{sujeto a} \quad T(e) = \sum_{g} t_{g\ell_g} \le B.$$

Es un problema NP-difícil en general, pero pequeño en la práctica. La *relajación lagrangiana* elimina la restricción: para cada $\lambda \ge 0$ se resuelve $\min_e \big[Q(e) + \lambda(T(e) - B)\big]$, que se separa por grupos (cada grupo elige el nivel que minimiza $q_{g\ell} + \lambda t_{g\ell}$). El multiplicador $\lambda$ es el «precio» de un milisegundo en unidades de calidad.

**Proposición 5.7 (Dualidad débil).** *Sea $D(\lambda) = \min_e \big[Q(e) + \lambda(T(e) - B)\big]$ y $Q^\star$ el óptimo del problema con restricción. Para todo $\lambda \ge 0$, $D(\lambda) \le Q^\star$. Si la elección $e_\lambda$ que alcanza $D(\lambda)$ es factible y $\lambda\big(T(e_\lambda) - B\big) = 0$, entonces $e_\lambda$ es óptima.*

*Demostración.* Sea $e^\star$ óptima. Como $T(e^\star) \le B$ y $\lambda \ge 0$, $D(\lambda) \le Q(e^\star) + \lambda(T(e^\star) - B) \le Q(e^\star) = Q^\star$. Si $e_\lambda$ es factible, $Q(e_\lambda) \ge Q^\star$; y con la condición de holgura complementaria, $Q(e_\lambda) = D(\lambda) \le Q^\star$. Luego $Q(e_\lambda) = Q^\star$. ∎

**Ejemplo 5.8 (Cuatro grupos de objetos en 4 ms).** *Contexto.* Un juego a 60 Hz (16,7 ms por fotograma) reserva 4,0 ms de GPU para la geometría de cuatro grupos. Cada grupo tiene cuatro niveles generados con QEM; los costes se midieron en el hardware de destino y la pérdida de calidad es el error en pantalla medio (px) ponderado por la importancia visual del grupo:

| Grupo | Peso | LOD0 | LOD1 | LOD2 | LOD3 |
|---|---|---|---|---|---|
| Árboles (400) | 1,0 | 3,20 / 0 | 1,70 / 0,6 | 0,90 / 1,5 | 0,45 / 4,0 |
| Rocas (800) | 0,5 | 2,40 / 0 | 1,30 / 0,4 | 0,70 / 1,2 | 0,35 / 3,0 |
| Edificios (60) | 1,5 | 1,80 / 0 | 1,00 / 0,3 | 0,55 / 0,9 | 0,30 / 2,5 |
| Personajes (20) | 3,0 | 1,20 / 0 | 0,70 / 0,5 | 0,40 / 1,6 | 0,25 / 4,5 |

(cada celda: ms / px; entre paréntesis, número de instancias).

*Formulación.* La de la definición 5.6 con $q_{g\ell} = w_g \cdot \mathrm{px}_{g\ell}$ y $B = 4{,}0$ ms. Todo en LOD0 cuesta 8,6 ms, más del doble del presupuesto.

*Métodos.* Con $4^4 = 256$ combinaciones, la enumeración es exacta e instantánea; la relajación lagrangiana escala a miles de grupos y da una cota inferior.

*Resultado.* De las 117 combinaciones factibles, la óptima es (LOD2, LOD2, LOD1, LOD0): árboles y rocas al nivel 2, edificios al 1 y personajes a máxima calidad, con 3,80 ms y pérdida

ponderada 2,55. La relajación lagrangiana alcanza su mejor cota, $D = 2{,}30$, en $\lambda = 1{,}25$, y la elección asociada coincide con el óptimo; el hueco de dualidad (0,25) existe porque la elección es discreta y la holgura no se anula ($T = 3{,}80 < 4$). En un motor, $\lambda$ se ajusta de un fotograma a otro según el tiempo medido, y la decisión por grupo se reduce a comparar $q_{g\ell} + \lambda t_{g\ell}$.

## 5.5 Compresión de texturas

Las GPU leen texturas comprimidas por bloques sin descomprimirlas en memoria. Los formatos BC de Direct3D [44] codifican bloques de $4 \times 4$ téxeles: BC1 usa 8 bytes por bloque (4 bits por téxel) para color RGB con alfa de un bit; BC4, 8 bytes para un canal; BC5, 16 bytes para dos canales; BC6H, 16 bytes para color HDR, y BC7, 16 bytes (8 bits por téxel) para RGB o RGBA de alta calidad, con ocho modos por bloque entre los que el codificador elige. Una cadena completa de mipmaps añade un tercio de memoria, porque $1 + \frac{1}{4} + \frac{1}{16} + \cdots = \frac{4}{3}$.

Codificar un bloque BC1 es un problema de optimización: se eligen dos colores extremos $e_0, e_1$ y, para cada téxel, un índice de dos bits que selecciona uno de los colores de la paleta $\{e_0,\ e_1,\ \frac{2}{3}e_0+\frac{1}{3}e_1,\ \frac{1}{3}e_0+\frac{2}{3}e_1\}$, minimizando el error cuadrático. Fijados los índices, el problema en $(e_0, e_1)$ es de mínimos cuadrados lineales (módulo 8, §2.2), porque cada téxel se aproxima por $(1 - w_i)e_0 + w_i e_1$ con $w_i \in \{0, \frac{1}{3}, \frac{2}{3}, 1\}$ conocido; fijados los extremos, cada índice se elige por cercanía. Alternar ambos pasos no aumenta nunca el error y termina en un número finito de pasos, como el algoritmo de las $k$-medias. La dirección inicial de $e_0e_1$ suele tomarse como la primera componente principal de los colores del bloque (módulo 3, §5.4).

## 5.6 Empaquetado UV

Para texturizar una malla se corta en *cartas* que se aplanan en el plano (parametrización) y se colocan dentro de un cuadrado de textura (empaquetado). El aplanado se plantea como minimización de una distorsión; el método LSCM de Lévy *et al.* [24] minimiza por mínimos cuadrados la desviación de la conformidad (preservación de ángulos), lo que lleva a un sistema lineal disperso. El empaquetado es una variante del problema de la mochila bidimensional: colocar rectángulos (o polígonos) sin solaparse maximizando el área útil o la densidad de téxeles uniforme, y es NP-difícil incluso para rectángulos. En la práctica se usan heurísticas voraces: por *estantes* (ordenar por altura y llenar filas), *skyline* o rectángulos máximos libres, con un margen (*padding*) entre cartas para que los mipmaps no mezclen colores de cartas vecinas. Herramientas como xatlas automatizan la segmentación, la parametrización y el empaquetado.

## 5.7 Contenido procedural y automatización con Blender

Un pipeline reproducible genera los niveles de detalle, las texturas y las comprobaciones con scripts, no a mano. Blender expone su modelo de datos en Python (`bpy`); el siguiente script crea tres niveles de detalle de cada malla de una escena con el modificador *Decimate* en modo de colapso de aristas [40]:

```
import bpy
ratios = {"LOD1": 0.5, "LOD2": 0.25, "LOD3": 0.1}
mallas = [o for o in bpy.data.objects if o.type == 'MESH']
for obj in mallas:
    for nombre, r in ratios.items():
        copia = obj.copy()
        copia.data = obj.data.copy()
        copia.name = obj.name + "_" + nombre
        bpy.context.collection.objects.link(copia)
        mod = copia.modifiers.new(name="Decimate", type='DECIMATE')
        mod.decimate_type = 'COLLAPSE'
        mod.ratio = r     # fracción de triángulos que se conserva
```

El mismo patrón sirve para el contenido procedural: un generador de rocas, edificios o vegetación es una función $G(\theta, \text{semilla})$ cuyos parámetros pueden ajustarse con optimización de caja negra contra métricas (número de triángulos, área de textura, tiempo de dibujado medido) o contra la valoración de un artista. Fijar las semillas y versionar los scripts hace que cada recurso sea reproducible, y permite integrar las comprobaciones del tema 6 en la integración continua.

| Herramienta | Para qué sirve | Licencia o tipo | Cuándo elegirla |
|---|---|---|---|
| meshoptimizer | simplificación, orden de caché de vértices, sobredibujado, cuantización | MIT | optimizar mallas para la GPU en C/C++ |
| Blender (`bpy`) | modelado, *Decimate*, scripts de pipeline | GPL-3.0 | automatizar LOD, exportaciones y comprobaciones |
| PyMeshLab | filtros de simplificación y reparación | GPL-3.0 | procesado por lotes fuera del motor |
| xatlas | segmentación, parametrización y empaquetado UV | MIT | atlas de texturas o de iluminación automáticos |
| DirectXTex | compresión BC1–BC7 y mipmaps | MIT | preparar texturas para Direct3D |
| Basis Universal | texturas supercomprimidas transcodificables | Apache-2.0 | distribuir una textura para varias plataformas |
| Unreal Engine (Nanite) | geometría virtualizada con LOD automático | comercial (licencia de Epic) | escenas con mallas de muy alta resolución |

Cuadro 5.1: Tecnologías del tema 5. Licencias según los repositorios o la documentación pública.

*Observación* 5.9 (Errores frecuentes en la práctica). (i) *Optimizar la media*: un presupuesto cumplido en promedio con picos de 40 ms produce tirones; se mide el percentil 99. (ii) *Medir en la máquina equivocada*: los costes de la mochila deben salir del hardware de destino. (iii) *Saltos visibles al cambiar de LOD* (*popping*): se mitigan con umbrales con histéresis o con transiciones graduales. (iv) *Costuras*: simplificar sin bloquear los bordes de las cartas UV o de las normales rompe la textura; meshoptimizer distingue la simplificación que respeta la topología y las costuras de la que no [43]. (v) *Formato de compresión inadecuado*: comprimir mapas de normales con BC1 introduce artefactos; para dos canales se usa BC5. (vi) *Empaquetado sin*

*margen*: los mipmaps mezclan cartas vecinas. (vii) *Sobreajuste a la escena de prueba*: un ajuste de presupuestos para una única cámara falla en el resto del nivel.

### Lecturas recomendadas para este tema

- M. Garland y P. S. Heckbert [17]: §3–5 (contracción de pares, cuádricas y su obtención).
- UCSD CSE 190 [8], clase 8, y Princeton COS 526 [3], «Mesh Simplification».
- B. Karis, R. Stubbe y G. Wihlidal [19]: *Nanite: A Deep Dive*.
- Microsoft [44]: *Texture Block Compression in Direct3D 11*.
- B. Lévy *et al.* [24]: mapas conformes por mínimos cuadrados.

## Ejercicios del tema 5

**Ejercicio 5.1** ⋆ Calcule el presupuesto por fotograma a 30, 60, 90, 120 y 144 Hz. Explique por qué un juego que cumple 16,7 ms de media puede verse mal a 60 Hz.

**Ejercicio 5.2** ⋆⋆ Un vértice en el origen pertenece a dos caras contenidas en los planos $z = 0$ y $x = 0$. (a) Escriba su cuádrica $Q$. (b) Calcule el error de moverlo a $(0{,}1;\ 0{,}2;\ 0{,}3)$. (c) ¿Tiene solución única el sistema de la posición óptima? Describa el conjunto de posiciones de error nulo y qué hace el algoritmo en este caso.

*Fuente:* adaptado de UCSD CSE 190 «Advanced Computer Graphics» (R. Ramamoorthi, primavera de 2015), clase 8 «Mesh simplification» [8].

**Ejercicio 5.3** ⋆⋆ Calcule la memoria (MiB) de un mapa de normales de $2048 \times 2048$ con cadena completa de mipmaps en RGBA8, en BC5 y en BC1. ¿Por qué se prefiere BC5 para normales?

**Ejercicio 5.4** ⋆⋆ Cuatro téxeles en escala de grises valen $(10,\ 40,\ 70,\ 95)$ y se les asignan los pesos de paleta $w = (0,\ \frac{1}{3},\ \frac{2}{3},\ 1)$. Calcule por mínimos cuadrados los extremos $e_0, e_1$, los valores reconstruidos y el error cuadrático total.

**Ejercicio 5.5** ⋆⋆ Un nivel de detalle tiene un error geométrico de 2 mm. Con campo de visión vertical de 60° y 1080 píxeles de alto, ¿a partir de qué distancia puede usarse con un error menor que 1 píxel? ¿Y con 2160 píxeles de alto?

**Ejercicio 5.6** ⋆⋆ Dos grupos, con presupuesto de 2,5 ms. Grupo 1 (peso 1): LOD0 2,0 ms / 0 px, LOD1 1,2 / 1, LOD2 0,5 / 3. Grupo 2 (peso 2): LOD0 1,5 / 0, LOD1 0,8 / 0,5, LOD2 0,4 / 2. (a) Encuentre la elección óptima por enumeración. (b) Calcule $D(\lambda)$ para $\lambda = 1{,}25$ y $\lambda = 10/7$ y compruebe la proposición 5.7.

**Ejercicio 5.7** ⋆⋆ Se empaquetan en un atlas de $1024 \times 1024$ cartas rectangulares de $512 \times 300$, $400 \times 300$, $300 \times 280$, $600 \times 250$, $350 \times 240$ y $500 \times 200$ (anchura por altura, en téxeles) con la heurística de estantes: ordenar por altura decreciente y llenar filas de izquierda a derecha. ¿Caben? ¿Qué fracción del atlas se aprovecha?

**Ejercicio 5.8** ⋆⋆⋆ Demuestre la proposición 5.3. Demuestre además que la cuádrica acumulada $Q_1 + Q_2$ mide la suma de distancias al cuadrado a la *unión* de los planos de ambos vértices, y explique por qué el error de un colapso nunca es menor que el mayor de los errores que habría tenido cada vértice por separado en esa posición.

*Fuente:* adaptado de Princeton COS 526 (T. Funkhouser, otoño de 2006), «Mesh Simplification» [3], y de M. Garland y P. S. Heckbert [17, §4–5].

**Ejercicio 5.9** ⋆⋆⋆ Demuestre la proposición 5.7 y demuestre que $D(\lambda)$ es una función cóncava de $\lambda$. Explique por qué en el ejemplo 5.8 la cota no alcanza el óptimo.

Tema 6

# IA generativa para 3D y validación de la geometría

Los modelos generativos producen ya objetos 3D a partir de un texto o de una imagen. Para un optimizador geométrico son a la vez una herramienta (generan candidatos o puntos de partida) y un riesgo (sus salidas pueden parecer correctas en una imagen y ser inservibles para fabricar o para un motor). Este tema describe tres familias de generadores con sus artículos originales: la destilación de un modelo de difusión 2D en DreamFusion [32], los generadores directos de funciones implícitas como Shap-E [18] y los generadores autorregresivos de mallas como MeshGPT [35]. Después formaliza la validación de la geometría para fabricación y para motores, y la integra en un bucle de generación y verificación. Se apoya en la tarea 4 del curso 16-825 de Carnegie Mellon [2] y en el filtro de fabricación aditiva de Langelaar [22].

## 6.1 Tres maneras de generar geometría

**Definición 6.1 (Muestreo por destilación de puntuación).** Sea $x = g(\theta)$ una imagen renderizada desde una vista aleatoria de una representación 3D con parámetros $\theta$ (en DreamFusion, un campo de radiancia), y sea $\hat{\epsilon}_\phi(z_t; y, t)$ la predicción del ruido de un modelo de difusión de imágenes preentrenado y congelado, condicionado por el texto $y$, sobre la versión ruidosa $z_t$ de $x$ en el instante $t$. El *muestreo por destilación de puntuación* (SDS) actualiza $\theta$ con el gradiente [32, §2.1]

$$\nabla_\theta \mathcal{L}_{\text{SDS}} = \mathbb{E}_{t,\epsilon}\Big[w(t)\big(\hat{\epsilon}_\phi(z_t; y, t) - \epsilon\big)\frac{\partial x}{\partial \theta}\Big],$$

donde $\epsilon$ es el ruido añadido y $w(t)$ una ponderación.

La idea es que el modelo de difusión «sabe» cómo debería parecer una imagen del objeto descrito, y la diferencia entre el ruido que predice y el que realmente se añadió señala en qué dirección corregir la imagen; la regla de la cadena a través del renderizador derivable (tema 4) lleva esa corrección a los parámetros 3D. DreamFusion usa el modelo base de $64 \times 64$ de Imagen sin modificarlo y un campo de tipo mip-NeRF 360, y encuentra que hace falta una guía sin clasificador muy intensa [32]. En la práctica se mezclan la predicción condicionada y la no condicionada, $\hat{\epsilon} = \hat{\epsilon}_u + s(\hat{\epsilon}_c - \hat{\epsilon}_u)$, con $s$ grande [2]. El método no necesita datos 3D, pero cada objeto es una optimización completa.

Los *generadores directos* aprenden de pares de objetos 3D y textos. Shap-E entrena primero un codificador que transforma cada objeto en los parámetros de una función implícita, que puede renderizarse como malla texturizada o como campo de radiancia, y después un modelo de difusión condicionado sobre esos parámetros; genera objetos en segundos [18]. Los *generadores autorregresivos de mallas* tratan la malla como un texto: MeshGPT aprende un vocabulario de *embeddings* geométricos de caras con un codificador de grafos y cuantización

vectorial residual, y un transformer de solo decodificador predice el siguiente índice del vocabulario, como un LLM predice el siguiente token (módulos 12 y 15) [35]. El resultado son mallas compactas, parecidas a las que haría un artista, a diferencia de las mallas densas que se obtienen extrayendo la isosuperficie de un campo; a cambio, el tamaño está limitado por el contexto (los autores entrenan con mallas de menos de 800 caras).

Ninguna de las tres familias garantiza que el resultado sea una malla cerrada, orientada, sin autointersecciones, con la escala correcta y fabricable. Esas propiedades hay que comprobarlas.

## 6.2 Validación de la geometría

**Definición 6.2 (Comprobaciones topológicas).** Sea $(V, F)$ una malla triangular. Se cuentan las *aristas dirigidas* $(a, b)$ de todas las caras (cada cara $(a, b, c)$ aporta $(a, b)$, $(b, c)$ y $(c, a)$) y las aristas no dirigidas $\{a, b\}$. La malla

(i) tiene *borde* si alguna arista no dirigida pertenece a una sola cara;

(ii) es *no variedad* si alguna arista pertenece a tres o más caras;

(iii) está *mal orientada* si alguna arista dirigida aparece más de una vez.

Una malla cerrada, variedad y orientada de forma coherente es *estanca*.

**Proposición 6.3 (Una cara invertida).** *Sea una malla estanca e invértase la orientación de una única cara $f$ cuyas tres aristas son compartidas con caras distintas de $f$. Entonces la malla sigue cerrada y sin aristas no variedad, exactamente tres aristas dirigidas aparecen dos veces, y el volumen de la proposición 1.6 disminuye en $\frac{1}{3}\det(v_a, v_b, v_c)$, el doble de la contribución original de $f$.*

*Demostración.* Las aristas no dirigidas no cambian, luego el recuento por arista sigue siendo dos: la malla sigue cerrada y es variedad. Antes, cada arista $\{a, b\}$ de $f$ se recorría como $(a, b)$ en $f$ y como $(b, a)$ en su vecina; tras invertir $f$, ambas la recorren como $(b, a)$, que aparece dos veces; esto ocurre para las tres aristas de $f$ y para ninguna otra. Finalmente, invertir la cara intercambia dos columnas del determinante, que cambia de signo: la suma pierde $\det/6$ y gana $-\det/6$, una variación de $-\det/3$. ∎

Por eso una cara invertida es un defecto silencioso: no abre agujeros y una comprobación de estanqueidad basada solo en recuentos de aristas no dirigidas no la detecta, pero estropea el volumen, la iluminación (la normal apunta hacia dentro) y la clasificación interior/exterior que usan los programas de laminado.

Para *fabricación aditiva* se añaden comprobaciones geométricas:

- *Voladizos*: con dirección de fabricación $b$ unitaria, una cara de normal $n$ que mira hacia abajo necesita soporte si forma con la horizontal un ángulo menor que un umbral, habitualmente $45°$; equivale a $n \cdot b < -\cos 45° \approx -0{,}707$. Las caras apoyadas en la bandeja se excluyen. El filtro de Langelaar incorpora una restricción de este tipo dentro de la optimización de topología [22].
- *Espesor mínimo*: con la SDF de la pieza (tema 1), el espesor local es aproximadamente el doble de la distancia al borde en el eje medial; en la práctica se lanzan rayos desde la superficie hacia dentro y se mide la distancia a la cara opuesta.

- *Volumen de impresión, unidades y escala*: la caja envolvente debe caber en la máquina y estar en milímetros.
- *Cavidades cerradas*: en procesos de lecho de polvo o resina atrapan material sin consolidar.

Para un *motor de videojuegos*: número de triángulos y de materiales dentro del presupuesto (tema 5), coordenadas UV presentes y sin solapes, normales y tangentes coherentes, escala y punto de pivote convenidos, niveles de detalle generados y nombres según la convención del proyecto.

## 6.3 Generar y verificar

El esquema robusto es un bucle: el generador propone, el validador comprueba, un reparador corrige lo automático (soldar vértices duplicados, reorientar caras, cerrar agujeros pequeños) y el resultado se vuelve a validar. Si cada candidato pasa la validación con probabilidad $p$, generar varios y quedarse con el primero válido es eficaz (figura 6.1).

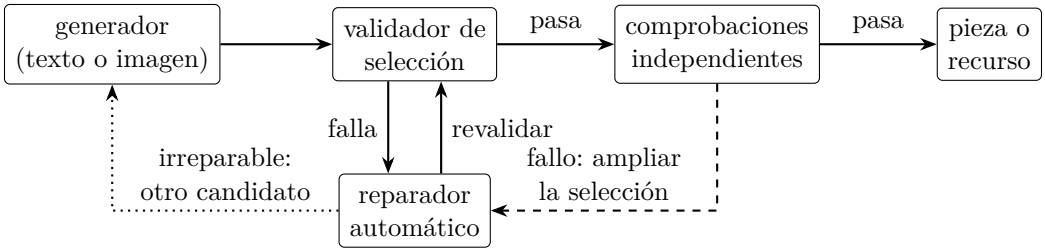



Figura 6.1: Bucle de generación y verificación. Las comprobaciones independientes no se usan para seleccionar ni para ajustar el generador.

**Proposición 6.4 (Lo mejor de $N$).** *Si $N$ candidatos superan la validación de forma independiente con probabilidad $p \in (0,1)$, la probabilidad de obtener al menos uno válido es $1-(1-p)^N$, y para alcanzar una probabilidad $1-\delta$ basta con*

$$N \geq \frac{\ln \delta}{\ln(1-p)}.$$

*Demostración.* Por independencia (0.8, §4.4), la probabilidad de que fallen todos es $(1-p)^N$. La condición $(1-p)^N \leq \delta$ equivale, tomando logaritmos y dividiendo por $\ln(1-p) < 0$, que invierte la desigualdad, a $N \geq \ln\delta / \ln(1-p)$. ■

El validador se convierte así en una función de recompensa verificable, análoga a las recompensas verificables con que se ajustan los LLM (módulo 21, §6.2). Y hereda su riesgo: si el generador se ajusta para pasar el validador, aprenderá a satisfacer las comprobaciones sin satisfacer la intención (por ejemplo, mallas estancas que se autointersecan si no se comprueban las autointersecciones). Es la ley de Goodhart que el módulo 21 (§6.4) estudia como sobreoptimización de la recompensa. La defensa es la misma: comprobaciones independientes que no se usan para ajustar el generador y revisión humana por muestreo.

## 6.4 Caso de estudio: validación de mallas generadas

**Ejemplo 6.5 (Un cubo con defectos y un voladizo).** *Contexto.* Un generador produce una pieza que debería ser un cubo de 40 mm de lado centrado en el origen, con 8 vértices y 12 triángulos. Se validan tres salidas: la correcta, una a la que le falta un triángulo y una con un triángulo invertido.

*Comprobaciones* (definición 6.2 y proposición 1.6):

| Salida | $\lvert E\rvert$ | borde | no variedad | dirigidas repetidas | $\chi$ | volumen (mm³) |
|---|---|---|---|---|---|---|
| correcta | 18 | 0 | 0 | 0 | 2 | 64 000 |
| falta un triángulo | 18 | 3 | 0 | 0 | 1 | 58 667 |
| un triángulo invertido | 18 | 0 | 0 | 3 | 2 | 53 333 |

*Lectura.* La malla incompleta tiene 3 aristas de borde y $\chi = 1$; su «volumen» $(64\,000 - 5333)$ no significa nada porque la superficie no encierra una región. La malla con un triángulo invertido pasa las pruebas de borde y de variedad, pero tiene 3 aristas dirigidas repetidas y su volumen es $\frac{5}{6}$ del real: cada uno de los 12 triángulos aporta $a^3/12 = 5333$ mm³ y, por la proposición 6.3, invertir uno resta el doble. Una validación que solo contara aristas no dirigidas la daría por buena.

*Voladizos.* Para imprimir el cubo en $+z$ inclinado 30° alrededor del eje $x$, las normales de las caras que miran hacia abajo tienen componente vertical $-\cos 30° = -0{,}866$ (la antigua base, que ya no apoya en la bandeja) y $-\sin 30° = -0{,}5$. Solo la primera cumple $n_z < -0{,}707$: necesita soporte 1600 mm² de 9600, un 16,7 %. La segunda forma 60° con la horizontal y se autosoporta. Apoyar el cubo sobre una cara elimina los soportes.

*Política de generación.* Si el generador produce piezas válidas con probabilidad $p = 0{,}3$ e independencia entre muestras, la proposición 6.4 con $\delta = 0{,}05$ da $N \geq \ln 0{,}05 / \ln 0{,}7 = 8{,}4$: generando 9 candidatos se obtiene al menos uno válido con probabilidad 0,960 (con 8 sería 0,942).

| Herramienta | Para qué sirve | Licencia o tipo | Cuándo elegirla |
|---|---|---|---|
| trimesh | estanqueidad, orientación, volumen, reparaciones simples | MIT | validador ligero en Python |
| PyMeshLab | filtros de reparación (no variedad, agujeros) | GPL-3.0 | reparación por lotes |
| Open3D | métricas geométricas y distancias entre mallas y nubes | MIT | comparar con una referencia |
| Blender (`bpy`) | comprobaciones y correcciones dentro del pipeline del artista | GPL-3.0 | validar recursos antes de exportarlos al motor |
| Shap-E | generación de funciones implícitas a partir de texto o imagen | MIT (pesos publicados) | prototipos y generación local reproducible |

Cuadro 6.1: Tecnologías del tema 6.

*Observación* 6.6 (Errores frecuentes en la práctica). (i) *Validar por la imagen*: un render convincente no garantiza una malla estanca, orientada y sin autointersecciones. (ii) *Escala y unidades*: los generadores suelen producir objetos normalizados; sin una escala explícita, una pieza de 40 mm puede llegar a la impresora como de 40 m o de 0,04 mm. (iii) *Validar*

*con los mismos casos con que se ajustó el generador*: los textos o imágenes de prueba deben ser distintos de los usados para ajustar o elegir el generador. (iv) *Goodhart*: un generador seleccionado o ajustado contra el validador aprende sus huecos; se reservan comprobaciones independientes. (v) *Propiedad intelectual y licencias*: los pesos y los datos de entrenamiento de los generadores tienen licencias propias que limitan el uso comercial de los resultados en algunos casos. (vi) *Reparaciones que cambian la pieza*: cerrar agujeros o soldar vértices puede alterar dimensiones funcionales; tras reparar se vuelve a medir.

**Lecturas recomendadas para este tema**

- B. Poole *et al.* [32]: §2–3 (muestreo por destilación de puntuación y el modelo de NeRF).
- H. Jun y A. Nichol [18]: generación de funciones implícitas condicionadas.
- Y. Siddiqui *et al.* [35]: generación de mallas con transformers de solo decodificador.
- CMU 16-825 [2], tarea 4, §2 «Diffusion-guided Optimization».
- M. Langelaar [22]: restricción de voladizo en la optimización de topología.

## Ejercicios del tema 6

**Ejercicio 6.1** ⋆ Enumere cuatro comprobaciones para una pieza que se va a imprimir y cuatro para un recurso que se va a cargar en un motor. ¿Cuáles son comunes?

**Ejercicio 6.2** ⋆⋆ En un ejemplo de juguete con dos píxeles, el ruido añadido es $\epsilon = (0{,}3;\ -0{,}2)$, la predicción condicionada es $\hat{\epsilon}_c = (0{,}5;\ 0{,}1)$ y la no condicionada $\hat{\epsilon}_u = (0{,}2;\ -0{,}1)$. Con guía $s = 7{,}5$, $w(t) = 1$ y $x = \theta$ (se optimiza directamente la imagen), calcule $\hat{\epsilon}$, el gradiente SDS y el nuevo $\theta$ tras un paso de descenso con tasa 0,01 desde $\theta = (0{,}5;\ 0{,}5)$.

*Fuente:* adaptado de CMU 16-825, «Learning for 3D Vision» (S. Tulsiani), tarea 4, §2.1 «SDS Loss + Image Optimization» [2].

**Ejercicio 6.3** ⋆⋆ Una malla tiene las caras $(0,1,2)$, $(0,2,3)$, $(0,3,1)$, $(1,3,2)$ y $(1,2,4)$. Clasifique sus aristas (de borde, interiores, no variedad), calcule $\chi$ y diga qué defectos tiene.

**Ejercicio 6.4** ⋆⋆ Una pieza se imprime en $+z$. Sus caras tienen áreas y componentes $n_z$: 400 mm² con $n_z = -1$ (apoyada en la bandeja), 300 mm² con $n_z = -0{,}8$, 200 mm² con $n_z = -0{,}6$, 500 mm² con $n_z = 0$ y 600 mm² con $n_z = 0{,}5$. Con umbral de 45°, ¿qué fracción del área total necesita soporte?

**Ejercicio 6.5** ⋆⋆ Un generador produce piezas válidas con probabilidad 0,2. Cada generación tarda 30 s y cada validación 2 s. ¿Cuántos candidatos hay que generar para tener al menos uno válido con probabilidad 0,9, y cuánto tarda el lote? ¿Cuál es el número esperado de candidatos hasta el primer válido si se generan de uno en uno?

**Ejercicio 6.6** ⋆⋆ El SDF de un campo neuronal se evalúa en una rejilla de $128^3$ nodos sobre $[-1{,}5;\ 1{,}5]^3$ para extraer la malla. (a) Calcule el paso de la rejilla y el número de evaluaciones, y cuántos lotes de 65 536 puntos hacen falta. (b) Si una unidad equivale a 50 mm, ¿se resolverá una pared de 0,8 mm?

*Fuente:* adaptado de Stanford CS348n (L. Guibas, primavera de 2023), tarea 2, problema 1(c) «Marching cubes» [5].

**Ejercicio 6.7** ⋆⋆⋆ (a) Para una malla triangular cerrada y conexa de género $g$, demuestre que $|V| - \frac{1}{2}|F| = 2 - 2g$. (b) Un toro se discretiza con una rejilla de $16 \times 8$ vértices con identificación periódica en ambas direcciones, dividiendo cada cuadrilátero en dos triángulos. Calcule $|V|$, $|E|$, $|F|$, $\chi$ y $g$. (c) Proponga cómo usar $g$ para validar la salida de un generador que debería producir una arandela.

**Ejercicio 6.8** ⋆⋆⋆ Diseñe razonadamente un bucle de generación y validación para producir soportes imprimibles a partir de descripciones de texto: indique qué comprobaciones usa para seleccionar candidatos, cuáles reserva como comprobaciones independientes, qué reparaciones automáticas admite y cómo detectaría que el generador ha empezado a «engañar» al validador.

Apéndice A

# Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro o cinco cifras significativas salvo que se indique otra cosa. Las longitudes van en milímetros salvo indicación contraria. Todos los valores numéricos se han comprobado con Python; los de los casos de estudio, con el cuaderno del módulo.

## A.1 Tema 1

### Solución del ejercicio 1.1

*Datos.* cuatro situaciones de uso.

*Planteamiento.* aplicar el cuadro 1.1 y la discusión de la sección 1.5.

*Desarrollo.*

1. (a) Abrir agujeros es un cambio de topología: hace falta una representación volumétrica (densidades por elemento, vóxeles, SDF o campo neuronal). Una malla con conectividad fija no puede.
2. (b) Un motor dibuja triángulos: malla con UV, normales y niveles de detalle (tema 5).
3. (c) Un LiDAR mide puntos en la superficie, sin conectividad: nube de puntos (con normales estimadas si hacen falta).
4. (d) PyTorch3D rasteriza mallas de forma derivable y ofrece pérdidas sobre ellas: malla deformable que parte de una esfera, optimizando las posiciones de los vértices (topología de esfera fija).

*Resultado.* **(a) volumétrica; (b) malla; (c) nube de puntos; (d) malla con conectividad fija**.

*Comprobación.* en (d) la forma final tendrá necesariamente $\chi = 2$; si el objeto tuviera un asa, haría falta otra representación, coherente con (a).

### Solución del ejercicio 1.2

*Datos.* tetraedro y cubo triangulado.

*Planteamiento.* contar elementos y aplicar $\chi = |V| - |E| + |F|$ y la proposición 1.3.

*Desarrollo.*

1. Tetraedro con vértices $0, 1, 2, 3$ y caras $(0, 2, 1)$, $(0, 1, 3)$, $(0, 3, 2)$, $(1, 2, 3)$: $|V| = 4$, $|F| = 4$ y aristas $\{0, 1\}$, $\{0, 2\}$, $\{0, 3\}$, $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$, es decir, $|E| = 6$.
2. $\chi = 4 - 6 + 4 = 2$ y $2|E| = 12 = 3|F|$.

3. Cubo: 8 vértices; 6 caras cuadradas, cada una en 2 triángulos: $|F| = 12$. Aristas: 12 del cubo más 6 diagonales (una por cara cuadrada), $|E| = 18$.
4. $\chi = 8 - 18 + 12 = 2$ y $2|E| = 36 = 3|F|$.

*Resultado.* **tetraedro:** $(4, 6, 4)$**; cubo triangulado:** $(8, 18, 12)$**; ambos con** $\chi = 2$.

*Comprobación.* el corolario 1.4 predice $|F| = 2|V| - 4$: $2 \cdot 4 - 4 = 4$ y $2 \cdot 8 - 4 = 12$. Correcto.

### Solución del ejercicio 1.3

*Datos.* malla cerrada con $\chi = 2$ y $|F| = 1280$; 12 bytes por vértice y 12 por cara.

*Planteamiento.* proposición 1.3 y corolario 1.4.

*Desarrollo.*

1. $|E| = \frac{3}{2}|F| = 1920$.
2. $|V| = (|F| + 4)/2 = 642$.
3. Memoria: $642 \cdot 12 + 1280 \cdot 12 = 7704 + 15\,360 = 23\,064$ bytes.

*Resultado.* $|\mathbf{E}| = \mathbf{1920}$, $|\mathbf{V}| = \mathbf{642}$; $23\,064$ **bytes (**22,5 **KiB)**.

*Comprobación.* $\chi = 642 - 1920 + 1280 = 2$. Una icosfera de nivel $k$ tiene $10 \cdot 4^k + 2$ vértices: con $k = 3$, 642.

### Solución del ejercicio 1.4

*Datos.* $f_e(x) = \|x\| - 20$; caja de semilados $b = (10, 10, 5)$; puntos $p_1$, $p_2$, $p_3$.

*Planteamiento.* evaluar las SDF; unión = min, intersección = max, diferencia = $\max(f_b, -f_e)$.

*Desarrollo.*

1. $p_1 = (0, 0, 0)$: $f_e = -20$; $q = (-10, -10, -5)$, $\|\max(q, 0)\| = 0$ y $\min(\max_i q_i, 0) = -5$, luego $f_b = -5$. Unión $-20$; intersección $-5$; diferencia $\max(-5, 20) = 20$.
2. $p_2 = (15, 0, 0)$: $f_e = -5$; $q = (5, -10, -5)$, $\|\max(q, 0)\| = 5$, $\min(5, 0) = 0$, luego $f_b = 5$. Unión $-5$; intersección 5; diferencia $\max(5, 5) = 5$.
3. $p_3 = (0, 0, 12)$: $f_e = -8$; $q = (-10, -10, 7)$, $f_b = 7$. Unión $-8$; intersección 7; diferencia $\max(7, 8) = 8$.

*Resultado.* en el orden ($f_e$, $f_b$, unión, intersección, diferencia): $p_1$**:** $(-20, -5, -20, -5, 20)$; $p_2$**:** $(-5, 5, -5, 5, 5)$; $p_3$**:** $(-8, 7, -8, 7, 8)$.

*Comprobación.* la caja está dentro de la esfera (su vértice más lejano está a $\sqrt{10^2 + 10^2 + 5^2} = 15$), así que la diferencia caja menos esfera es vacía: su función es positiva en todos los puntos, como se obtiene.

### Solución del ejercicio 1.5

*Datos.* (a) $f = -0{,}3$ en $x = 2$, $f = 0{,}5$ en $x = 3$. (b) $A = \{(0, 0), (1, 0)\}$, $B = \{(0, 1), (2, 0)\}$.

*Planteamiento.* (a) interpolación lineal $t = f_0/(f_0 - f_1)$; (b) definición 1.5.

*Desarrollo.*

1. $t = -0{,}3/(-0{,}3 - 0{,}5) = 0{,}375$; vértice en $x = 2 + 0{,}375 \cdot 1 = 2{,}375$ mm.

2. De $A$ a $B$: para $(0,0)$, distancias al cuadrado 1 y 4, mínimo 1; para $(1,0)$, 2 y 1, mínimo 1. Media 1.
3. De $B$ a $A$: para $(0,1)$, 1 y 2, mínimo 1; para $(2,0)$, 4 y 1, mínimo 1. Media 1.
4. $\mathrm{CD} = 1 + 1 = 2$.

*Resultado.* **(a)** $x = 2{,}375$ **mm; (b)** $\mathrm{CD}(A, B) = 2$.

*Comprobación.* la interpolación de $f$ en 2,375 vale $-0{,}3 + 0{,}375 \cdot 0{,}8 = 0$.

### Solución del ejercicio 1.6

*Datos.* predicciones $(0{,}05;\ 0{,}30;\ -0{,}08;\ -0{,}40)$; verdades $(0{,}02;\ 0{,}12;\ 0{,}15;\ -0{,}02)$; $\delta = 0{,}1$.

*Planteamiento.* $\ell = |\mathrm{clip}(\hat{f}, -\delta, \delta) - \mathrm{clip}(s, -\delta, \delta)|$ y media.

*Desarrollo.*

1. Punto 1: ningún recorte; $|0{,}05 - 0{,}02| = 0{,}03$.
2. Punto 2: $\mathrm{clip}(0{,}30) = 0{,}1$ y $\mathrm{clip}(0{,}12) = 0{,}1$; pérdida 0.
3. Punto 3: $\mathrm{clip}(-0{,}08) = -0{,}08$ y $\mathrm{clip}(0{,}15) = 0{,}1$; pérdida 0,18.
4. Punto 4: $\mathrm{clip}(-0{,}40) = -0{,}1$ y $-0{,}02$; pérdida 0,08.
5. Media: $(0{,}03 + 0 + 0{,}18 + 0{,}08)/4 = 0{,}0725$.

*Resultado.* **pérdida media** 0,0725. El punto 2 no aporta: lejos de la superficie (a más de $\delta$), basta con acertar el signo y el valor exacto no se penaliza.

*Comprobación.* el punto 3 tiene el signo equivocado (dice dentro cuando está fuera) y es el que más pesa, como debe.

### Solución del ejercicio 1.7

*Datos.* tetraedro del ejemplo 1.7, $a = 30$ mm; desplazamiento de 0,5 mm de $v_1$ en $x$.

*Planteamiento.* proposición 1.6 con las caras que contienen $v_1$ en orden cíclico $(1, b, c)$.

*Desarrollo.*

1. Caras con el vértice 1: $(0,2,1) \to (1,0,2)$; $(0,1,3) \to (1,3,0)$; $(1,2,3)$.
2. $\nabla_{v_1} \mathrm{Vol} = \frac{1}{6}(v_0 \times v_2 + v_3 \times v_0 + v_2 \times v_3) = \frac{1}{6}\big(0 + 0 + (0,a,0) \times (0,0,a)\big) = \frac{1}{6}(a^2, 0, 0)$.
3. Con $a = 30$: $(150, 0, 0)$ mm².
4. Base por altura: la cara opuesta a $v_1$ (plano $x = 0$) tiene área $a^2/2 = 450$ mm²; $\partial \mathrm{Vol}\,/\partial x_1 = 450/3 = 150$ mm².
5. $\Delta \mathrm{Vol} \approx 150 \cdot 0{,}5 = 75$ mm³.

*Resultado.* $\nabla_{\mathbf{v_1}} \mathrm{Vol} = (\mathbf{150}, \mathbf{0}, \mathbf{0})$ **mm²**; $\Delta \mathrm{Vol} = 75$ **mm³**.

*Comprobación.* el volumen es $x_1 \cdot a \cdot a/6$, lineal en $x_1$: con $x_1 = 30{,}5$, $\mathrm{Vol} = 30{,}5 \cdot 900/6 = 4575$, exactamente 75 más.

### Solución del ejercicio 1.8

*Datos.* $f = \|x - c\| - r$; $g = \varphi \circ f$ con $\varphi' > 0$ y $\varphi(0) = 0$; $g(x) = \|x\|^2 - 4$ en $(2,0,0)$.

*Planteamiento.* derivar la norma; regla de la cadena (módulo 4, tema 2).

*Desarrollo.*

1. (a) Para $x \neq c$, $\nabla\|x-c\| = (x-c)/\|x-c\|$, de norma 1; la constante $r$ no influye.
2. (b) $\varphi$ estrictamente creciente con $\varphi(0)=0$ cumple $\operatorname{signo}\varphi(t) = \operatorname{signo} t$, luego $\{g \leq 0\} = \{f \leq 0\}$ y $\{g=0\} = \{f=0\}$: el mismo sólido y la misma superficie.
3. Por la regla de la cadena, $\nabla g = \varphi'(f)\nabla f$, y con $\|\nabla f\| = 1$, $\|\nabla g\| = \varphi'(f)$.
4. (c) $\|x\|^2 - 4 = (\|x\|-2)(\|x\|+2) = f(f+4)$ con $f = \|x\|-2$, es decir, $\varphi(t) = t^2+4t$, creciente en el rango $t \geq -2$ de $f$. En la superficie, $\varphi'(0) = 4$.
5. Directamente, $\nabla g = 2x = (4,0,0)$ en $(2,0,0)$, de norma 4; término eikonal $(4-1)^2 = 9$.

*Resultado.* **(c) $\|\nabla g\| = 4$ y término eikonal 9.** La regularización empuja a la red hacia la reparametrización con $\varphi' = 1$, la única que hace que los valores sean distancias.

*Comprobación.* para el punto $(3,0,0)$, a distancia 1 de la superficie, $g = 5$ mientras que $f = 1$: $g$ sobrestima la distancia, coherente con $\varphi' > 1$.

### Solución del ejercicio 1.9

*Datos.* malla cerrada y orientada de forma coherente; traslación $t$.

*Planteamiento.* desarrollar el producto vectorial como suma sobre aristas dirigidas; derivar respecto a $t$.

*Desarrollo.*

1. $(v_b - v_a)\times(v_c - v_a) = v_b\times v_c - v_b\times v_a - v_a\times v_c = v_a\times v_b + v_b\times v_c + v_c\times v_a$: es la suma de $u\times w$ sobre las tres aristas dirigidas $(u,w)$ de la cara.
2. En una malla cerrada y coherente, cada arista $\{u,w\}$ aparece una vez como $(u,w)$ y otra como $(w,u)$, y $u\times w + w\times u = 0$. Sumando sobre todas las caras, todo se cancela: (a) queda probado.
3. Sea $S(t) = \frac{1}{6}\sum_f \det(v_a+t, v_b+t, v_c+t)$. Por multilinealidad, $\nabla_t S = \frac{1}{6}\sum_f \big[(v_b+t)\times(v_c+t) + (v_c+t)\times(v_a+t) + (v_a+t)\times(v_b+t)\big]$, que es la suma del paso 1 para la malla trasladada, nula por (a).
4. $S$ tiene gradiente nulo en todo $t$, luego es constante: la fórmula no depende de la traslación.
5. Trasladar todos los vértices es mover cada $v_i$ en $t$, así que $\nabla_t S\big|_{t=0} = \sum_i \nabla_{v_i}\operatorname{Vol} = 0$.

*Resultado.* $\sum_f (v_b - v_a)\times(v_c - v_a) = 0$**; el volumen es invariante por traslación y** $\sum_i \nabla_{v_i}\operatorname{Vol} = 0$.

*Comprobación.* en el ejemplo 1.7, $\nabla_{v_3}\operatorname{Vol} = (0,0,150)$ y, por simetría, $\nabla_{v_1} = (150,0,0)$ y $\nabla_{v_2} = (0,150,0)$; para $v_0$, la cara opuesta tiene normal $(1,1,1)/\sqrt{3}$ y área $\frac{\sqrt{3}}{2}a^2$, de modo que $\nabla_{v_0} = -\frac{1}{3}\cdot\frac{\sqrt{3}}{2}a^2\cdot(1,1,1)/\sqrt{3} = (-150,-150,-150)$. La suma es nula.

## A.2 Tema 2

### Solución del ejercicio 2.1

*Datos.* cuatro afirmaciones.

*Planteamiento.* teorema 2.3, definición 2.2, definición 2.1 y definición 2.9.

*Desarrollo.*

1. (a) Falsa. Con $p = 1$ la rigidez es proporcional al material y no hay penalización de los grises; el óptimo (convexo) suele tener grandes zonas intermedias.
2. (b) Verdadera. Con $E_{\min} = 0$, un elemento vacío no aporta rigidez y los nodos rodeados de vacío quedan sueltos: $K$ es singular.
3. (c) Falsa. La flexibilidad $F^\top U$ es el trabajo de las cargas: pondera los desplazamientos en los puntos cargados; puede haber puntos sin carga con desplazamientos grandes.
4. (d) Falsa. El filtro de sensibilidades es heurístico: el vector filtrado no es, en general, el gradiente de ninguna función; el filtro de densidades sí da el gradiente exacto del problema filtrado.

*Resultado.* **F, V, F, F**.

*Comprobación.* con el código del cuaderno, la viga MBB con $p = 3$ tiene una medida de grises de 0,265; con $p = 1$ (mismo filtro) sube a 0,597 y la flexibilidad baja a 165,3 porque el material intermedio no se penaliza: el diseño gris no es fabricable.

### Solución del ejercicio 2.2

*Datos.* diseño generativo comercial.

*Planteamiento.* qué omite el problema de flexibilidad y qué cambia el posprocesado (sección 2.7).

*Desarrollo.*

1. El diseño analizado durante la optimización es un campo de densidades en una malla gruesa; la pieza fabricada es una geometría suavizada y reconstruida, distinta.
2. El objetivo es la rigidez global; no se controlan las tensiones locales (concentraciones en esquinas), el pandeo de barras esbeltas ni la fatiga.
3. Los modelos de material y de fabricación (anisotropía de la impresión, tensiones residuales, soportes) no están en el problema.

*Resultado.* **hay que reanalizar porque la geometría cambia al posprocesar y porque el problema no incluye, por ejemplo, tensiones admisibles ni pandeo**.

*Comprobación.* la sección 2.7 indica que el flujo habitual termina con un análisis de elementos finitos más fino que incluye tensiones.

### Solución del ejercicio 2.3

*Datos.* $x_e = 0{,}5$; $p = 1, 2, 3$; $E_{\min} \approx 0$.

*Planteamiento.* $E_e/(E_0 x_e) = x_e^{p-1}$; resolver $x^3 = 0{,}5$.

*Desarrollo.*

1. $p = 1$: $0{,}5^0 = 1$; $p = 2$: 0,5; $p = 3$: 0,25.
2. Media rigidez con $p = 3$: $x^3 = 0{,}5 \Rightarrow x = 0{,}5^{1/3} = 0{,}7937$.

*Resultado.* **relaciones** 1**;** 0,5**;** 0,25**; densidad necesaria** 0,794.

*Comprobación.* $0{,}7937^3 = 0{,}5000$. Con $p = 3$ hace falta el 79 % del material para obtener el 50 % de la rigidez: los grises son caros.

### Solución del ejercicio 2.4

*Datos.* $x = (0{,}5; 0{,}5; 0{,}5)$; $-\partial c/\partial x = (4;\ 1;\ 0{,}25)$; $\eta = 1/2$; $m = 0{,}2$; volumen objetivo 1,5.

*Planteamiento.* $x_e^{\text{nuevo}} = \text{clip}\big(x_e\sqrt{-\partial_e c/\Lambda},\ [0{,}3;\ 0{,}7]\big)$; ajustar $\Lambda$.

*Desarrollo.*

1. Sin recortar, $x_e\sqrt{-\partial_e c/\Lambda} = (1;\ 0{,}5;\ 0{,}25)/\sqrt{\Lambda}$.
2. El primero se recorta a 0,7 si $1/\sqrt{\Lambda} \geq 0{,}7$, es decir, $\Lambda \leq 2{,}04$, y el tercero a 0,3 si $0{,}25/\sqrt{\Lambda} \leq 0{,}3$, es decir, $\Lambda \geq 0{,}69$. Si ambos se recortan, la suma es $0{,}7 + 0{,}5/\sqrt{\Lambda} + 0{,}3$, que debe valer 1,5.
3. $0{,}5/\sqrt{\Lambda} = 0{,}5 \Rightarrow \Lambda = 1$, dentro de $[0{,}69;\ 2{,}04]$.
4. $x^{\text{nuevo}} = (0{,}7;\ 0{,}5;\ 0{,}3)$.

*Resultado.* $\boldsymbol{\Lambda = 1}$, $\mathbf{x}^{\text{nuevo}} = (\mathbf{0{,}7};\ \mathbf{0{,}5};\ \mathbf{0{,}3})$.

*Comprobación.* volumen 1,5; el elemento más sensible gana material y el menos sensible lo pierde, ambos limitados por $m = 0{,}2$. En el interior, $B_2 = 1/(\Lambda \cdot 1) = 1$, la condición de optimalidad.

### Solución del ejercicio 2.5

*Datos.* $r_{\min} = 1{,}5$; sensibilidades: centro $-4$, laterales $-2$, diagonales $-1$; densidades uniformes.

*Planteamiento.* definición 2.9; con $x$ uniforme, $x_i$ y $\max(\gamma, x_e)$ se cancelan y queda la media ponderada.

*Desarrollo.*

1. Pesos: centro $1{,}5 - 0 = 1{,}5$; laterales $1{,}5 - 1 = 0{,}5$; diagonales $1{,}5 - \sqrt{2} = 0{,}08579$. Los demás elementos están a distancia $\geq 2$ y pesan cero.
2. Suma de pesos: $1{,}5 + 4 \cdot 0{,}5 + 4 \cdot 0{,}08579 = 3{,}8431$.
3. Numerador: $1{,}5(-4) + 4 \cdot 0{,}5(-2) + 4 \cdot 0{,}08579(-1) = -6 - 4 - 0{,}3431 = -10{,}3431$.
4. Cociente: $-10{,}3431/3{,}8431 = -2{,}6913$.

*Resultado.* **sensibilidad filtrada** $-2{,}691$.

*Comprobación.* es una media ponderada de valores entre $-4$ y $-1$, y queda entre ellos; el cuaderno comprueba que la suma de pesos es 3,8431.

### Solución del ejercicio 2.6

*Datos.* $H = \begin{pmatrix} 1{,}5 & 0{,}5 & 0 \\ 0{,}5 & 1{,}5 & 0{,}5 \\ 0 & 0{,}5 & 1{,}5 \end{pmatrix}$; $x = (1; 0{,}5; 0)$; $\partial c/\partial \tilde{x} = (-3; -2; -1)$.

*Planteamiento.* $\tilde{x}_e = \sum_i H_{ei} x_i / \sum_i H_{ei}$ y $\partial c/\partial x_j = \sum_e (H_{ej}/\sum_i H_{ei})\, \partial c/\partial \tilde{x}_e$.

*Desarrollo.*

1. Sumas de filas: $(2;\ 2{,}5;\ 2)$.
2. $\tilde{x}_1 = (1{,}5 + 0{,}25)/2 = 0{,}875$; $\tilde{x}_2 = (0{,}5 + 0{,}75 + 0)/2{,}5 = 0{,}5$; $\tilde{x}_3 = (0{,}25 + 0)/2 = 0{,}125$.
3. $\partial c/\partial x_1 = \frac{1{,}5}{2}(-3) + \frac{0{,}5}{2{,}5}(-2) = -2{,}25 - 0{,}4 = -2{,}65$.
4. $\partial c/\partial x_2 = \frac{0{,}5}{2}(-3) + \frac{1{,}5}{2{,}5}(-2) + \frac{0{,}5}{2}(-1) = -0{,}75 - 1{,}2 - 0{,}25 = -2{,}2$.

5. $\partial c/\partial x_3 = \frac{0{,}5}{2{,}5}(-2) + \frac{1{,}5}{2}(-1) = -0{,}4 - 0{,}75 = -1{,}15$.

*Resultado.* $\tilde{\mathbf{x}} = (\mathbf{0{,}875};\ \mathbf{0{,}5};\ \mathbf{0{,}125})$; $\partial \mathbf{c}/\partial \mathbf{x} = (\mathbf{-2{,}65};\ \mathbf{-2{,}2};\ \mathbf{-1{,}15})$.

*Comprobación.* la suma se conserva, $-2{,}65 - 2{,}2 - 1{,}15 = -6 = -3 - 2 - 1$, porque cada fila de $H$ dividida por su suma suma 1. El volumen físico, 1,5, coincide con el de diseño.

### Solución del ejercicio 2.7

*Datos.* $\bar{c} = 216{,}81$; $E_0 = 200$ GPa; $t = 5$ mm; $F = 2$ kN; semimodelo de $60 \times 20$ mm al 50 %; 7,85 g/cm$^3$.

*Planteamiento.* $c = \bar{c}F^2/(E_0 t)$, flecha $\delta = c/F$, masa $= \rho \cdot f_V \cdot$volumen.

*Desarrollo.*

1. $c = 216{,}81 \cdot (2000)^2/(2 \cdot 10^{11} \cdot 0{,}005) = 216{,}81 \cdot 4 \cdot 10^6/10^9 = 0{,}8672$ J.
2. $\delta = 0{,}8672/2000 = 4{,}336 \cdot 10^{-4}$ m $= 0{,}434$ mm.
3. Volumen de material: $0{,}5 \cdot 60 \cdot 20 \cdot 5 = 3000$ mm$^3 = 3$ cm$^3$; masa $3 \cdot 7{,}85 = 23{,}55$ g.

*Resultado.* $c = 0{,}867$ **J; flecha** $0{,}434$ **mm; masa** $23{,}6$ **g**.

*Comprobación.* frente al aluminio del ejemplo 2.10 (0,310 J con 1 kN): la carga doble multiplica por 4, $E_0 t$ pasa de $7 \cdot 10^8$ a $10^9$ N/m y $0{,}310 \cdot 4 \cdot 0{,}7 = 0{,}867$.

### Solución del ejercicio 2.8

*Datos.* $g = L^\top U$; dos muelles en serie, $k = (512;\ 125)$ N/mm, $k_e = x_e^3 k_0$, $k_0 = 1000$ N/mm, $x = (0{,}8;\ 0{,}5)$, $F = 100$ N en el extremo; salida $g = u_1$.

*Planteamiento.* teorema 2.4 con $\partial g/\partial U = L^\top$.

*Desarrollo.*

1. (a) Adjunto: $K\lambda = L$ ($K$ simétrica). Sensibilidad: $dg/dx_e = -\lambda^\top(\partial K/\partial x_e)U = -p\, x_e^{p-1}(E_0 - E_{\min})\, \lambda_e^\top k_0 u_e$. Cuesta dos resoluciones: $U$ y $\lambda$, para cualquier número de variables.
2. (b) $K = \begin{pmatrix} k_1+k_2 & -k_2 \\ -k_2 & k_2 \end{pmatrix}$, $L = (1,0)^\top$. Se comprueba que $\lambda = (1/k_1,\ 1/k_1)$: primera fila $(k_1 + k_2 - k_2)/k_1 = 1$, segunda $(-k_2 + k_2)/k_1 = 0$.
3. $U = (F/k_1,\ F/k_1 + F/k_2)$, así que $u_1 = 100/512 = 0{,}19531$ mm.
4. Barra 1 (entre el apoyo y el nodo 1): $\partial K/\partial x_1 = 3x_1^2 k_0 \left(\begin{smallmatrix} 1 & 0 \\ 0 & 0 \end{smallmatrix}\right)$, luego $dg/dx_1 = -3x_1^2 k_0 \lambda_1 u_1 = -1920 \cdot \frac{1}{512} \cdot 0{,}19531 = -0{,}73242$ mm.
5. Barra 2: $\partial K/\partial x_2 \propto \left(\begin{smallmatrix} 1 & -1 \\ -1 & 1 \end{smallmatrix}\right)$ y $dg/dx_2 \propto -(\lambda_1 - \lambda_2)(u_1 - u_2) = 0$.

*Resultado.* $dg/dx_1 = -0{,}7324$ **mm y** $dg/dx_2 = 0$, con dos resoluciones.

*Comprobación.* directamente, $g = F/(x_1^3 k_0)$, $dg/dx_1 = -3F/(x_1^4 k_0) = -300/(0{,}4096 \cdot 1000) = -0{,}73242$ mm, y $g$ no depende de $x_2$.

### Solución del ejercicio 2.9

*Datos.* problema de la definición 2.1 con SIMP y cotas $0 \le x_e \le 1$.

*Planteamiento.* corolario 2.5; condiciones de Karush–Kuhn–Tucker con multiplicadores $\Lambda \ge 0$ (volumen), $\mu_e^+ \ge 0$ ($x_e \le 1$) y $\mu_e^- \ge 0$ ($x_e \ge 0$).

*Desarrollo.*

1. (a) Por el corolario 2.5, $\partial c/\partial x_e = -p\,x_e^{p-1}(E_0 - E_{\min})u_e^\top k_0 u_e \le 0$ porque $k_0$ es semidefinida positiva. Añadir material nunca empeora $c$.
2. Si la restricción fuera $\sum x_e \le N f_V$ y estuviera inactiva en un óptimo con algún $0 < x_e < 1$ y $\partial c/\partial x_e < 0$, aumentar ligeramente $x_e$ seguiría siendo factible y reduciría $c$, contradicción. Luego la restricción como desigualdad es activa y equivale a la igualdad.
3. (b) Lagrangiana: $\mathcal{L} = c + \Lambda(\sum_e x_e - N f_V) + \sum_e \mu_e^+(x_e - 1) - \sum_e \mu_e^- x_e$. Estacionariedad: $\partial c/\partial x_e + \Lambda + \mu_e^+ - \mu_e^- = 0$.
4. Si $0 < x_e < 1$, holgura complementaria: $\mu_e^\pm = 0$, luego $-\partial c/\partial x_e = \Lambda$, es decir, $B_e = 1$.
5. Si $x_e = 1$: $\mu_e^- = 0$ y $-\partial c/\partial x_e = \Lambda + \mu_e^+ \ge \Lambda$, es decir, $B_e \ge 1$. Análogamente, si $x_e = 0$, $B_e \le 1$.

*Resultado.* **la restricción de volumen es activa; $B_e = 1$ en los elementos intermedios, $B_e \ge 1$ en los sólidos y $B_e \le 1$ en los vacíos**.

*Comprobación.* la actualización OC (definición 2.7) no mueve un elemento con $B_e = 1$, aumenta los de $B_e > 1$ hasta la cota superior y disminuye los de $B_e < 1$: sus puntos fijos son exactamente los puntos KKT.

## A.3 Tema 3

### Solución del ejercicio 3.1

*Datos.* tres escenarios.

*Planteamiento.* coste por gradiente: diferencias finitas $n + 1$ simulaciones; adjunto o diferenciación automática, $O(1)$ si el código es derivable; sustituto, $M$ simulaciones iniciales.

*Desarrollo.*

1. (a) 3 parámetros, caja negra de 6 h: no se puede derivar el código; las diferencias finitas cuestan 4 simulaciones (24 h) por gradiente y son ruidosas. Un sustituto (proceso gaussiano o MLP) con unas decenas de simulaciones y verificación del óptimo.
2. (b) 5000 densidades y código propio: el método adjunto (o reescribir el resolvedor en un marco derivable). Diferencias finitas costarían 5001 resoluciones por iteración; un sustituto en 5000 dimensiones necesitaría demasiadas muestras.
3. (c) Choques con contactos: el simulador es discontinuo y la diferenciación automática da gradientes poco útiles; con 20 parámetros, un sustituto (u optimización sin derivadas) es más fiable.

*Resultado.* **(a) sustituto; (b) adjunto; (c) sustituto u optimización sin derivadas**.

*Comprobación.* coincide con el caso de la viga MBB (adjunto, 1200 variables) y con el de la aleta (2 variables: ambos métodos sirven).

### Solución del ejercicio 3.2

*Datos.* aleta rectangular: $k\,t\,\theta'' - 2h\theta = 0$, $\theta(0) = \theta_b$, $\theta'(L) = 0$.

*Planteamiento.* la pérdida PINN suma el residuo en puntos de colocación y las condiciones de contorno [33].

*Desarrollo.*

1. Residuo: $\mathcal{L}_R = \frac{1}{M}\sum_{j=1}^{M}\big(k\,t\,\theta''_\varphi(x_j) - 2h\,\theta_\varphi(x_j)\big)^2$ con $x_j \in (0, L)$; $\theta''_\varphi$ se obtiene por diferenciación automática respecto a la entrada.
2. Contorno: $\mathcal{L}_B = \big(\theta_\varphi(0) - \theta_b\big)^2 + \big(\theta'_\varphi(L)\big)^2$.
3. Pérdida: $\mathcal{L} = \mathcal{L}_R + w_B\mathcal{L}_B$, con un peso $w_B$ que equilibra las escalas.

*Resultado.* $\mathcal{L} = \frac{1}{M}\sum_j (kt\theta''_\varphi - 2h\theta_\varphi)^2(x_j) + w_B\big[(\theta_\varphi(0) - \theta_b)^2 + \theta'_\varphi(L)^2\big]$. No hace falta malla porque los puntos de colocación pueden ser aleatorios y las derivadas se calculan sobre la red, no con diferencias entre nodos.

*Comprobación.* la solución exacta $\theta = \theta_b \cosh(m(L-x))/\cosh(mL)$ anula ambos términos.

### Solución del ejercicio 3.3

*Datos.* $L = 0{,}04$ m; $t = 0{,}0015$ m; $h = 60$; $k = 200$; $\theta_b = 50$ K.

*Planteamiento.* $m = \sqrt{2h/(kt)}$, $q = \sqrt{2hkt}\,\theta_b \tanh(mL)$, $\eta = \tanh(mL)/(mL)$.

*Desarrollo.*

1. $m = \sqrt{120/(200 \cdot 0{,}0015)} = \sqrt{400} = 20$ m$^{-1}$; $mL = 0{,}8$.
2. $\tanh 0{,}8 = 0{,}66404$; $\sqrt{2hkt} = \sqrt{120 \cdot 200 \cdot 0{,}0015} = \sqrt{36} = 6$ W/(m · K).
3. $q = 6 \cdot 50 \cdot 0{,}66404 = 199{,}21$ W/m.
4. $\eta = 0{,}66404/0{,}8 = 0{,}830$; Biot $= 60 \cdot 0{,}0015/400 = 2{,}25 \cdot 10^{-4}$.

*Resultado.* $m = 20$ **m$^{-1}$,** $q = 199{,}2$ **W/m,** $\eta = 0{,}830$**,** Bi $= 2{,}3 \cdot 10^{-4}$.

*Comprobación.* el cuaderno da $q(0{,}04) = 199{,}21$ W/m; está por debajo del óptimo (220,4 W/m en $L = 58{,}6$ mm), porque esta aleta es demasiado corta y gruesa.

### Solución del ejercicio 3.4

*Datos.* $N = 1200$ variables; 10 ms por resolución; 100 iteraciones.

*Planteamiento.* diferencias hacia delante: $N + 1$ resoluciones; adjunto: la del estado más una adjunta (cero si es autoadjunto).

*Desarrollo.*

1. Diferencias finitas: $1201 \cdot 10$ ms $= 12{,}01$ s por gradiente; 100 iteraciones, 1201 s ($\approx$ 20 min).
2. Adjunto: 1 resolución (flexibilidad autoadjunta), 10 ms; en general 2, 20 ms. 100 iteraciones: entre 1 y 2 s.
3. Cociente: 1201 (o 600 en el caso general).

*Resultado.* **12 s frente a 0,01–0,02 s por gradiente; 20 min frente a 1–2 s en total**.

*Comprobación.* el cuaderno resuelve la viga MBB en 106 iteraciones y unos 1,6 s, coherente con el adjunto.

### Solución del ejercicio 3.5

*Datos.* $A(0) = \left(\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right)$; $b = (1, 0)^\top$; $J = u_1 + u_2$; $\partial A/\partial\theta = \left(\begin{smallmatrix} 1 & 0 \\ 0 & 0 \end{smallmatrix}\right)$.

*Planteamiento.* teorema 3.1: $\partial R/\partial\theta = (\partial A/\partial\theta)u$, adjunto $A^\top\lambda = (1, 1)^\top$.

*Desarrollo.*

1. $\det A = 3$; $u = A^{-1}b = \frac{1}{3}\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)\left(\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right) = (2/3,\ 1/3)$; $J = 1$.
2. $A^\top \lambda = (1,1)^\top$: como $A(1,1)^\top = (1,1)^\top$, $\lambda = (1,1)$.
3. $dJ/d\theta = -\lambda^\top (\partial A/\partial\theta) u = -\lambda_1 u_1 = -2/3$.
4. Diferencia finita: con $\theta = 0{,}01$, $\det = 4{,}02 - 1 = 3{,}02$, $u = (2/3{,}02,\ 1/3{,}02)$, $J = 3/3{,}02 = 0{,}99338$; pendiente $(0{,}99338 - 1)/0{,}01 = -0{,}662$.

*Resultado.* $\mathbf{dJ/d\theta = -2/3 \approx -0{,}667}$.

*Comprobación.* la diferencia finita hacia delante, $-0{,}662$, difiere en $O(\Delta\theta)$, como corresponde a un esquema de primer orden.

### Solución del ejercicio 3.6

*Datos.* $(L, q) = (30;\ 165{,}377)$, $(60;\ 220{,}336)$, $(90;\ 198{,}201)$; óptimo verdadero $L = 58{,}6$ mm, $q = 220{,}42$ W/m.

*Planteamiento.* con tres puntos, el ajuste por mínimos cuadrados interpola; para nodos equiespaciados ($h = 30$), $\beta_2 = (q_3 - 2q_2 + q_1)/(2h^2)$ y el vértice está en $L^\star = L_2 - h(q_3 - q_1)/\big(2(q_3 - 2q_2 + q_1)\big)$.

*Desarrollo.*

1. Segunda diferencia: $198{,}201 - 440{,}673 + 165{,}377 = -77{,}095$; $\beta_2 = -77{,}095/1800 = -0{,}04283$ W/(m · mm²).
2. $q_3 - q_1 = 32{,}824$; $L^\star = 60 - 30 \cdot 32{,}824/(2 \cdot (-77{,}095)) = 60 + 6{,}386 = 66{,}39$ mm.
3. Máximo predicho: $q_2 - (q_3 - q_1)^2/\big(8(q_3 - 2q_2 + q_1)\big) = 220{,}336 + 1077{,}4/616{,}76 = 222{,}08$ W/m.
4. Verificación con el simulador en $L^\star$: 218,13 W/m.

*Resultado.* **el sustituto propone $L = 66{,}4$ mm y predice 222,1 W/m; el valor real es 218,1 W/m, un 1,0 % por debajo del óptimo verdadero y 4 W/m por debajo de lo prometido**.

*Comprobación.* la curva real es asimétrica (crece deprisa y decae despacio), y una parábola desplaza el vértice hacia la derecha. Se ilustra la proposición 3.3: el error del sustituto se traduce en pérdida de calidad, y la verificación lo revela.

### Solución del ejercicio 3.7

*Datos.* $\theta_{k+1} = (1 - a\Delta t)\theta_k$; $\theta_0 = 50$; $\Delta t = 1$; $a = 0{,}1$; $N = 3$.

*Planteamiento.* derivada de $\theta_3 = \theta_0(1 - a\Delta t)^3$; recurrencia adjunta con $\Phi(\theta, a) = (1 - a\Delta t)\theta$, $\partial\Phi/\partial\theta = 1 - a\Delta t$, $\partial\Phi/\partial a = -\Delta t\, \theta$.

*Desarrollo.*

1. Estados: $\theta_1 = 45$; $\theta_2 = 40{,}5$; $\theta_3 = 36{,}45$ K.
2. Cerrada: $d\theta_3/da = -3\Delta t\, \theta_0 (1 - a\Delta t)^2 = -3 \cdot 50 \cdot 0{,}81 = -121{,}5$ K · s.
3. Adjunta: $\lambda_3 = 1$; $\lambda_2 = 0{,}9$; $\lambda_1 = 0{,}81$.
4. $d\theta_3/da = \sum_{k=1}^{3} \lambda_k (-\Delta t\, \theta_{k-1}) = -(1 \cdot 40{,}5 + 0{,}9 \cdot 45 + 0{,}81 \cdot 50) = -(40{,}5 + 40{,}5 + 40{,}5) = -121{,}5$.

*Resultado.* $\theta_3 = \mathbf{36{,}45\ K}$; $\mathbf{d\theta_3/da = -121{,}5\ K \cdot s}$.

*Comprobación.* diferencia finita: con $a = 0{,}101$, $\theta_3 = 50 \cdot 0{,}899^3 = 36{,}329$; $(36{,}329 - 36{,}45)/0{,}001 = -121{,}4$.

### Solución del ejercicio 3.8

*Datos.* $\sup|\hat{s} - s| \le \varepsilon$; validación en tres puntos.

*Planteamiento.* demostración de la proposición 3.3; contraejemplo con un sustituto que coincide en los puntos de validación.

*Desarrollo.*

1. (a) $s(\hat{\theta}) \le \hat{s}(\hat{\theta}) + \varepsilon \le \hat{s}(\theta^\star) + \varepsilon \le s(\theta^\star) + 2\varepsilon$.
2. (b) Sea $s(\theta) = \theta$ en $[0, 1]$, con mínimo en 0, y sea $\hat{s}(\theta) = \theta - 10\,\phi(\theta)$, donde $\phi$ es un pulso continuo que vale 1 en $\theta = 0{,}9$ y 0 fuera de $(0{,}85;\ 0{,}95)$.
3. Validación en $\{0{,}2;\ 0{,}5;\ 0{,}8\}$: error nulo en los tres.
4. El mínimo de $\hat{s}$ está en 0,9 ($\hat{s} = -9{,}1$), donde $s = 0{,}9$: el peor valor salvo un 10 % del intervalo.

*Resultado.* **la cota exige error uniforme; un error de validación nulo en unos puntos no la garantiza**. El optimizador va precisamente a las zonas no validadas en que el sustituto es optimista; por eso se verifica el óptimo con el simulador y se añade a los datos.

*Comprobación.* en el ejemplo, $\varepsilon = \sup|\hat{s} - s| = 10$ y la cota da $s(\hat{\theta}) \le 20$, que se cumple (0,9): la proposición no se viola, solo es inútil si $\varepsilon$ es grande.

### Solución del ejercicio 3.9

*Datos.* $q(L) = \sqrt{2hkA/L}\,\theta_b \tanh(cL^{3/2})$, $c = \sqrt{2h/(kA)}$; (b) $h = 40$, $k = 170$, $A = 5 \cdot 10^{-5}$ m$^2$, $\theta_b = 50$ K.

*Planteamiento.* derivar $\ln q$ respecto a $L$ e igualar a cero; cambiar a la variable $\beta = cL^{3/2}$.

*Desarrollo.*

1. Con $t = A/L$, $m = \sqrt{2h/(kt)} = \sqrt{2hL/(kA)} = cL^{1/2}$ y $mL = cL^{3/2} = \beta$; además $\sqrt{2hkt} = \sqrt{2hkA/L}$. Esto da la expresión de $q$.
2. $\ln q = \text{cte} - \frac{1}{2}\ln L + \ln\tanh\beta$, y $d\beta/dL = \frac{3}{2}\beta/L$.
3. $\dfrac{d\ln q}{dL} = -\dfrac{1}{2L} + \dfrac{\operatorname{sech}^2\beta}{\tanh\beta}\cdot\dfrac{3\beta}{2L} = 0 \iff \tanh\beta = 3\beta\operatorname{sech}^2\beta \iff \sinh\beta\cosh\beta = 3\beta \iff \sinh 2\beta = 6\beta$.
4. (b) $c = \sqrt{80/(170 \cdot 5 \cdot 10^{-5})} = \sqrt{9411{,}8} = 97{,}014\ \text{m}^{-3/2}$.
5. $L = (1{,}4192/97{,}014)^{2/3} = 0{,}05982$ m; $t = A/L = 0{,}836$ mm.
6. $q = \sqrt{2 \cdot 40 \cdot 170 \cdot 5 \cdot 10^{-5}/0{,}05982} \cdot 50 \cdot \tanh 1{,}4192 = 3{,}3717 \cdot 50 \cdot 0{,}8894 = 149{,}9$ W/m.

*Resultado.* $\sinh 2\beta = 6\beta$**,** $\beta = 1{,}4192$**;** $L = 59{,}8$ **mm,** $t = 0{,}836$ **mm,** $q = 149{,}9$ **W/m**.

*Comprobación.* $\sinh(2{,}8384) = 8{,}5154$ y $6 \cdot 1{,}4192 = 8{,}5153$. Para $\beta$ pequeño, $\sinh 2\beta \approx 2\beta < 6\beta$, y para $\beta$ grande, $\sinh 2\beta > 6\beta$: hay una única raíz positiva, el máximo.

## A.4 Tema 4

### Solución del ejercicio 4.1

*Datos.* rasterizador clásico y pérdida por píxeles.

*Planteamiento.* proposición 4.1 y definición 4.2.

*Desarrollo.*

1. Cada píxel vale 0 o 1 y es constante en un entorno de casi todas las configuraciones de vértices: el gradiente de la pérdida respecto a los vértices es nulo casi en todas partes.
2. Por tanto, el descenso de gradiente no mueve la malla, aunque la silueta esté desplazada.
3. SoftRas sustituye la indicadora por una sigmoide de la distancia al borde: los píxeles cercanos al contorno, dentro y fuera, dependen de forma suave de los vértices.
4. Compromiso de $\sigma$: grande, gradientes desde lejos pero silueta borrosa y sesgada; pequeño, fidelidad pero gradientes solo en una franja estrecha.

*Resultado.* **el gradiente es nulo casi en todas partes; SoftRas lo hace informativo a cambio de un sesgo controlado por $\sigma$, que conviene reducir progresivamente**.

*Comprobación.* en el ejemplo 4.4, el recocido de $\tau$ (el análogo de $\sigma$) eleva la IoU volumétrica de 0,744 a 0,991.

### Solución del ejercicio 4.2

*Datos.* taza con asa; 50 siluetas alrededor.

*Planteamiento.* definición de casco visual [23].

*Desarrollo.*

1. Una silueta solo informa de qué rayos cortan el objeto. El casco visual es la intersección de los conos de visión de todas las siluetas: el mayor sólido compatible con ellas.
2. El hueco interior de la taza está rodeado por la pared en todas las direcciones horizontales: ningún rayo de una vista lateral lo atraviesa sin cortar la pared. Desde arriba, el rayo que entra en el hueco corta el fondo, así que la silueta tampoco lo revela.
3. El casco visual es, por tanto, una taza «maciza»; el asa sí se recupera, porque su agujero se ve de perfil desde algunas vistas.
4. Información adicional: color o sombreado (consistencia fotométrica entre vistas, como en NeRF), mapas de profundidad o luz estructurada.

*Resultado.* **no; el interior es una concavidad que ninguna silueta ve, y hace falta información fotométrica o de profundidad**.

*Comprobación.* el agujero del asa es una característica de género (un asa, $g = 1$) visible en las siluetas laterales, mientras que el hueco de la taza no cambia ninguna silueta: coherente con la definición.

### Solución del ejercicio 4.3

*Datos.* $\sigma = 10^{-4}$; distancias 0,01 (dentro), 0,01 y 0,02 (fuera); dos triángulos con $D = 0{,}7311$ y 0,2689.

*Planteamiento.* $D = \mathrm{sig}(\delta d^2/\sigma)$; $I = 1 - \prod(1 - D_j)$.

*Desarrollo.*

1. Dentro, $d = 0{,}01$: $d^2/\sigma = 10^{-4}/10^{-4} = 1$; $D = \mathrm{sig}(1) = 0{,}7311$.
2. Fuera, $d = 0{,}01$: $D = \mathrm{sig}(-1) = 0{,}2689$.
3. Fuera, $d = 0{,}02$: $d^2/\sigma = 4$; $D = \mathrm{sig}(-4) = 0{,}01799$.
4. Agregado: $1 - (1 - 0{,}7311)(1 - 0{,}2689) = 1 - 0{,}2689 \cdot 0{,}7311 = 1 - 0{,}1966 = 0{,}8034$.

*Resultado.* $D = 0{,}731$**;** $0{,}269$**;** $0{,}0180$**; silueta agregada** $0{,}803$.

*Comprobación.* $\mathrm{sig}(1) + \mathrm{sig}(-1) = 1$; el agregado supera al mayor de los dos valores, como corresponde a la probabilidad de una unión.

### Solución del ejercicio 4.4

*Datos.* $\delta_i = 0{,}5$; $\sigma = (0{,}2;\ 1{,}0;\ 3{,}0)$; $c = (0{,}1;\ 0{,}8;\ 0{,}4)$; $t = (1{,}0;\ 1{,}5;\ 2{,}0)$.

*Planteamiento.* cuadratura de NeRF: $\alpha_i = 1 - e^{-\sigma_i \delta_i}$, $T_i = \prod_{j<i}(1 - \alpha_j)$, $w_i = T_i \alpha_i$.

*Desarrollo.*

1. $\sigma\delta = (0{,}1;\ 0{,}5;\ 1{,}5)$; $\alpha = (0{,}09516;\ 0{,}39347;\ 0{,}77687)$.
2. $T = (1;\ e^{-0{,}1};\ e^{-0{,}6}) = (1;\ 0{,}90484;\ 0{,}54881)$.
3. $w = (0{,}09516;\ 0{,}35603;\ 0{,}42636)$.
4. Color: $0{,}1 \cdot 0{,}09516 + 0{,}8 \cdot 0{,}35603 + 0{,}4 \cdot 0{,}42636 = 0{,}00952 + 0{,}28482 + 0{,}17054 = 0{,}46488$.
5. Profundidad: $\sum w_i t_i = 0{,}09516 + 0{,}53405 + 0{,}85271 = 1{,}4819$; normalizada por $\sum w_i$: $1{,}6887$.

*Resultado.* **pesos** $(0{,}0952;\ 0{,}3560;\ 0{,}4264)$**; color** $0{,}465$**; profundidad** $1{,}482$ **(o** $1{,}689$ **normalizada)**.

*Comprobación.* $\sum w_i = 0{,}87754 = 1 - e^{-2{,}1}$, como predice la proposición 4.3; el resto, $0{,}1225$, es la transmitancia hacia el fondo.

### Solución del ejercicio 4.5

*Datos.* $o = (0{,}8;\ 0{,}6;\ 0{,}9)$; $P = (-0{,}5;\ -0{,}1;\ -2{,}0)$; colores rojo, verde y azul.

*Planteamiento.* $\alpha_i = o_i e^{P_i}$, $T_i = \prod_{j<i}(1 - \alpha_j)$, $C = \sum_i T_i \alpha_i c_i$.

*Desarrollo.*

1. $\alpha_1 = 0{,}8 \cdot 0{,}60653 = 0{,}48522$; $\alpha_2 = 0{,}6 \cdot 0{,}90484 = 0{,}54290$; $\alpha_3 = 0{,}9 \cdot 0{,}13534 = 0{,}12180$.
2. $T_1 = 1$; $T_2 = 0{,}51478$; $T_3 = 0{,}51478 \cdot 0{,}45710 = 0{,}23530$.
3. Contribuciones: $0{,}48522$ (R), $0{,}51478 \cdot 0{,}54290 = 0{,}27947$ (G), $0{,}23530 \cdot 0{,}12180 = 0{,}02866$ (B).

*Resultado.* $\mathbf{C} = (\mathbf{0{,}485};\ \mathbf{0{,}279};\ \mathbf{0{,}029})$.

*Comprobación.* opacidad acumulada $0{,}79336 = 1 - T_4$, con $T_4 = 0{,}23530 \cdot 0{,}87820 = 0{,}20664$. Aunque la tercera gaussiana es muy opaca, aporta poco porque está detrás y el píxel queda lejos de su centro.

### Solución del ejercicio 4.6

*Datos.* $p = 0{,}25$; $L = 2$; posición con $L = 10$ y dirección con $L = 4$.

*Planteamiento.* $\gamma(p) = (\sin \pi p, \cos \pi p, \sin 2\pi p, \cos 2\pi p, \dots)$, aplicada a cada coordenada.

*Desarrollo.*

1. $\sin(\pi/4) = 0{,}7071$; $\cos(\pi/4) = 0{,}7071$; $\sin(\pi/2) = 1$; $\cos(\pi/2) = 0$.
2. Posición: 3 coordenadas $\times$ 2 funciones $\times$ 10 frecuencias $= 60$.
3. Dirección (vector unitario de 3 componentes): $3 \cdot 2 \cdot 4 = 24$.

*Resultado.* $\gamma(\mathbf{0{,}25}) = (\mathbf{0{,}7071};\ \mathbf{0{,}7071};\ \mathbf{1};\ \mathbf{0})$; **60 y 24 componentes**.

*Comprobación.* cada par $(\sin, \cos)$ tiene norma 1: $0{,}7071^2 + 0{,}7071^2 = 1$ y $1^2 + 0^2 = 1$. La codificación es la misma idea que la posicional sinusoidal del Transformer (módulo 14, §4.2).

### Solución del ejercicio 4.7

*Datos.* $S = (0{,}9;\ 0{,}6;\ 0{,}2;\ 0)$; $T = (1;\ 1;\ 0;\ 0)$.

*Planteamiento.* $I = \sum S_i T_i$, $U = \sum (S_i + T_i - S_i T_i)$; $\partial I / \partial S_i = T_i$, $\partial U / \partial S_i = 1 - T_i$.

*Desarrollo.*

1. $I = 0{,}9 + 0{,}6 = 1{,}5$; $U = 1 + 1 + 0{,}2 + 0 = 2{,}2$.
2. IoU $= 1{,}5/2{,}2 = 0{,}6818$; pérdida $0{,}3182$.
3. $\dfrac{\partial \,\mathrm{IoU}}{\partial S_3} = \dfrac{T_3 U - I(1 - T_3)}{U^2} = \dfrac{0 - 1{,}5}{4{,}84} = -0{,}3099$.

*Resultado.* IoU $= 0{,}682$**, pérdida** $0{,}318$**,** $\partial\,\mathrm{IoU}\,/\partial S_3 = -0{,}310$. El signo negativo indica que el píxel 3, que está fuera del objetivo, debe apagarse.

*Comprobación.* para un píxel dentro del objetivo, $\partial\,\mathrm{IoU}\,/\partial S_1 = (1 \cdot 2{,}2 - 0)/4{,}84 = 0{,}4545 > 0$: hay que encenderlo más. Los signos son los esperados.

### Solución del ejercicio 4.8

*Datos.* cuadratura de NeRF con $\sigma_i \geq 0$.

*Planteamiento.* suma telescópica; división de un intervalo en dos.

*Desarrollo.*

1. (a) Es la demostración de la proposición 4.3: $w_i = T_i - T_{i+1}$ y $\sum_i w_i = 1 - T_{N+1} \in [0, 1]$.
2. (b) Sea un intervalo de longitud $\delta$ y densidad $\sigma$ con transmitancia de entrada $T$. Sin dividir, su peso es $T(1 - e^{-\sigma\delta})$.
3. Dividido en dos mitades: $\alpha' = 1 - e^{-\sigma\delta/2}$; pesos $T\alpha'$ y $T(1-\alpha')\alpha'$; suma $T\big(1-(1-\alpha')^2\big) = T(1 - e^{-\sigma\delta})$, el peso original.
4. La transmitancia de salida es $T(1 - \alpha')^2 = Te^{-\sigma\delta}$, la misma que sin dividir, de modo que los pesos posteriores no cambian.
5. (c) La fórmula del *splatting* es $\sum_i c_i \alpha_i \prod_{j<i}(1 - \alpha_j) = \sum_i T_i \alpha_i c_i$, la misma composición con $\alpha_i$ dado directamente por la gaussiana en lugar de por $1 - e^{-\sigma_i \delta_i}$.

*Resultado.* **los pesos son no negativos y suman $1 - T_{N+1}$; la cuadratura es coherente al refinar; NeRF y *splatting* comparten la composición alfa**.

*Comprobación.* con los datos del ejercicio 4.4, la primera muestra ($\sigma\delta = 0{,}1$) dividida en dos da $0{,}04877 + 0{,}95123 \cdot 0{,}04877 = 0{,}09516$, igual que antes.

### Solución del ejercicio 4.9

*Datos.* $\Sigma = RSS^\top R^\top$, $R$ ortogonal, $S = \operatorname{diag}(s_1, s_2, s_3)$; $\Sigma' = JW\Sigma W^\top J^\top$.

*Planteamiento.* escribir $\Sigma = MM^\top$ y usar $y^\top MM^\top y = \|M^\top y\|^2$.

*Desarrollo.*

1. (a) Con $M = RS$, $\Sigma = MM^\top$, que es simétrica: $(MM^\top)^\top = MM^\top$.
2. $y^\top \Sigma y = \|M^\top y\|^2 \geq 0$: semidefinida positiva.
3. $y^\top \Sigma y = 0 \iff M^\top y = 0$. Como $M^\top = SR^\top$ y $R^\top$ es invertible, $M^\top$ es invertible si y solo si $\det S = s_1 s_2 s_3 \neq 0$. Así, $\Sigma$ es definida positiva si y solo si todas las $s_i$ son no nulas (sus autovalores son $s_i^2$ y sus autovectores, las columnas de $R$, por el teorema espectral del módulo 3).
4. (b) $\Sigma' = (JWM)(JWM)^\top$, de la misma forma, luego semidefinida positiva.
5. (c) Un paso de gradiente sobre las seis entradas libres puede dar una matriz con un autovalor negativo (por ejemplo, $\left(\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right)$ tiene autovalores 3 y $-1$), que no es una covarianza; la gaussiana dejaría de estar definida.

*Resultado.* **la parametrización $RSS^\top R^\top$ garantiza covarianzas válidas por construcción; la directa no**.

*Comprobación.* con $R = I$ y $S = \operatorname{diag}(2, 1, 0{,}5)$, $\Sigma = \operatorname{diag}(4, 1, 0{,}25)$, definida positiva; con $s_3 = 0$ la gaussiana degenera en un disco plano.

## A.5 Tema 5

### Solución del ejercicio 5.1

*Datos.* $\nu = 30, 60, 90, 120, 144$ Hz.

*Planteamiento.* $1000/\nu$ ms (definición 5.1).

*Desarrollo.*

1. 33,33; 16,67; 11,11; 8,33; 6,94 ms.
2. Una media de 16,7 ms puede esconder fotogramas de 30 ms compensados por otros de 10 ms: cada fotograma que supera el presupuesto se pierde o se repite, y el jugador percibe un tirón.

*Resultado.* 33,3 / 16,7 / 11,1 / 8,3 / 6,9 **ms; el presupuesto debe cumplirse en un percentil alto, no en la media**.

*Comprobación.* $16{,}67 \cdot 60 = 1000$ ms.

### Solución del ejercicio 5.2

*Datos.* planos $z = 0$, $p_1 = (0, 0, 1, 0)$, y $x = 0$, $p_2 = (1, 0, 0, 0)$; punto $(0{,}1;\ 0{,}2;\ 0{,}3)$.

*Planteamiento.* $Q = p_1 p_1^\top + p_2 p_2^\top$ y $\Delta(v) = \bar{v}^\top Q \bar{v}$ (definición 5.2).

*Desarrollo.*

1. $Q = \mathrm{diag}(1, 0, 1, 0)$.
2. $\Delta = 0{,}1^2 + 0{,}3^2 = 0{,}01 + 0{,}09 = 0{,}10$, la suma de las distancias al cuadrado a $x = 0$ y $z = 0$.
3. El bloque $A = \mathrm{diag}(1, 0, 1)$ es singular porque las normales $(0, 0, 1)$ y $(1, 0, 0)$ no generan $\mathbb{R}^3$ (proposición 5.3). Error nulo en toda la recta $x = z = 0$ (el eje $y$, la arista entre los dos planos).
4. El algoritmo no resuelve el sistema: evalúa el error en los extremos de la arista y en su punto medio y toma el menor [17, §4].

*Resultado.* $Q = \mathrm{diag}(1, 0, 1, 0)$**;** $\Delta = 0{,}10$**; sin solución única (recta** $x = z = 0$**); se recurre a los extremos o al punto medio**.

*Comprobación.* el vértice puede deslizarse por la arista del diedro sin cambiar la forma: es lo esperado en un borde recto.

### Solución del ejercicio 5.3

*Datos.* $2048 \times 2048$ téxeles; RGBA8 (32 bits), BC5 (8 bits), BC1 (4 bits); mipmaps $\times 4/3$.

*Planteamiento.* memoria $= 2048^2 \cdot \mathrm{bpp}/8 \cdot 4/3$ bytes.

*Desarrollo.*

1. $2048^2 = 4\,194\,304$ téxeles.
2. RGBA8: $4\,194\,304 \cdot 4 \cdot 4/3 = 22\,369\,621$ B $= 21{,}33$ MiB.
3. BC5: un cuarto, 5,33 MiB; BC1: un octavo, 2,67 MiB.

*Resultado.* 21,33 / 5,33 / 2,67 **MiB**. BC5 codifica dos canales independientes con 8 bytes cada uno: las componentes $x, y$ de la normal se guardan con precisión y $z$ se reconstruye con $z = \sqrt{1 - x^2 - y^2}$; BC1 obliga a que los tres canales compartan una única recta de color por bloque, lo que distorsiona las normales.

*Comprobación.* la cuenta coincide con la del cuaderno escalada por 1/4 (allí $4096^2$): 85,33/4 = 21,33.

### Solución del ejercicio 5.4

*Datos.* $y = (10, 40, 70, 95)$; $w = (0, \frac{1}{3}, \frac{2}{3}, 1)$.

*Planteamiento.* mínimos cuadrados con filas $(1 - w_i,\ w_i)$: ecuaciones normales $M^\top M e = M^\top y$ (módulo 8, §2.2).

*Desarrollo.*

1. $M^\top M = \begin{pmatrix} \sum (1-w)^2 & \sum w(1-w) \\ \sum w(1-w) & \sum w^2 \end{pmatrix} = \begin{pmatrix} 14/9 & 4/9 \\ 4/9 & 14/9 \end{pmatrix}$.
2. $M^\top y = \left(10 + \frac{2}{3}40 + \frac{1}{3}70,\ \frac{1}{3}40 + \frac{2}{3}70 + 95\right) = (60,\ 155)$.
3. Multiplicando por 9: $14e_0 + 4e_1 = 540$ y $4e_0 + 14e_1 = 1395$. Restando las ecuaciones: $10(e_1 - e_0) = 855$, $e_1 - e_0 = 85{,}5$; sumándolas: $18(e_0 + e_1) = 1935$, $e_0 + e_1 = 107{,}5$.
4. $e_0 = 11$, $e_1 = 96{,}5$. Reconstrucción: (11; 39,5; 68; 96,5); residuos $(-1;\ 0{,}5;\ 2;\ -1{,}5)$.

5. Error cuadrático: $1 + 0{,}25 + 4 + 2{,}25 = 7{,}5$.

*Resultado.* **$\mathbf{e_0} = \mathbf{11}$, $\mathbf{e_1} = \mathbf{96{,}5}$; error cuadrático** 7,5.

*Comprobación.* los residuos son ortogonales a las columnas de $M$: $\sum(1 - w_i)r_i = -1 + \frac{2}{3}0{,}5 + \frac{1}{3}2 + 0 = 0$ y $\sum w_i r_i = \frac{1}{3}0{,}5 + \frac{2}{3}2 - 1{,}5 = 0$.

### Solución del ejercicio 5.5

*Datos.* $\varepsilon = 2$ mm; $\phi_v = 60°$; $H = 1080$ y 2160; $\tau = 1$ px.

*Planteamiento.* proposición 5.5: $d^\star = \varepsilon H/(2\tau \tan 30°)$.

*Desarrollo.*

1. $\tan 30° = 0{,}57735$.
2. $H = 1080$: $d^\star = 0{,}002 \cdot 1080/(2 \cdot 0{,}57735) = 1{,}871$ m.
3. $H = 2160$: el doble, 3,741 m.

*Resultado.* **a partir de** 1,87 **m en 1080p y de** 3,74 **m en 2160p**.

*Comprobación.* a 1,871 m la altura visible es $2 \cdot 1{,}871 \cdot 0{,}57735 = 2{,}160$ m en 1080 píxeles: 2 mm por píxel, justo el error.

### Solución del ejercicio 5.6

*Datos.* grupo 1 (peso 1): $(2{,}0; 0)$, $(1{,}2; 1)$, $(0{,}5; 3)$; grupo 2 (peso 2): $(1{,}5; 0)$, $(0{,}8; 0{,}5)$, $(0{,}4; 2)$; $B = 2{,}5$ ms.

*Planteamiento.* enumerar las 9 combinaciones; $D(\lambda) = \sum_g \min_\ell(q_{g\ell} + \lambda t_{g\ell}) - \lambda B$.

*Desarrollo.*

1. Pérdidas ponderadas del grupo 2: 0; 1; 4.
2. Factibles ($T \le 2{,}5$): $(0, 2)$: 2,4 ms, $Q = 4$; $(1, 1)$: 2,0 ms, $Q = 2$; $(1, 2)$: 1,6, $Q = 5$; $(2, 0)$: 2,0, $Q = 3$; $(2, 1)$: 1,3, $Q = 4$; $(2, 2)$: 0,9, $Q = 7$. Óptimo: $(1, 1)$ con $Q^\star = 2$.
3. $\lambda = 1{,}25$: en el grupo 1, los valores de $q + \lambda t$ son 2,5, 2,5 y 3,625, con mínimo 2,5; en el grupo 2, son 1,875, 2,0 y 4,5, con mínimo 1,875. Entonces $D = 4{,}375 - 3{,}125 = 1{,}25$.
4. $\lambda = 10/7$: en el grupo 1, los valores de $q + \lambda t$ son 2,857, 2,714 y 3,714, con mínimo 2,714; en el grupo 2, son 2,143, 2,143 y 4,571, con mínimo 2,143 (empate). Entonces $D = 4{,}857 - 3{,}571 = 1{,}286 = 9/7$.

*Resultado.* **óptimo** $(\mathbf{LOD1}, \mathbf{LOD1})$ **con 2,0 ms y** $Q^\star = 2$**;** $D(1{,}25) = 1{,}25$ **y** $D(10/7) = 9/7 \approx 1{,}29$, ambas menores que $Q^\star$.

*Comprobación.* la proposición 5.7 se cumple: $1{,}25 \le 2$ y $1{,}29 \le 2$. El hueco de 5/7 se debe a que la elección es discreta y el óptimo deja 0,5 ms sin usar.

### Solución del ejercicio 5.7

*Datos.* atlas $1024 \times 1024$; seis cartas.

*Planteamiento.* ordenar por altura decreciente y llenar estantes de izquierda a derecha; la altura del estante es la de su primera carta.

*Desarrollo.*

1. Orden: $512 \times 300$, $400 \times 300$, $300 \times 280$, $600 \times 250$, $350 \times 240$, $500 \times 200$.
2. Estante 1 (alto 300): $512 + 400 = 912$; la siguiente (300) no cabe ($1212 > 1024$).
3. Estante 2 (alto 280): $300 + 600 = 900$; la siguiente (350) no cabe.
4. Estante 3 (alto 240): $350 + 500 = 850$.
5. Altura total: $300 + 280 + 240 = 820 \leq 1024$: caben.
6. Área útil: $153\,600 + 120\,000 + 84\,000 + 150\,000 + 84\,000 + 100\,000 = 691\,600$; fracción $691\,600/1\,048\,576 = 0{,}660$.

*Resultado.* **caben en tres estantes; se aprovecha el 66 % del atlas**.

*Comprobación.* queda una franja libre de $1024 \times 204$ (20 %) y huecos dentro de los estantes; una heurística más fina (*skyline*) podría colocar la carta de $600 \times 250$ en otro sitio.

### Solución del ejercicio 5.8

*Datos.* cuádricas de la definición 5.2.

*Planteamiento.* demostración de la proposición 5.3; linealidad de $\bar{v}^\top Q \bar{v}$ en $Q$.

*Desarrollo.*

1. (a)–(c) Como en la demostración de la proposición 5.3.
2. $\bar{v}^\top (Q_1 + Q_2)\bar{v} = \bar{v}^\top Q_1 \bar{v} + \bar{v}^\top Q_2 \bar{v} = \sum_{p \in P_1} d_p(v)^2 + \sum_{p \in P_2} d_p(v)^2$: la suma de distancias al cuadrado a todos los planos de ambos conjuntos. Los planos de las caras compartidas por los dos vértices se cuentan dos veces; el artículo lo acepta como aproximación [17, §5].
3. Como cada término es $\geq 0$, $\Delta_{Q_1+Q_2}(v) \geq \max\big(\Delta_{Q_1}(v), \Delta_{Q_2}(v)\big)$.

*Resultado.* **la cuádrica acumulada mide la distancia a la unión de los planos, y el error del colapso domina a cada error individual**. Por eso los costes crecen a lo largo de la simplificación y el montículo prioriza primero las zonas planas.

*Comprobación.* en el ejemplo 5.4, la esquina tiene error nulo en el origen; colapsar dos vértices de la misma cara plana también da error nulo, porque ambos comparten el plano.

### Solución del ejercicio 5.9

*Datos.* problema de la definición 5.6; ejemplo 5.8.

*Planteamiento.* demostración de la proposición 5.7; mínimo de funciones afines.

*Desarrollo.*

1. La dualidad débil y el criterio de optimalidad se demuestran como en la proposición 5.7.
2. Para cada $e$ fija, $\lambda \mapsto Q(e) + \lambda(T(e) - B)$ es afín. $D$ es el mínimo de un número finito de funciones afines, luego cóncava (el mínimo puntual de funciones cóncavas es cóncavo).
3. En el ejemplo 5.8, la mejor cota (2,30) se alcanza en $\lambda = 1{,}25$, cuya elección $(2, 2, 1, 0)$ es factible pero deja holgura ($T = 3{,}8 < 4$), así que $\lambda(T - B) = -0{,}25 \neq 0$ y no se cumple la holgura complementaria.
4. Ninguna elección consume exactamente 4,0 ms con menos pérdida: la discreción impide cerrar el hueco.

*Resultado.* $D(\lambda) \leq Q^\star$**,** $D$ **es cóncava, y el hueco del ejemplo,** $2{,}55 - 2{,}30 = 0{,}25$**, se debe a la discreción**.

*Comprobación.* $D(1{,}25) = Q(e) + 1{,}25(3{,}8 - 4) = 2{,}55 - 0{,}25 = 2{,}30$, que coincide con el valor del cuaderno.

## A.6 Tema 6

### Solución del ejercicio 6.1

*Datos.* pieza para imprimir y recurso para motor.

*Planteamiento.* sección 6.2.

*Desarrollo.*

1. Impresión: estanqueidad (cerrada, variedad, orientada); voladizos respecto a la dirección de impresión; espesor mínimo; volumen de impresión y unidades; cavidades cerradas.
2. Motor: número de triángulos y materiales; UV presentes y sin solapes; normales coherentes; escala y pivote; niveles de detalle; nombres.
3. Comunes: orientación coherente de las caras (normales hacia fuera), escala y unidades correctas, y ausencia de caras degeneradas.

*Resultado.* **comunes: orientación, escala y unidades, caras degeneradas**; la estanqueidad es imprescindible en impresión y conveniente, aunque no siempre exigida, en un motor.

*Comprobación.* la cara invertida del ejemplo 6.5 afecta a ambos usos: volumen erróneo al imprimir y una cara negra o invisible en el motor.

### Solución del ejercicio 6.2

*Datos.* $\epsilon = (0{,}3; -0{,}2)$; $\hat{\epsilon}_c = (0{,}5; 0{,}1)$; $\hat{\epsilon}_u = (0{,}2; -0{,}1)$; $s = 7{,}5$; $w = 1$; $x = \theta = (0{,}5; 0{,}5)$; tasa $0{,}01$.

*Planteamiento.* $\hat{\epsilon} = \hat{\epsilon}_u + s(\hat{\epsilon}_c - \hat{\epsilon}_u)$; gradiente $w(\hat{\epsilon} - \epsilon)\,\partial x/\partial\theta$ con $\partial x/\partial\theta = I$.

*Desarrollo.*

1. $\hat{\epsilon}_c - \hat{\epsilon}_u = (0{,}3;\ 0{,}2)$; $\hat{\epsilon} = (0{,}2 + 2{,}25;\ -0{,}1 + 1{,}5) = (2{,}45;\ 1{,}4)$.
2. Gradiente: $(2{,}45 - 0{,}3;\ 1{,}4 + 0{,}2) = (2{,}15;\ 1{,}6)$.
3. Paso: $\theta = (0{,}5 - 0{,}0215;\ 0{,}5 - 0{,}016) = (0{,}4785;\ 0{,}484)$.

*Resultado.* $\hat{\epsilon} = (\mathbf{2{,}45};\ \mathbf{1{,}4})$; **gradiente** $(2{,}15;\ 1{,}6)$**;** $\theta = (0{,}4785;\ 0{,}484)$.

*Comprobación.* sin guía ($s = 1$), $\hat{\epsilon} = \hat{\epsilon}_c$ y el gradiente sería $(0{,}2;\ 0{,}3)$: la guía amplifica la dirección que distingue el texto de la imagen incondicional, como describe la tarea.

### Solución del ejercicio 6.3

*Datos.* caras $(0, 1, 2)$, $(0, 2, 3)$, $(0, 3, 1)$, $(1, 3, 2)$, $(1, 2, 4)$.

*Planteamiento.* contar caras por arista no dirigida (definición 6.2).

*Desarrollo.*

1. Las cuatro primeras caras forman un tetraedro estanco con aristas $\{0,1\}$, $\{0,2\}$, $\{0,3\}$, $\{1,2\}$, $\{1,3\}$ y $\{2,3\}$, cada una en dos caras.
2. La quinta cara añade $\{1,2\}$ (que pasa a tener 3 caras), $\{2,4\}$ y $\{1,4\}$ (una cara cada una).
3. Clasificación: 5 interiores, 1 no variedad ($\{1,2\}$) y 2 de borde ($\{2,4\}$, $\{1,4\}$). Total $|E| = 8$.
4. $\chi = 5 - 8 + 5 = 2$.

*Resultado.* $|E| = 8$**: 5 interiores, 1 no variedad, 2 de borde;** $\chi = 2$. Defectos: una «aleta» triangular pegada a una arista del tetraedro, que hace la malla no variedad y abierta.

*Comprobación.* $\chi = 2$ no garantiza nada: una malla defectuosa puede tener la característica de una esfera. Las comprobaciones locales de aristas son imprescindibles.

### Solución del ejercicio 6.4

*Datos.* áreas $(400, 300, 200, 500, 600)$ mm² con $n_z = (-1;\ -0{,}8;\ -0{,}6;\ 0;\ 0{,}5)$; la primera apoyada; umbral $-\cos 45° = -0{,}7071$.

*Planteamiento.* soporte si $n_z < -0{,}7071$ y la cara no está en la bandeja.

*Desarrollo.*

1. Cara de 400 mm²: apoyada, excluida.
2. Cara de 300 mm²: $-0{,}8 < -0{,}7071$, necesita soporte (forma 36,9° con la horizontal).
3. Cara de 200 mm²: $-0{,}6 > -0{,}7071$, se autosoporta (53,1°).
4. Las demás miran a un lado o hacia arriba.
5. Fracción: $300/2000 = 0{,}15$.

*Resultado.* **el 15 % del área necesita soporte**.

*Comprobación.* el ángulo de una cara con la horizontal es el que forma su normal con la vertical descendente, $\arccos(-n_z)$: $\arccos 0{,}8 = 36{,}87° < 45°$ (soporte) y $\arccos 0{,}6 = 53{,}13° \geq 45°$ (autosoportada).

### Solución del ejercicio 6.5

*Datos.* $p = 0{,}2$; $1 - \delta = 0{,}9$; 30 s por generación y 2 s por validación.

*Planteamiento.* proposición 6.4; distribución geométrica para el número esperado.

*Desarrollo.*

1. $N \geq \ln 0{,}1 / \ln 0{,}8 = (-2{,}3026)/(-0{,}22314) = 10{,}32$, luego $N = 11$.
2. $1 - 0{,}8^{11} = 0{,}914$ (con 10, $0{,}893 < 0{,}9$).
3. Tiempo del lote: $11 \cdot (30 + 2) = 352$ s, casi 6 min.
4. De uno en uno, el número de intentos hasta el primer éxito es geométrico, de media $1/p = 5$.

*Resultado.* $N = 11$**; 352 s; 5 intentos de media**.

*Comprobación.* generar en serie y parar en el primer válido cuesta de media $5 \cdot 32 = 160$ s, menos que el lote, pero con una cola larga: la probabilidad de necesitar más de 11 intentos es $0{,}8^{11} = 0{,}086$.

### Solución del ejercicio 6.6

*Datos.* rejilla $128^3$ en $[-1{,}5;\ 1{,}5]^3$; lotes de 65 536; 1 unidad = 50 mm; pared de 0,8 mm.

*Planteamiento.* paso $= 3/127$; número de nodos $128^3$.

*Desarrollo.*

1. Paso: $3/127 = 0{,}02362$ unidades.
2. Evaluaciones: $128^3 = 2\,097\,152$; lotes: $2\,097\,152/65\,536 = 32$.
3. En milímetros: $0{,}02362 \cdot 50 = 1{,}18$ mm.
4. Una pared de 0,8 mm es más fina que el paso: puede caer entre dos nodos sin que ninguno tenga $f < 0$ y desaparecer de la malla.
5. Para que al menos un nodo quede dentro conviene un paso de la mitad del espesor, 0,4 mm: $150/0{,}4 = 375$ intervalos, 376 nodos por eje ($5{,}3 \cdot 10^7$ evaluaciones).

*Resultado.* **paso de** 0,0236 **unidades (**1,18 **mm),** $2{,}1 \cdot 10^6$ **evaluaciones en 32 lotes; la pared de** 0,8 **mm no se resuelve**.

*Comprobación.* es el error (iv) de la observación 1.11.

### Solución del ejercicio 6.7

*Datos.* malla cerrada, conexa, de género $g$; toro de $16 \times 8$ vértices periódicos.

*Planteamiento.* proposición 1.3 y $\chi = 2 - 2g$.

*Desarrollo.*

1. (a) $|E| = \frac{3}{2}|F|$, luego $\chi = |V| - \frac{3}{2}|F| + |F| = |V| - \frac{1}{2}|F| = 2 - 2g$.
2. (b) $|V| = 128$; cuadriláteros $16 \cdot 8 = 128$ (periodicidad: tantos como vértices), luego $|F| = 256$ y $|E| = 384$.
3. $\chi = 128 - 384 + 256 = 0$, de donde $g = 1$.
4. (c) Una arandela es un toro topológico: el validador comprueba que la malla es estanca, conexa y con $|V| - \frac{1}{2}|F| = 0$; si da 2, el agujero se ha cerrado; si da $-2$, hay un asa espuria.

*Resultado.* **(a)** $|V| - \frac{1}{2}|F| = 2 - 2g$**; (b)** $(128, 384, 256)$**,** $\chi = 0$**,** $g = 1$**; (c) exigir** $g = 1$.

*Comprobación.* para el cubo triangulado ($|V| = 8$, $|F| = 12$), $8 - 6 = 2$ y $g = 0$.

### Solución del ejercicio 6.8

*Datos.* generador de soportes a partir de texto; fabricación aditiva.

*Planteamiento.* combinar la sección 6.3, la proposición 6.4 y la observación 6.6.

*Desarrollo.*

1. *Selección* (baratas, automáticas): estanqueidad y orientación (definición 6.2), escala y caja envolvente, volumen positivo, género esperado, fracción de área con voladizo y espesor mínimo por SDF.

2. *Comprobaciones independientes* (no usadas para seleccionar ni ajustar): autointersecciones, análisis de elementos finitos con las cargas del soporte (tensión máxima y flecha), simulación del laminado y revisión humana de una muestra aleatoria.
3. *Reparaciones admitidas*: soldar vértices duplicados, reorientar caras y cerrar agujeros menores que una tolerancia; nunca reparaciones que cambien cotas funcionales sin volver a medirlas.
4. *Política*: generar $N$ candidatos según la proposición 6.4 y elegir el mejor entre los que pasan la selección.
5. *Detección del engaño*: vigilar la tasa de fallos en las comprobaciones independientes entre los candidatos que pasan la selección. Si la tasa de aprobados en la selección sube mientras la de las independientes baja, el generador está explotando el validador (ley de Goodhart, módulo 21, §6.4); se amplía entonces la selección con la comprobación que falla y se renueva el conjunto independiente.

*Resultado.* **un validador en dos niveles (selección e independiente), reparaciones limitadas y seguimiento de la divergencia entre ambos niveles**.

*Comprobación.* el diseño responde a los errores (i), (iii), (iv) y (vi) de la observación 6.6.

# Bibliografía


[1] P. Bright, A. Edelman y S. G. Johnson. *Matrix Calculus (for Machine Learning and Beyond)*. Notas del curso MIT 18.S096, IAP 2023. §6.3 «Reverse-mode "Adjoint" Differentiation» (6.3.1–6.3.3) y cap. 4 «Finite-Difference Approximations». https://ocw.mit.edu/courses/18-s096-matrix-calculus-for-machine-learning-and-beyond-january-iap-2023/

[2] S. Tulsiani y equipo docente. 16-825: Learning for 3D Vision. Carnegie Mellon University (ediciones de 2022 a 2025). Tareas 1 «Rendering Basics with PyTorch3D», 2 «Single View to 3D» (§1 «Exploring loss functions»), 3 «Volume Rendering, NeRF, and 3D Gaussian Splatting» (§1.5 y §5.4) y 4 (§2 «Diffusion-guided Optimization»). https://github.com/learning3d

[3] T. Funkhouser. COS 526: Advanced Computer Graphics. Clase «Mesh Simplification». Princeton University, otoño de 2006. https://www.cs.princeton.edu/courses/archive/fall06/cos526/lectures/simplification.pdf

[4] L. Guibas. CS348n: Neural Representations and Generative Models for 3D Geometry. Stanford University, invierno de 2022 y primavera de 2023. Clases «Classical 3D Geometry Representations», «Generative Models I» (DeepSDF) y «Neural Functions: 3D from 2D Supervision». https://graphics.stanford.edu/courses/cs348n-22-winter/schedule.html

[5] L. Guibas y equipo docente de CS348n. Homework 2: Neural Implicit 3D Representations. Stanford University, primavera de 2023. Problema 1: (a) «Supervised SDF fitting», (b) entrenamiento eikonal, (c) *marching cubes*. https://graphics.stanford.edu/courses/cs348n-23-spring/Handouts/hmwk02.pdf

[6] F. Durand y W. Matusik. 6.837: Computer Graphics. MIT OpenCourseWare, otoño de 2012. Clases «Graphics Pipeline and Rasterization» I y II. https://ocw.mit.edu/courses/6-837-computer-graphics-fall-2012/

[7] S. G. Johnson. Adjoint methods and sensitivity analysis for recurrence relations. Notas para MIT 18.335J «Introduction to Numerical Methods», otoño de 2010 (fechadas en 2007). https://ocw.mit.edu/courses/18-335j-introduction-to-numerical-methods-fall-2010/

[8] R. Ramamoorthi. CSE 190: Advanced Computer Graphics. Clase 8 «Mesh simplification». University of California, San Diego, primavera de 2015. https://cseweb.ucsd.edu/~ravir/190/2015/lectures/190-lecture8.pdf

[9] A. Tovar. ME 60602: Topology Optimization. Programa del curso. Indiana University–Purdue University Indianapolis. Clases 8 «Sensitivity analysis», 10 «Duality and the optimality criteria method» y 12 «Well-posedness, numerical problems, filters». https://edrl.sitehost.iu.edu/courses/TopologyOptimization_Syllabus.pdf

[10] M. P. Bendsøe y O. Sigmund. *Topology Optimization: Theory, Methods, and Applications*, 2.ª ed. Springer, Berlín, 2004. Cap. 1 «Topology optimization by distribution of isotropic material».

[11] F. P. Incropera, D. P. DeWitt, T. L. Bergman y A. S. Lavine. *Fundamentals of Heat and Mass Transfer*. Wiley. Cap. 3 «One-Dimensional, Steady-State Conduction» (superficies extendidas).

[12] M. J. Kochenderfer y T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. Caps. «Sampling Plans», «Surrogate Models», «Probabilistic Surrogate Models» y «Surrogate Optimization».

[13] E. Andreassen, A. Clausen, M. Schevenels, B. S. Lazarov y O. Sigmund. Efficient topology optimization in MATLAB using 88 lines of code. *Structural and Multidisciplinary Optimization*, 43(1):1–16, 2011. https://www.topopt.mek.dtu.dk/apps-and-software/efficient-topology-optimization-in-matlab

[14] M. P. Bendsøe. Optimal shape design as a material distribution problem. *Structural Optimization*, 1(4):193–202, 1989.

[15] B. Bourdin. Filters in topology optimization. *International Journal for Numerical Methods in Engineering*, 50:2143–2158, 2001.

[16] Y. Hu, L. Anderson, T.-M. Li, Q. Sun, N. Carr, J. Ragan-Kelley y F. Durand. DiffTaichi: Differentiable programming for physical simulation. En *International Conference on Learning Representations (ICLR)*, 2020. https://arxiv.org/abs/1910.00935

[17] M. Garland y P. S. Heckbert. Surface simplification using quadric error metrics. En *Proceedings of SIGGRAPH 97*, ACM, 1997.

[18] H. Jun y A. Nichol. Shap-E: Generating conditional 3D implicit functions. Prepublicación, 2023. https://arxiv.org/abs/2305.02463

[19] B. Karis, R. Stubbe y G. Wihlidal. Nanite: A deep dive. Curso «Advances in Real-Time Rendering in Games», SIGGRAPH 2021. https://advances.realtimerendering.com/s2021/Karis_Nanite_SIGGRAPH_Advances_2021_final.pdf

[20] B. Kerbl, G. Kopanas, T. Leimkühler y G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), 2023. https://arxiv.org/abs/2308.04079

[21] S. Laine, J. Hellsten, T. Karras, Y. Seol, J. Lehtinen y T. Aila. Modular primitives for high-performance differentiable rendering. *ACM Transactions on Graphics*, 39(6), 2020. https://github.com/NVlabs/nvdiffrast

[22] M. Langelaar. An additive manufacturing filter for topology optimization of print-ready designs. *Structural and Multidisciplinary Optimization*, 2017. https://doi.org/10.1007/s00158-016-1522-2

[23] A. Laurentini. The visual hull concept for silhouette-based image understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(2), 1994.

[24] B. Lévy, S. Petitjean, N. Ray y J. Maillot. Least squares conformal maps for automatic texture atlas generation. En *SIGGRAPH 2002*, ACM, 2002.

[25] Z. Li, N. B. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart y A. Anandkumar. Fourier neural operator for parametric partial differential equations. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.08895

[26] S. Liu, T. Li, W. Chen y H. Li. Soft Rasterizer: A differentiable renderer for image-based 3D reasoning. En *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. https://arxiv.org/abs/1904.01786

[27] W. E. Lorensen y H. E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. En *Proceedings of SIGGRAPH '87*, ACM, 1987, págs. 163–169.

[28] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi y R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. En *European Conference on Computer Vision (ECCV)*, 2020. https://arxiv.org/abs/2003.08934

[29] T. Müller, A. Evans, C. Schied y A. Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics* (SIGGRAPH), 2022. https://arxiv.org/abs/2201.05989

[30] J. J. Park, P. Florence, J. Straub, R. Newcombe y S. Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. En *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. https://arxiv.org/abs/1901.05103

[31] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez y P. W. Battaglia. Learning mesh-based simulation with graph networks. En *International Conference on Learning Representations (ICLR)*, 2021. https://arxiv.org/abs/2010.03409

[32] B. Poole, A. Jain, J. T. Barron y B. Mildenhall. DreamFusion: Text-to-3D using 2D diffusion. En *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2209.14988

[33] M. Raissi, P. Perdikaris y G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[34] N. Ravi, J. Reizenstein, D. Novotny, T. Gordon, W.-Y. Lo, J. Johnson y G. Gkioxari. Accelerating 3D deep learning with PyTorch3D. Prepublicación, 2020. https://arxiv.org/abs/2007.08501

[35] Y. Siddiqui, A. Alliegro, A. Artemov, T. Tommasi, D. Sirigatti, V. Rosov, A. Dai y M. Nießner. MeshGPT: Generating triangle meshes with decoder-only transformers. En *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. https://arxiv.org/abs/2311.15475

[36] O. Sigmund. A 99 line topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization*, 21(2):120–127, 2001.

[37] O. Sigmund y J. Petersson. Numerical instabilities in topology optimization: A survey on procedures dealing with checkerboards, mesh-dependencies and local minima. *Structural Optimization*, 16:68–75, 1998.

[38] Autodesk. Driving a lighter, more efficient future of automotive part design at GM. Artículo de Autodesk Design & Make, 1 de agosto de 2018. https://www.autodesk.com/design-make/articles/automotive-design

[39] Autodesk. Manufacturing methods in the Generative Design workspace. Ayuda de Autodesk Fusion. https://help.autodesk.com/cloudhelp/ENU/Fusion-GenerativeDesign/files/GD-MFG-METHODS.htm

[40] Blender Foundation. Blender Python API: `bpy.types.DecimateModifier`. https://docs.blender.org/api/3.4/bpy.types.DecimateModifier.html

[41] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne y Q. Zhang. JAX: composable transformations of Python+NumPy programs. Software, 2018. https://github.com/jax-ml/jax

[42] W. Jakob, S. Speierer, N. Roussel, M. Nimier-David, D. Vicini, T. Zeltner, B. Nicolet, M. Crespo, V. Leroy y Z. Zhang. Mitsuba 3 renderer. Software, 2022. https://github.com/mitsuba-renderer/mitsuba3

[43] A. Kapoulkine. meshoptimizer: mesh optimization library. Software (licencia MIT). https://github.com/zeux/meshoptimizer

[44] Microsoft. Texture block compression in Direct3D 11. Documentación de Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/direct3d11/texture-block-compression-in-direct3d-11

[45] nTop. Implicit modeling for mechanical design. Blog de nTop. https://www.ntop.com/resources/blog/implicit-modeling-for-mechanical-design/

[46] NVIDIA. NVIDIA PhysicsNeMo (antes Modulus). https://developer.nvidia.com/physicsnemo

[47] NVIDIA. Warp: a Python framework for GPU-accelerated simulation, robotics, and machine learning. Software (licencia Apache-2.0). https://github.com/NVIDIA/warp